

Machine Learning

Francesca Claudia Riccaboni, Elisa Stabilini

A.A. 2021-2021

Nota

La dispensa, originariamente scritta dalla dott. Francesca Claudia Riccaboni è stata aggiornata, a seguito delle lezioni tenutesi durante l'anno accademico 2023 - 2024 da Elisa Stabilini. Ringraziamo Francesca per aver pensato di scrivere una dispensa per il corso così ben strutturata e per averci messo a disposizione il suo lavoro, consentendoci di integrare quanto da lei già scritto con ulteriore materiale. In particolare sono state fatte una revisione ed integrazione completa per tutti i capitoli eccetto i capitoli 2 e 4 il cui materiale è stato già organizzato in appunti in lateX dal docente, pertanto i capitoli 2 e 4 sono presenti nella loro stesura originale.

Elisa Stabilini

Indice

1	Cos'è il Machine Learning	4
2	Metodi di Regressione Lineare	10
2.1	Richiamo di Nozioni Statistiche	25
2.1.1	Probabilità	29
2.2	Approfondimento Regressioni	29
3	Selezione e Valutazione dei Modelli	39
3.1	Importanza dell'errore	39
3.2	Tradeoff Bias Varianza	41
3.3	Decomposizione Bias-Varianza per Regressione Lineare	43
3.3.1	Caso del metodo dei prossimi vicini	45
3.4	Metodi di Ricampionamento	47
3.4.1	Cross-validation	47
3.4.2	Bootstrap	50
3.4.3	Jackknife	51
4	Metodi di Classificazione Lineare	53
4.1	Regressione Lineare e Matrice di Indice	53
4.2	Analisi del Discriminante	56
4.2.1	Discriminante Lineare	56
4.2.2	Discriminante Quadratico	58
4.3	Regressione Logistica	60
4.3.1	Esempio	63
5	Discesa del Gradiente e Metodi di Minimizzazione	66
5.1	Problemi del Metodo di Newton-Raphson	69
5.2	Metodi Iterativi	72
5.2.1	Esempio regressione lineare	73
5.3	Metodi Stocastici	74
6	Reti Neurali	82
6.1	Percettroni	83
6.1.1	Singolo Percettrone	83
6.1.2	Funzioni di attivazione	87
6.2	Teorema di approssimazione universale	90
6.3	Feed Forward Neural Network	92
6.4	Back-Propagation Algorithm	94
6.4.1	Dimostrazione	94

6.5	Convolutional Neural Network	98
6.5.1	Reti Neurali Standard vs. Reti Neurali Convoluzionali	98
6.5.2	Convoluzione	99
6.6	Recurrent Neural Networks	103
6.7	Equazioni Differenziali Ordinarie	107
6.7.1	Note sull'approccio	107
6.7.2	Costruzione di una rete per la soluzione di ODE	107
6.7.3	Equazioni alle Derivate Parziali	109
7	Riduzione di Dimensionalità e Analisi della Componenti Principali (PCA)	110
7.1	Curse of Dimensionality	110
7.2	Principal Components Analysis	111
7.2.1	Teorema PCA	114
8	Alberi Decisionali e Random Forest	122
8.1	Algoritmo CART	122
8.1.1	Problemi di regressione	124
8.1.2	Classificazione	125
8.1.3	Pruning	127
8.1.4	Instabilità	128
8.2	Random Forests	128
8.2.1	Voting Classifiers	128
8.2.2	Bagging and Pasting	129
9	Support Vector Machines	131
9.0.1	Condizioni Karush-Kuhn-Tucker	131
9.1	Support Vector Machines	132
9.1.1	Caviats	133
9.2	Kernels di Mercer e SVMs	136
9.3	Esempio di utilizzo di SVMs	137

Capitolo 1

Cos'è il Machine Learning

Il *machine learning* non è un argomento a sé stante, è una serie di tecniche computazionali e statistiche che si usano per trattare i dati. Si è sviluppato negli ultimi trent'anni, e in diversi ambiti, come reti neurali, ecc... L'inizio della materia è legata allo sviluppo dell'intelligenza artificiale, da cui oggi siamo ancora molto lontani. Il senso base del *machine learning* è che il computer impari autonomamente informazioni dai dati, senza bisogno di una persona che li rielabori. Senza dare una programmazione esplicita, il computer riesce a riconoscere dei pattern e dei comportamenti autonomamente.

Il campo si è sviluppato attraverso gli anni, anche grazie ad alcune librerie (Scikit-learn, Tensorflow, PyTorch e Keras). Altre applicazioni sono ad esempio le macchine che si guidano da sole, l'estrazione di informazioni dai social network per lo sviluppo di pubblicità più efficaci, piuttosto che la rielaborazione dei dati degli acceleratori di particelle del CERN, o la risoluzione dell'equazione di molti corpi usando reti neurali. Ciò che si fa è partire dai dati, sviluppare un modello, ovvero una funzione inizialmente nota (o non nota) che si cerca poi di approssimare al meglio. Il campo è molto ricco ed ha applicazioni di tutti i tipi.

Si possono classificare le tecniche di *machine learning* in tre macro-categorie:

- Apprendimento Automatico Supervisionato o *supervised learning*: si hanno già dei dati di campionamenti in cui si hanno variabili note come input e si ha l'output. Bisogna trovare una funzione che approssima i dati campionati al meglio possibile. Non esiste sempre una funzione esatta, bisogna cercare di valutare quale sia il modello migliore e più sofisticato possibile, con la migliore probabilità di essere quello corretto.
- Apprendimento Automatico Non Supervisionato o *unsupervised learning*: non si conosce l'output, non si ha un valore di risposta agli input, non si ha una funzione che assegna ad ogni dato campionato il suo output. Dai dati si cerca di estrarre proprietà comuni, non si ha una conoscenza a priori della relazione tra i vari dati.
- Addestramento Rinforzato o *reinforcement learning*: si danno al computer delle task, delle cose da fare ed ogni volta che il computer dà la risposta giusta si dà un premio al modello, oppure questo viene cambiato; la conoscenza si acquista con un metodo del tipo *trial-and-error*.

Esistono anche altri modi per classificare i tipi di *machine learning*. Possiamo ad esempio, avere valori di output discreti oppure continui: se si misura la temperatura

esterna notturna ed in base alla quantità di acqua che è piovuta, si cerca di dedurre la temperatura a mezzogiorno, si cerca un algoritmo che indovini la temperatura. La variabile è continua e si parla di problemi di regressione.

Ci sono alcuni casi in cui la funzione di output è discreta. Se un paziente ha un infarto e si vuole sapere se tra 5 anni lo avrà di nuovo, la risposta è sì oppure no. Oppure se si riescono a riconoscere delle lettere scritte a mano, l'uscita è discreta. In questo caso si parla di *classificazione*.

Un'altra classe di problemi del *machine learning* è costituita dai problemi di *clustering*, in cui si dividono i dati in gruppi, a priori non noti, da cui si estraggono caratteristiche (*features*) comuni (è quindi una forma di *unsupervised learning*).

Ci sono altri elementi essenziali che sono generali, sono comuni a tutti i problemi di *machine learning*. Il primo ingrediente è la presenza di un set di dati: può essere diviso in diversi modi, possiamo prenderne una percentuale per fare una cosa e una percentuale per farne un'altra, come testare, fittare, ecc... Nelle applicazioni è molto importante e difficile decidere quali dati utilizzare come dati di training, valutazione, test, ecc...

Normalmente si ha un modello che è una funzione di alcuni parametri. Si passa dal modello più banale di un fit lineare, da cui si estraggono i parametri della linea, fino a cose più complicate, come la creazione di una rete neurale. La funzione è parametrizzata con un set di dati che vanno fittati ed adattati durante l'apprendimento in modo da arrivare al risultato finale.

Si ha poi la funzione di costo, o di perdita, ovvero il confronto del modello con i dati veri, che si minimizza quando il modello è molto efficiente ed è grande quando il modello non lo è.

Come si riconosce un buon modello? Abbiamo un set di punti e vogliamo estrarne un modello. Possiamo fare fit di ogni tipo, dall'approssimazione lineare a funzioni più complesse. Bisogna saper scegliere il modello, senza incappare in un over-fitting. Trovare il modello corretto è una cosa abbastanza elusiva. Se si parte da un set di dati di cui non si conosce la fisica dietro, è quasi impossibile trovare una funzione corretta, perché ci sono un'infinità di parametri che si dovrebbero considerare. Bisogna cercare una serie di funzioni che si basano al meglio sul nostro sistema. Per esempio, consideriamo un modello dei temporali per valutare la velocità delle gocce di pioggia; sappiamo che esse raggiungeranno una certa velocità e poi si fermeranno per la velocità limite, ma poi tutte gli altri parametri ci sono sconosciuti. Cerchiamo una serie di modernizzazioni che si comportano al meglio possibile.

Facciamo di seguito una carrellata di esempi di applicazioni di tecniche di *machine learning*

Mail SPAM

Si può cercare di capire lo spam detection nelle mail: è un modello che non è mai perfetto, ci sono spesso errori. Nella tabella di figura 1.1 sono riportate le frequenze di alcune parole o lettere che rientrano in un messaggio spam, da cui si possono ricavare dei modelli per decidere se la mail è spazzatura oppure no. Nell'implementazione in figura 1.2 è molto facile capire che il sistema è soggetto a moltissimi errori. Quello che si può fare è scrivere un programma che fa imparare al computer man mano che si leggono le mail e che si cestinano, cosa le caratterizzi.

Average percentage of words or characters in an email message equal to the indicated word or character. We have chosen the words and characters showing the largest difference between spam and email.

	george	you	your	hp	free	hpl	!	our	re	edu	remove
spam	0.00	2.26	1.38	0.02	0.52	0.01	0.51	0.51	0.13	0.01	0.28
email	1.27	1.27	0.44	0.90	0.07	0.43	0.11	0.18	0.42	0.29	0.01

Figura 1.1: Esempio di decision tree per SPAM detection

```

if (%george < 0.6) & (%you > 1.5)  then spam
                                     else email.

```

Another form of a rule might be:

```

if (0.2 · %you − 0.3 · %george) > 0  then spam
                                       else email.

```

Figura 1.2: Tabella con frequenza di presenza di determinati caratteri

Tumore alla Prostata

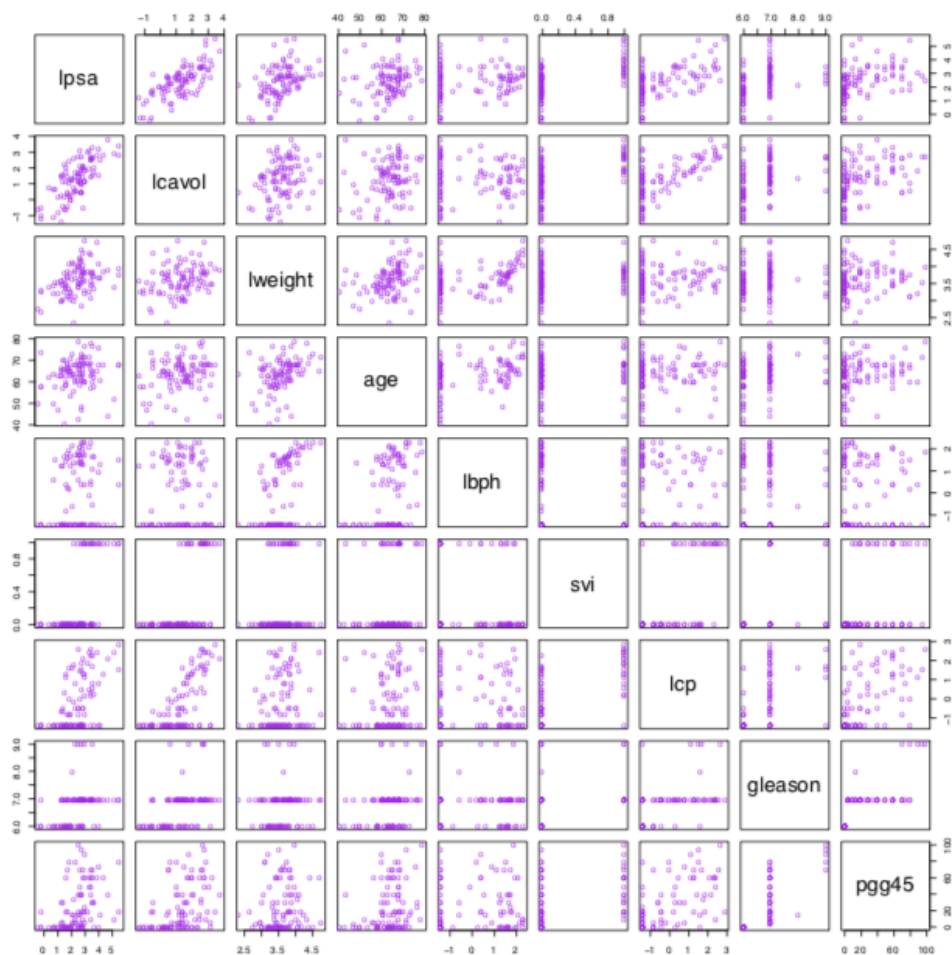
Parliamo di uno studio con l'uso di *scatter-plot*. Abbiamo una serie di pazienti con tumore alla prostata con vari parametri, come età, peso, ecc... e se ne cercano correlazioni. Quando arriva un paziente con un set di dati distribuiti in un certo modo, allora la probabilità che abbia il tumore sarà più o meno alta, oppure può essere molto interessante per capire il funzionamento del tumore.

CAP

Si parla di codici postali. Le cifre da 0 a 9 scritte da studenti delle elementari e si cerca di creare un programma che riesca a riconoscerle in maniera abbastanza veloce. Si prende una foto del CAP, le cifre vengono estratte e normalizzate all'interno di un quadrato e si prende un *pattern recognition* in cui si insegna al computer come distinguere i decimali.

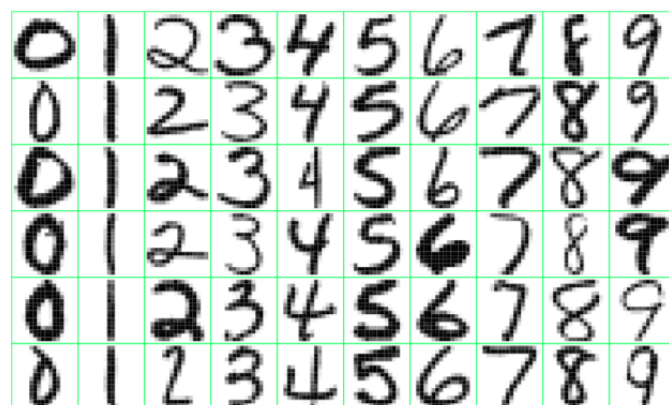
Tumori e DNA

Sequenze del DNA: si estraggono RNA marcatori di geni differenti. In ogni punto della lastra viene inserito un gene di RNA particolare nelle righe, mentre nelle colonne è inserito il DNA di un set di diversi pazienti, sani o malati di qualche cosa, e si confrontano le capacità di reazione (gene attivo o non attivo, da verde a rosso). Se si riescono ad estrarre delle correlazioni si può imparare qualcosa.



----- Scatterplot matrix of the prostate cancer data. The first row shows the response against each of the predictors in turn. Two of the predictors, svi and gleason, are categorical.

Figura 1.3: Esempio di studio di correlazione per lo studio delle features nel caso di dati di pazienti con tumore alla prostata



Examples of handwritten digits from U.S. postal envelopes.

Figura 1.4: Esempio di parte di dataset per il riconoscimento di cifre all'interno di un'immagine



DNA microarray data: expression matrix of 6830 genes (rows) and 64 samples (columns), for the human tumor data. Only a random sample of 100 rows are shown. The display is a heat map, ranging from bright green (negative, under expressed) to bright red (positive, over expressed). Missing values are gray. The rows and columns are displayed in a randomly chosen order.

Figure 1.5

Capitolo 2

Metodi di Regressione Lineare

La regressione lineare è una situazione in cui si ha una serie di variabili di input con una variabile di output e si cerca di approssimare linearmente la funzione tra input e output. La regressione lineare è interessante per un certo numero di motivi: è una buona scelta per fittare funzioni continue, anche se non sono necessariamente lineari: infatti possiamo esprimere una funzione di una variabile non lineare come una funzione lineare delle varie potenze. È il metodo che di solito funziona meglio. È anche pedagogico, è utile per capire cose più avanzate come reti neurali e *Support Vector Machines*. Tra le altre cose, si può estrarre un'espressione analitica con la quale si riesce a lavorare matematicamente.

Abbiamo una variabile y che viene chiamata il dipendente, l'*outcome*, la risposta. Abbiamo un vettore $\underline{x}$ di variabili di ingresso. La funzione non è nota e probabilmente non è possibile estrarla dalla teoria; si vuole cercare di capire come si fa a fare un'estrapolazione. Immaginiamo di avere un set di n dati ed un set di variabili di risposta $y = \{y_i\}$ ad un set di punti $\underline{x}_i$. Vogliamo cercare la funzione che soddisfi $f(\underline{x}) = y$ che approssimi al meglio le variabili y ; non deve essere una funzione necessariamente lineare. Ci sono mille modelli che fittano i dati, ma se vogliamo predire cosa succederà in un punto nuovo, possiamo trovare la probabilità di avere un valore y piuttosto che un altro, che è una probabilità condizionata $p(y|\underline{x})$. La probabilità condizionata è una funzione di y per ogni valore fissato di $\underline{x}$ che mostra quale probabilità abbiamo che l'output sia y , normalizzata dunque a 1. Per ogni input abbiamo una distribuzione di probabilità che ci indica quale valore di output sia più probabile.

Nella regressione, non necessariamente lineare, si parla di p caratteristiche, che sono il numero di punti di input, con n campioni rappresentati da una matrice X , chiamata matrice di design (o di Vandermonde):

$$\mathbf{X} = \begin{bmatrix} 1 & x_0^1 & x_0^2 & \cdots & x_0^{p-1} \\ 1 & x_1^1 & x_1^2 & \cdots & x_1^{p-1} \\ 1 & x_2^1 & x_2^2 & \cdots & x_2^{p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1}^1 & x_{n-1}^2 & \cdots & x_{n-1}^{p-1} \end{bmatrix}$$

Per ogni riga di $\mathbf{X}$ c'è un valore di y , con una relazione $y_i = f(\mathbf{X}_{i,*})$. Si parla di un modello di regressione lineare basato su p parametri β_i . Si approssima

$$y_i = f(\mathbf{X}_{i,*}) = \beta^T \mathbf{X}_{i,*} + \varepsilon_i$$

Esempio 1

Approssimazione del modello a goccia liquido, che si basa sull'equazione di Bethe-Weizsacher, uno dei modelli più semplici con cui si può approssimare l'energia di legame di un nucleo:

$$BE(A) = a_0 + a_1A + a_2A^{2/3} + a_3A^{-1/3} + a_4A^{-1}.$$

L'input è il numero di nucleoni A , da cui abbiamo 5 predittori, che sono le diverse potenze di A , $p = 0, 1, 2, 3, 4$ e la nostra matrice X sarà di dimensione $p \times n$, dove n è il numero di campioni, che è il numero di nuclei possibili.

Esempio 2

C'è stato un *data leak* nel 2005, per cui sono stati pubblicati 24 dati delle carte di credito di 30000 persone in Taiwan. Con dati di questo tipo si possono fare regressioni lineari ed imparare informazioni sul comportamento delle persone. Le carte di credito di Taiwan, una volta trapelati si riescono a fare emergere moltissime caratteristiche personali ed economiche.

In generale, se abbiamo un set di n variabili x_i con output $y_i = f(x_i)$, possiamo cercare di fare previsioni sul valore y in un punto non presente nei dati. L'approccio più semplice è parametrizzare la funzione secondo un polinomio di grado $p - 1$:

$$y = y(x) \rightarrow y(x_i) = \tilde{y}_i + \varepsilon_i = \sum_{j=0}^{p-1} \beta_j x_i^j + \varepsilon_i$$

Si può riscrivere il problema come

$$\begin{aligned} y_0 &= \beta_0 + \beta_1 x_0^1 + \beta_2 x_0^2 + \cdots + \beta_{p-1} x_0^{p-1} + \varepsilon_0 \\ y_1 &= \beta_0 + \beta_1 x_1^1 + \beta_2 x_1^2 + \cdots + \beta_{p-1} x_1^{p-1} + \varepsilon_1 \\ y_2 &= \beta_0 + \beta_1 x_2^1 + \beta_2 x_2^2 + \cdots + \beta_{p-1} x_2^{p-1} + \varepsilon_2 \\ &\vdots \\ y_{n-1} &= \beta_0 + \beta_1 x_{n-1}^1 + \beta_2 x_{n-1}^2 + \cdots + \beta_{p-1} x_{n-1}^{p-1} + \varepsilon_{n-1} \end{aligned}$$

che ha una matrice di design scritta come in precedenza. Possiamo scrivere la soluzione come

$$y = \mathbf{X}\beta + \varepsilon$$

che è l'esempio di come un polinomio possa essere riportato ad un'equazione lineare.

Altrimenti si può fare una trasformata di Fourier, per cui le potenze x_i^j vengono sostituite da $\cos(jx_i)$ o da $y = \sin(jx_i)$. In questo caso avremmo

$$\begin{aligned} y_0 &= \beta_0 x_{00} + \beta_1 x_{00} + \beta_2 x_{01} + \cdots + \beta_{n-1} x_{0,n-1} + \varepsilon_0 \\ y_1 &= \beta_0 x_{10} + \beta_1 x_{11} + \beta_2 x_{12} + \cdots + \beta_{n-1} x_{1,n-1} + \varepsilon_1 \\ y_2 &= \beta_0 x_{20} + \beta_1 x_{21} + \beta_2 x_{22} + \cdots + \beta_{n-1} x_{2,n-1} + \varepsilon_2 \\ &\vdots \\ y_i &= \beta_0 x_{i0} + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{n-1} x_{i,n-1} + \varepsilon_i \\ &\vdots \\ y_{n-1} &= \beta_0 x_{n-1,0} + \beta_1 x_{n-1,1} + \beta_2 x_{n-1,2} + \cdots + \beta_{n-1} x_{n-1,n-1} + \varepsilon_{n-1} \end{aligned}$$

in cui $p = n$ e si ha una matrice di design quadrata. In generale è più frequente avere un numero di dati n maggiore dei parametri p .

La funzione trovata darà degli errori ε_i . Quello che vogliamo fare è minimizzarne il valore.

Possiamo scrivere in maniera compatta gli $n - 1$ risultati dell'approssimazione

$$\tilde{y} = \mathbf{X}\beta$$

cercando di ridurre la differenza tra $\tilde{y}$ e le y misurate. Possiamo scrivere la varianza degli errori come

$$C(\beta) = \frac{1}{N} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \frac{1}{n} \{(y - \tilde{y})^T (y - \tilde{y})\}$$

assumendo che le variabili siano reali (abbiamo la trasposta e non l'aggiunto). Possiamo riscrivere C , la funzione di costo, come

$$C(\beta) = \frac{1}{N} \{(y - X\beta)^T (y - X\beta)\}$$

che descrive quanto male va il modello rispetto ai dati sui quali viene aggiustato, è necessario cambiare i parametri per minimizzare C . La funzione C si può anche definire come

$$C(\beta) = \frac{1}{2n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2$$

La funzione è connessa con la varianza ed in realtà è proprio quella, infatti la differenza tra $\tilde{y}$ e y è determinata unicamente (se il modello è buono) da errori stocastici. Il valore $\tilde{y}_i$ può essere interpretato come una media dei valori sui quali si fa il campionamento

$$\tilde{y}_i = \langle y_i \rangle = \beta_0 x_{i0} + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{n-1} x_{i,p-1}$$

cerchiamo di minimizzare C

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \{(y - X\beta)^T (y - X\beta)\}$$

Prendiamo la derivata

$$\frac{\partial C(\beta)}{\partial \beta_j} = \frac{\partial}{\partial \beta_j} \left[\frac{1}{n} \sum_{i=0}^{n-1} (y_i - \beta_0 x_{i0} - \beta_1 x_{i1} - \beta_2 x_{i2} - \cdots - \beta_{n-1} x_{i,n-1})^2 \right] = 0$$

da cui $\forall j$

$$\frac{\partial C(\beta)}{\partial \beta_j} = -\frac{2}{n} \left[\sum_{i=0}^{n-1} x_{ij} (y_i - \beta_0 x_{i0} - \beta_1 x_{i1} - \beta_2 x_{i2} - \cdots - \beta_{n-1} x_{i,n-1}) \right] = 0$$

e sostanzialmente:

$$\frac{\partial C(\beta)}{\partial \beta} = 0 = X^T (y - X\beta).$$

Ci sono alcune relazioni utili sulla variazione di matrici e vettori da tenere a mente (dove $|A|$ è il determinante della matrice):

$$\frac{\partial (b^T a)}{\partial a} = b, \quad \frac{\partial \text{Tr}(BA)}{\partial A} = B^T, \quad \frac{\partial (a^T A a)}{\partial a} = (A + A^T)a, \quad \frac{\partial \log |A|}{\partial A} = (A^{-1})^T.$$

Partendo dall'annullamento della derivata possiamo scrivere

$$X^T y = X^T X \beta$$

e se la matrice $X^T X$ è invertibile, abbiamo la soluzione

$$\beta = (X^T X)^{-1} X^T y.$$

Generalmente quando si hanno casi pratici abbiamo dei modelli con un numero limitato di parametri e la matrice β non è necessariamente grande (in Bethe-Weizsacker c'è $p = 5$) e tipicamente $p \ll n$, quindi fare l'inversione ed estrarre β non è necessariamente complicato. Normalmente si può fare l'inversione con l'algoritmo LU, QR o altri metodi di algebra lineare normalmente già implementati nei computer.

L'errore ε è

$$\varepsilon = y - \tilde{y} = y - X\beta$$

e la condizione di minimizzazione non dice altro che

$$X^T \varepsilon = X^T (y - X\beta) = 0$$

il vettore β è scelto in modo da minimizzare i residui.

Spesso quando c'è un set di dati, alcuni sono ottenuti con una certa precisione ed altri con precisioni minori. Se si fanno determinati campionamenti c'è sempre un errore sperimentale. Se si cerca un modello bisogna considerare che alcuni punti sono più importanti di altri. In genere si divide gli scarti al quadrati per la varianza e si ha il chi quadro:

$$\chi^2(\beta) = \frac{1}{n} \sum_{i=0}^{n-1} \frac{(y_i - \tilde{y}_i)^2}{\sigma_i^2} = \frac{1}{n} \left\{ (y - \tilde{y})^T \frac{1}{\Sigma^2} (y - \tilde{y}) \right\},$$

dove Σ è la matrice diagonale dei σ_i .

Quello che dobbiamo fare è ancora minimizzare la funzione, ma nella derivazione i predittori sono divisi per la deviazione standard

$$\begin{aligned} \frac{\partial \chi^2(\beta)}{\partial \beta_j} &= \frac{\partial}{\partial \beta_j} \left[\frac{1}{n} \sum_{i=0}^{n-1} \left(\frac{y_i - \beta_0 x_{i0} - \beta_1 x_{i1} - \beta_2 x_{i2} - \dots - \beta_{n-1} x_{i,n-1}}{\sigma_i} \right)^2 \right] \\ &= -\frac{2}{n} \left[\sum_{i=0}^{n-1} \frac{x_{ij}}{\sigma_i} \left(\frac{y_i - \beta_0 x_{i0} - \beta_1 x_{i1} - \beta_2 x_{i2} - \dots - \beta_{n-1} x_{i,n-1}}{\sigma_i} \right) \right] = 0 \end{aligned}$$

che si sintetizza in

$$\frac{\partial \chi^2(\beta)}{\partial \beta} = 0 = A^T (b - A\beta)$$

dove $A = X/\Sigma$ con elementi di matrice $a_{ij} = x_{ij}/\sigma_i$ e dove $b_i = y_i/\sigma_i$. Analogamente a prima otteniamo

$$A^T b = A^T A \beta,$$

la cui soluzione è

$$\beta = (A^T A)^{-1} A^T b.$$

Se le varianze fossero tutte uguali, nella soluzione di β si cancellano e si ottiene la soluzione ottenuta prima senza considerare gli errori.

Introduciamo la matrice

$$H = (A^T A)^{-1}$$

ed esprimiamo i parametri β_j :

$$\beta_j = \sum_{k=0}^{p-1} h_{jk} \sum_{i=0}^{n-1} \frac{y_i}{\sigma_i} \frac{x_{ik}}{\sigma_i} = \sum_{k=0}^{p-1} h_{jk} \sum_{i=0}^{n-1} b_i a_{ik}.$$

Se calcoliamo l'incertezza sui parametri β :

$$\sigma^2(\beta_j) = \sum_{i=0}^{n-1} \sigma_i^2 \left(\frac{\partial \beta_j}{\partial y_i} \right)^2,$$

che equivale a

$$\sigma^2(\beta_j) = \left(\sum_{k=0}^{p-1} h_{jk} \sum_{i=0}^{n-1} a_{ik} \right) \left(\sum_{l=0}^{p-1} h_{jl} \sum_{m=0}^{n-1} a_{ml} \right) = h_{jj};$$

gli elementi diagonali della matrice H mi danno l'errore su β_j .

Un caso particolare che si ha quando il sistema è abbastanza semplice, per cui per esempio c'è solo una variabile y di uscita che dipende da una variabile x ed il sistema si comporta più o meno in forma lineare: possiamo fittare con una retta:

$$y = y(x) \rightarrow y(x_i) \approx \beta_0 + \beta_1 x_i$$

abbiamo due predittori, al matrice di design ha due colonne e le potenze sono 0 e 1. Possiamo calcolare, quindi:

$$\frac{\partial \chi^2}{\partial \beta_0} = -\frac{2}{n} \left[\sum_{i=0}^{n-1} \left(\frac{y_i - \beta_0 - \beta_1 x_i}{\sigma_i^2} \right) \right] = 0$$

e

$$\frac{\partial \chi^2}{\partial \beta_1} = -\frac{2}{n} \left[\sum_{i=0}^{n-1} x_i \left(\frac{y_i - \beta_0 - \beta_1 x_i}{\sigma_i^2} \right) \right] = 0$$

Possiamo definire

$$\begin{aligned} \gamma &= \sum_{i=0}^{n-1} \frac{1}{\sigma_i^2} & \gamma_x &= \sum_{i=0}^{n-1} \frac{x_i}{\sigma_i^2} & \gamma_y &= \sum_{i=0}^{n-1} \frac{y_i}{\sigma_i^2} \\ \gamma_{xx} &= \sum_{i=0}^{n-1} \frac{x_i x_i}{\sigma_i^2} & \gamma_{yy} &= \sum_{i=0}^{n-1} \frac{y_i y_i}{\sigma_i^2} & \gamma_{xy} &= \sum_{i=0}^{n-1} \frac{x_i y_i}{\sigma_i^2} \end{aligned}$$

e possiamo inserirli nelle equazioni:

$$\begin{aligned} \beta_0 &= \frac{\gamma_{xx} \gamma_y - \gamma_x \gamma_y}{\gamma \gamma_{xx} - \gamma_x^2} \\ \beta_1 &= \frac{\gamma_{xy} \gamma - \gamma_x \gamma_y}{\gamma \gamma_{xx} - \gamma_x^2} \end{aligned}$$

Quando non è possibile invertire la matrice (il denominatore è pari a 0) non è possibile utilizzare questo metodo, ma si possono utilizzare altri approcci, come la *Singular Value Decomposition* (SVD). Si può in qualche modo evitare il problema con metodi di regressione *Ridge* e *Lasso*.

Qualunque sia la matrice si può utilizzare l'algoritmo SVD. La soluzione del problema lineare esiste sempre, ci sarà sempre un β che minimizza gli errori, anche se non è unico.

La matrice X non è necessariamente quadrata, perché come numero di colonne ha il numero di predittori p e il numero di righe è il numero di dati n . Mostriamo qualche esempio. La matrice

$$X = \begin{bmatrix} 1 & -1 & 2 \\ 1 & 0 & 1 \\ 1 & 2 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

non è invertibile, poiché ha colonne linearmente indipendenti e così anche la matrice

$$X = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}.$$

Ci si può trovare in una situazione in cui i dati con gli errori formano una matrice non del tutto invertibile, che tende ad essere singolare.

Possiamo provare a rattoppare la matrice non invertibile aggiungendo un qualcosa sulla diagonale (Ridge e Lasso):

$$X^T X \rightarrow X^T X + \lambda I$$

e in tal modo (con λ più piccolo possibile) rendiamo invertibile la matrice.

Oppure, un approccio utile ed interessante è la decomposizione ai valori singolari. Immaginiamo di avere una matrice $X \in \mathbb{R}^{n \times n}$. Se $XX^T = X^T X$ ¹ allora si può dimostrare che la matrice è diagonalizzabile: se la matrice è diagonalizzabile esiste un set completo di autovalori e autovettori:

$$(\lambda_1, u_1), \dots, (\lambda_n, u_n)$$

con autovalori dati dalla matrice diagonale

$$\Sigma = \text{Diag}(\lambda_1, \dots, \lambda_n)$$

e possiamo scrivere

$$X = U \Sigma U^T$$

dove $UU^T = I$ ², anche se la matrice non è invertibile. Se uno degli autovalori è nullo, allora questa non sarà invertibile. Si può dimostrare che la decomposizione appena sopra esiste per tutte le matrici (anche non quadrate), dove U e U^T vengono rimpiazzate da due matrici U e V ortogonali.

Se abbiamo una qualunque matrice $X \in \mathbb{R}^{m \times n}$, non necessariamente invertibile e quadrata, possiamo scrivere

$$X = U \Sigma V^T$$

¹Oppure, se $X \in \mathbb{C}^{n \times n}$, $XX^\dagger = X^\dagger X$.

²Analogamente alla nota 1, $UU^\dagger = I$.

dove $U \in \mathbb{R}^{m \times m}$ e $V \in \mathbb{R}^{n \times n}$. Possiamo scrivere la matrice di prima come

$$X = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = U \Sigma V^T,$$

dove Σ ha autovalori $\sigma_1 = 2$ e $\sigma_2 = 0$.

La decomposizione ai valori singolari c'è sempre, si può sempre scegliere questo percorso. In generale se X ha dimensione $n \times p$, $U \in \mathbb{R}^{n \times n}$ è ortogonale, $V \in \mathbb{R}^{p \times p}$ è ortogonale e Σ è la matrice diagonale dei valori singolari di X , che ha rango $r = \min(n, p)$ pari al rango di X .

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} = \underbrace{\begin{pmatrix} u_{11} & u_{12} & \dots & u_{1r} \\ u_{21} & u_{22} & \dots & u_{2r} \\ \vdots & \vdots & \ddots & \vdots \\ u_{m1} & u_{m2} & \dots & u_{mr} \end{pmatrix}}_U \cdot \underbrace{\begin{pmatrix} s_1 & 0 & \dots & 0 \\ 0 & s_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & s_r \end{pmatrix}}_\Sigma \cdot \underbrace{\begin{pmatrix} v_{11} & v_{12} & \dots & v_{1n} \\ v_{21} & v_{22} & \dots & v_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ v_{r1} & v_{r2} & \dots & v_{rn} \end{pmatrix}}_{V^*}$$

Figura 2.1

C'è un modo per ridurre il problema? Come mostrato in figura 2.1 Se si ha una matrice A con rango r , allora si ha che $s_1 \geq s_2 \geq \dots \geq s_r > s_{r+1} = \dots = s_n = 0$ e si può ridurre la matrice Σ togliendo le ultime colonne e righe. Si possono, quindi, ridurre anche le matrici U e V , poiché sono moltiplicate per righe e colonne nulle di Σ , il che porta a meno tempo di computazione (il tempo normalmente va come poco più di n^3). Le colonne e righe cancellate esistono, ma è uno spazio moltiplicato per 0 e quindi non è necessario al calcolo.

Applichiamo tutto questo a una regressione ordinaria ai minimi quadrati (OLS):

$$\tilde{y} = X\beta = X(X^T X)^{-1} X^T y,$$

dove β è la soluzione dell'equazione di prima. Possiamo scrivere

$$X^T X = V \Sigma^T U^T U \Sigma V^T$$

e visto che U è ortogonale

$$X^T X = V \Sigma^T \Sigma V^T = V D V^T,$$

dove D è una matrice diagonale i cui valori sono i valori singolari al quadrato. Sostituendo otteniamo

$$(X^T X)V = VD.$$

Gli autovettori di $(X^T X)$ sono dati dalle colonne di V e gli autovalori sono i valori singolari al quadrato. Analogamente possiamo fare il contrario e calcolare

$$(X X^T)U = UD,$$

per cui gli autovettori di $(X X^T)$ sono le colonne della matrice U e gli autovalori sono ancora i valori singolari al quadrato.

Possiamo scrivere

$$X\beta = X(V D V^T)^{-1} X^T y = U \Sigma V^T (V D V^T)^{-1} (U \Sigma V^T)^T y = U U^T y.$$

La minimizzazione del *Mean Squared Error* (MSE) utilizzata per definire la funzione di costo e le equazioni per il metodo OLS è

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \{ (y - X\beta)^T (y - X\beta) \}$$

che può essere riscritta come

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \frac{1}{n} \|y - X\beta\|_2^2$$

dove abbiamo definito la norma come

$$\|x\|_2 = \sqrt{\sum_i x_i^2}$$

In questi casi si può scegliere una funzione di costo differente (*metodo di Ridge*) e al posto di minimizzare solo la funzione di costo dei quadrati, si aggiunge la norma del vettore β moltiplicata per un certo parametro λ :

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2.$$

Questo si ha in problemi in cui si vuole minimizzare, ma per qualche motivo fisico o del modello, non si vuole che la norma del vettore β dei parametri diventi infinita. In questo caso scriviamo la funzione di costo come

$$C(X, \beta) = \frac{1}{n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2.$$

Un altro modo è usare la norma lineare per riscrivere la funzione di costo (*metodo di Lasso*)

$$C(X, \beta) = \frac{1}{n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1^2,$$

dove

$$\|x\|_1 = \sum_i |x_i|.$$

Se usiamo la regressione di Ridge, la funzione di costo è

$$C(X, \beta) = \frac{1}{n} \{ (y - X\beta)^T (y - X\beta) \} + \lambda \beta^T \beta,$$

la cui derivata è

$$\frac{\partial C}{\partial \beta} \propto -X^T (y - X\beta) + \lambda \beta = 0$$

da cui si ottiene

$$\beta^{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

Stiamo mettendo un'altra costrizione sui parametri che vanno minimizzati entro un certo limite, si impedisce alla minimizzazione di andare in una certa regione, ovvero nello spazio dei parametri in cui ci sono problemi. La matrice I è una matrice identità di dimensione $p \times p$ e dalla funzione di costo emerge l'imposizione

$$\sum_{i=0}^{p-1} \beta_i^2 \leq t.$$

Il metodo di Ridge è come OLS ma con l'aggiunta di un termine a $X^T X$. Usando l'equazione

$$(X X^T)U = U D$$

e analizzando la soluzione dell'OLS

$$X\beta = X(V D V^T)^{-1} X^T y = U \Sigma V^T (V D V^T)^{-1} (U \Sigma V^T)^T y = U U^T y,$$

otteniamo che

$$\begin{aligned} X\beta^{\text{Ridge}} &= U \Sigma V^T (V D V^T + \lambda I)^{-1} (U \Sigma V^T)^T y = U \Sigma V^T (V (D + \lambda) V^T)^{-1} (V \Sigma^T U^T) y \\ &= U \Sigma V^T V (D + \lambda)^{-1} V^T V \Sigma^T U^T y = \sum_{j=0}^{p-1} u_j u_j^T \frac{\sigma_j^2}{\sigma_j^2 + \lambda} y \end{aligned}$$

Più λ cresce, più le componenti vengono soppresse, in particolare le ultime, poiché sappiamo che i valori singolari soddisfano $\sigma_i \geq \sigma_{i+1}$, e diminuiscono quindi i gradi di libertà. Se $\lambda = 0$ siamo nel caso del metodo OLS; se $\lambda > 0$ si riducono i gradi di libertà effettivi che si ottengono.

Per la regressione LASSO non c'è una formula analitica per β come nella regressione Ridge, ma bisogna usare un algoritmo per risolverlo.

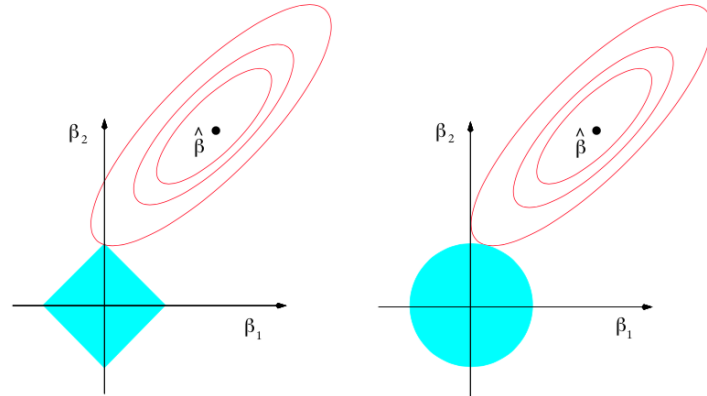
Calcolando β con OLS, assumendo che X sia ortonormale, ovvero che $X^T X = (X^T X)^{-1} = I$, abbiamo:

$$\beta^{\text{OLS}} = X^T y = \sum_{i=0}^{p-1} u_i u_i^T y,$$

mentre nel caso di Ridge i parametri vengono riscalati di un fattore $(1 + \lambda)^{-1}$:

$$\beta^{\text{Ridge}} = (I + \lambda I)^{-1} X^T y = (1 + \lambda)^{-1} \beta^{\text{OLS}},$$

quindi si riducono i valori di β . Questo discorso è simile anche per LASSO.



Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \leq t$ and $\beta_1^2 + \beta_2^2 \leq t^2$, respectively, while the red ellipses are the contours of the least squares error function.

Figura 2.2

In figura 2.2 è mostrato lo spazio di due parametri β_1 e β_2 : con il metodo OLS i punti che si discostano dal minimo stanno su delle ellissi (disegnate in rosso), infatti i termini della funzione di costo che non coinvolgono λ sono quadratici. A destra è mostrata la regressione Ridge, che limita la somma dei quadrati entro un cerchio: in questo modo si prende il valore di beta che va a toccare il cerchio, ma che non può spingersi oltre. A sinistra è mostrata la regressione LASSO, che fa la medesima cosa ma con un rombo. In Ridge, il valore del vettore β viene riscalato ad un vettore di dimensioni inferiori, sul cerchio. Nel caso di LASSO non è riscalato facilmente, poiché è molto più facile che il vettore venga portato vicino a uno degli assi. Sostanzialmente LASSO elimina parametri che creano instabilità e ne riscalda altri, mentre il Ridge tende a riscalarli tutti.

Estimators of β_j in the case of orthonormal columns of $\mathbf{X}$. M and λ are constants chosen by the corresponding techniques; sign denotes the sign of its argument (± 1), and x_+ denotes “positive part” of x . Below the table, estimators are shown by broken red lines. The 45° line in gray shows the unrestricted estimate for reference.

Estimator	Formula
Best subset (size M)	$\hat{\beta}_j \cdot I(\hat{\beta}_j \geq \hat{\beta}_{(M)})$
Ridge	$\hat{\beta}_j / (1 + \lambda)$
Lasso	$\text{sign}(\hat{\beta}_j)(\hat{\beta}_j - \lambda)_+$

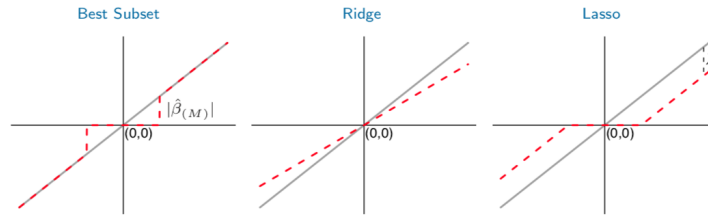


Figura 2.3

In figura 2.3 sono mostrati i riscalamenti dei parametri in tre casi (nel primo i parametri non vengono inseriti a meno che non siano importanti).

Ci sono diversi metodi di minimizzazione che possono essere scritti come

$$\tilde{\beta} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^N x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^N |\beta_j|^q \right\}$$

e che si possono visualizzare in figura 2.4.

Nel caso generale la soluzione è:

$$\beta^{\text{OLS}} = (X^T X)^{-1} X^T y = X^T y.$$

Riscrivendo con la decomposizione a valori singolari $X = U \Sigma V^T$:

$$X \beta^{\text{OLS}} = X (X^T X)^{-1} X^T y = U U^T y = \sum_{j=1}^p u_j u_j^T y.$$

Questa uguaglianza non è altro che una proiezione nello spazio che diagonalizza X : si parte con il vettore y dei dati dei campionamenti che si hanno e si proietta sullo spazio

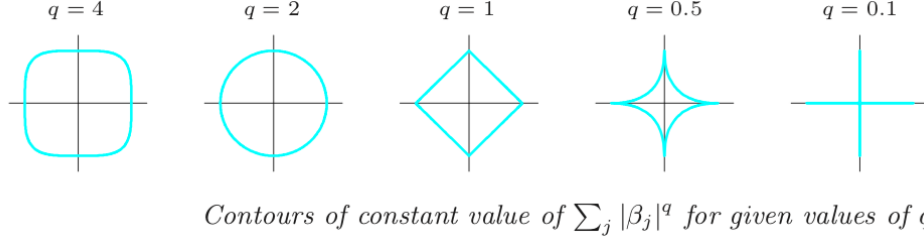


Figura 2.4

che diagonalizza X , dato da un iperpiano lineare definito da tutti i vettori di input X . Come predizione $\tilde{y}$ si prende la proiezione su questo piano, come si vede in figura 2.5. La sommatoria si ferma ai primi p elementi poiché, come citato in precedenza, sono

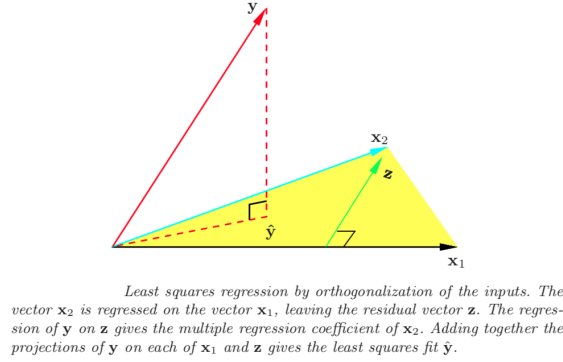


Figura 2.5

gli unici valori che sopravvivono alla moltiplicazione per Σ : supponendo che $X^T X$ sia invertibile, abbiamo una linearizzazione su p parametri e quindi solo p colonne indipendenti nella matrice X e quindi $UU^T y$ è la proiezione sullo spazio di dimensione p . Se X è degenere ed alcuni valori singolari sono nulli, allora la proiezione sarà su un sottospazio di dimensione inferiore a p : il modello lineare è definito su uno spazio p , ma se abbiamo dati che si muovono in uno spazio più piccolo, è facile che i parametri β vadano a valori infiniti, poiché si muovono nella parte di spazio che non influisce sul fitting dei dati.

Nella regressione di Ridge si cerca di risolvere questo problema aggiungendo un'identità con un piccolo valori in modo di rendere invertibile la matrice X :

$$X\beta^{\text{Ridge}} = X(X^T X + \lambda I)^{-1} X^T y = U D (D^2 + \lambda I)^{-1} D U^T y = \sum_{j=1}^p u_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} u_j^T y.$$

Si ottiene una proiezione riscaldando gli autovalori in modo che siano un pochino minori degli autovalori di OLS. La riduzione, o *shrinking*, dipende da λ e dal valore singolare σ_j : ogni direzione viene ridotta in maniera differente. Quale direzione viene ridotta di più dipende dalla varianza dei dati di input. Prendiamo la matrice X e facciamone la decomposizione

$$X^T X = V \Sigma U^T U \Sigma V^T = V \Sigma^2 V^T.$$

Gli autovettori della matrice V (ovvero le sue colonne), sono i vettori che definiscono direzioni indipendenti nello spazio delle variabili di input ed ogni direzione corrisponde ad un valore singolare differente. È una trasformazione dallo spazio p dimensionale dei parametri del modello, che può essere definito dai p valori di β . Contraendo v_j con X troviamo

$$z_j = Xv_j, \text{ con } j = 1, \dots, p,$$

che sono dei vettori nello spazio delle predizioni (dove le predizioni sono $y = X\beta$). Le direzioni sono definite dagli autovettori v_j e prendendo la varianza di z possiamo scrivere (assumendo media nulla):

$$\text{Var}(z_j) = \text{Var}(Xv_j) = \frac{1}{n} \sum_i [(x_j)_i - \bar{x}_j]^2 = \frac{\sigma_j^2}{n}.$$

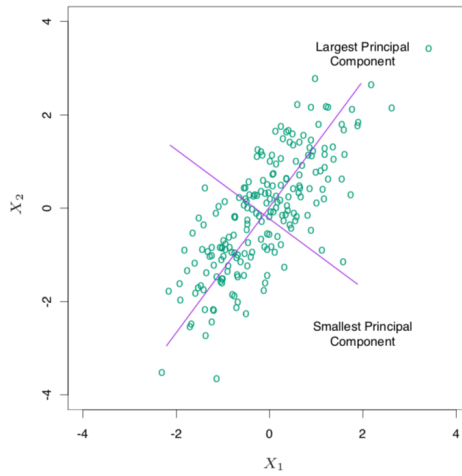


Figura 2.6

La varianza dei dati di input dei campionamenti nella direzione j -esima dello spazio dei parametri β è data dal valore singolare al quadrato diviso n . Le direzioni nello spazio dei parametri con valori singolari maggiori sono quelle con una varianza maggiore, in cui i dati sono più distribuiti. In figura 2.6 abbiamo un modello a 2 parametri x_1 e x_2 , in cui si vanno a prendere tanti campionamenti, in questo caso 3 numeri (x_1, x_2, y) che sono distribuiti in una certa maniera, che dipende da com'è fatto l'esperimento, per cui i campionamenti possono essere distribuiti su tutto lo spazio dei parametri oppure limitarsi ad una regione. In figura si vede come in una direzione ci sia una grande varianza, mentre nell'altra la varianza sia piccola: facendo il campionamento si è andata ad esplorare una regione nello spazio dei parametri in cui l'esplorazione è maggiore in una direzione rispetto che all'altra. Quando si fa il *rescaling*, maggiore è σ_j , più vicino sarà ad 1 il fattore di *shrinking*. Se la varianza è grande e quindi abbiamo preso dei dati molto sparsi, il metodo Ridge fa la proiezione come in OLS e la lascia praticamente invariata.

Se prendiamo, invece, dei parametri in una direzione in cui il modello è insensibile, perché non abbiamo abbastanza dati per fare una buona costruzione del modello, la varianza è piccola e così anche σ_j , che porta ad avere una riduzione molto più drastica. Riducendo l'importanza di una determinata direzione, si riduce il rischio di ritrovare

soluzioni "strane" date dalla mancanza di informazioni necessarie alla compilazione di un buon modello.

Proviamo ad avere una visione un po' più completa. Consideriamo il caso in cui la matrice X è

$$X = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}, \quad \text{con } n = p$$

dove ogni misura è in un punto ortogonale agli altri. Nel metodo dei minimi quadrati si fa la minimizzazione

$$\beta^{\text{OLS}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n} \|(y - X\beta)\|_2^2 = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \sum_{i=0}^{n-1} (y_i - \beta_i)^2,$$

da cui si trova:

$$\beta_i^{\text{OLS}} = y_i.$$

Se per ogni variabile x_i il valore y_i dipende unicamente da x_i e non da tutte le altre (perché la matrice è diagonale e stiamo considerando un modello lineare) è ovvio che $y_i \propto x_i$ ed in particolare la costante di proporzionalità è 1.

Analogamente calcoliamo il risultato con il metodo di Ridge

$$\beta^{\text{Ridge}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \sum_i (y_i - \beta_i)^2 + \lambda \sum_j \beta_j^2$$

annullando la derivata:

$$\sum_i \delta_{ij} [- (y_i - \beta_i) + \lambda \beta_i] = 0 \implies \beta^{\text{Ridge}} = \frac{y_i}{1 + \lambda},$$

dove abbiamo utilizzato il fatto che il valore singolare è 1. Il termine λ può servire sia per rendere invertibile la matrice X , sia per "far pagare un prezzo alto" quando qualche valore di β tende a diventare grande.

Applichiamo lo stesso ragionamento al metodo LASSO

$$\beta^{\text{LASSO}} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \sum_i (y_i - \beta_i)^2 + \lambda \sum_j |\beta_j|,$$

annullando la derivata:

$$-2(y_i - \beta_i) + \lambda \frac{\beta_i}{|\beta_i|} = 0$$

da cui si ricava la soluzione

$$\beta_i^{\text{LASSO}} = \begin{cases} y_i - \frac{\lambda}{2} & \text{se } y_i > \frac{\lambda}{2}; \\ y_i + \frac{\lambda}{2} & \text{se } y_i < -\frac{\lambda}{2}; \\ 0 & \text{se } |y_i| \leq \frac{\lambda}{2}. \end{cases}$$

L'ultima casistica è determinata dal fatto che se y è molto piccolo, il termine in λ è dominante, quindi per ottenere il minimo è necessario fissare $\beta_i = 0$.

Ci sono altri modelli (*Best Subset*) per cui si tengono i set di componenti β_j più importanti e si annullano le altre (vedi figura 2.4).

Esempio Cancro alla Prostata

Si sono andati a prendere dati da pazienti operati di cancro alla prostata e si sono analizzati diversi parametri. Da questo dataset si possono estrarre combinazioni di informazioni sui dati, oppure si possono prendere i dati e farne un fitting con un modello lineare. In figura 2.7 sono mostrati i risultati e quelli che ci interessano maggiormente

TABLE 3.3. *Estimated coefficients and test error results, for different subset and shrinkage methods applied to the prostate data. The blank entries correspond to variables omitted.*

Term	LS	Best Subset	Ridge	Lasso	PCR	PLS
Intercept	2.465	2.477	2.452	2.468	2.497	2.452
lcavol	0.680	0.740	0.420	0.533	0.543	0.419
lweight	0.263	0.316	0.238	0.169	0.289	0.344
age	-0.141		-0.046		-0.152	-0.026
lbph	0.210		0.162	0.002	0.214	0.220
svi	0.305		0.227	0.094	0.315	0.243
lcp	-0.288		0.000		-0.051	0.079
gleason	-0.021		0.040		0.232	0.011
pgg45	0.267		0.133		-0.056	0.084
Test Error	0.521	0.492	0.492	0.479	0.449	0.528
Std Error	0.179	0.143	0.165	0.164	0.105	0.152

Figura 2.7: Prova

sono il metodo dei minimi quadrati, il metodo Ridge e il metodo LASSO, ed infine il *best subset*. Nel *best subset* si vanno a prendere i predittori più correlati di altri, mentre in Ridge e LASSO si tengono in principio tutti i parametri, ma il LASSO ne rimuove alcuni (età, lcp, gleason e pgg45). In questo caso il fit è stato fatto in maniera un po' complessa e si è diviso il dataset in due gruppi, con un set di *training* ed un set di *error*. Una volta fatto il modello sui dati di *training* con il suo errore, si calcola anche l'errore quadratico medio sui dati di test, che dà un errore maggiore. Fittare un modello su un tot di dati quasi per definizione dà l'errore più basso, ma non è detto che funzioni su tutti i dati ed è per questo che l'errore cresce.

Nel caso di Ridge e LASSO l'errore diminuisce, anche se in teoria non dovrebbe, poiché la funzione di costo è maggiore rispetto a quella del LS, tuttavia stiamo facendo una statistica su vari sottogruppi del test di training e quindi ci può essere questa differenza. Pagando il prezzo della riduzione dei vettori β per evitare che divergano, che si traduce in una funzione di costo un pochino maggiore, si ottiene un errore medio sulle predizioni minore perché si evitano regioni dove il modello non è sottocontrollo.

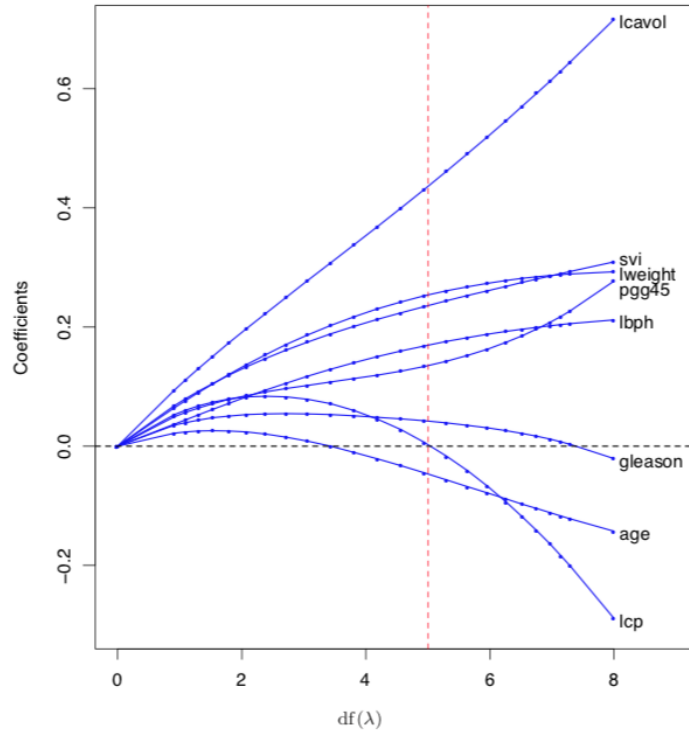
Quando scriviamo la funzione di costo, dobbiamo aggiungere al termine derivato da LS, una parte dipendente da λ :

$$C = LS + \lambda \sum_i |\beta_i|^q,$$

che equivale a richiedere che

$$\sum_i |\beta_i|^q \leq t.$$

In figura 2.8 è mostrato cosa succede al variare del parametro λ nel metodo di Ridge, avendo per $\lambda = 0$, 8 parametri.



Profiles of ridge coefficients for the prostate cancer example, as the tuning parameter λ is varied. Coefficients are plotted versus $df(\lambda)$, the effective degrees of freedom. A vertical line is drawn at $df = 5.0$, the value chosen by cross-validation.

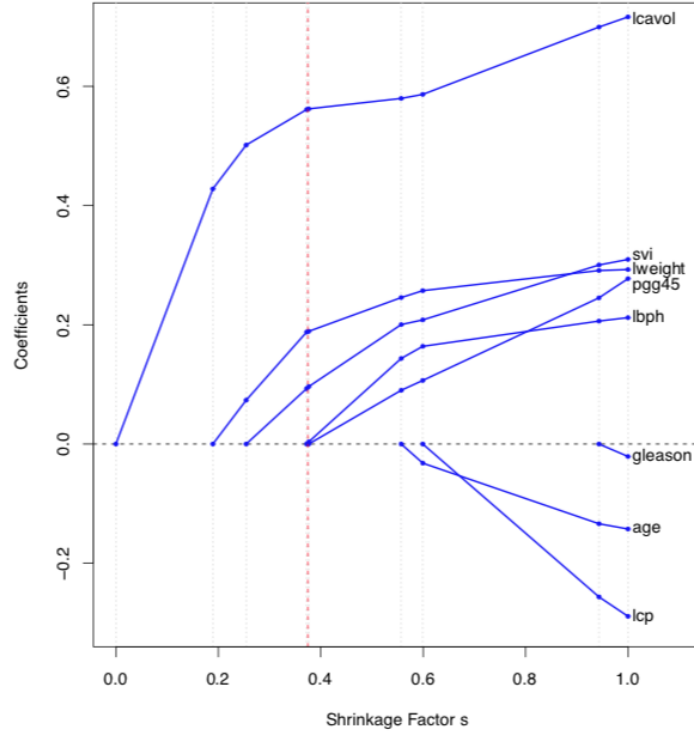
Figura 2.8

Il numero totale di gradi di libertà si può stimare con

$$df(\lambda) = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + \lambda};$$

quando $\lambda = 0$ ogni valore singolare corrisponde ad 1 grado di libertà, mentre quando λ cresce, il valore di β viene riscalato nella direzione j -esima e il numero di gradi di libertà effettivi diminuisce. Nel caso di una matrice $X^T X$ non invertibile, si hanno valori singolari nulli e questi non verrebbero considerati come gradi di libertà effettivi. Per $\lambda \rightarrow \infty$ i gradi di libertà sono nulli e tutti i parametri vengono ridotti a 0. In figura si passa da valori di $\lambda = \infty$ a sinistra a valori di $\lambda = 0$ a destra. Per $\lambda = 0$ si vedono altissime correlazioni ed al crescere di λ i parametri vengono pian piano diminuiti e si evitano correlazioni infinite. C'è un valore di λ per cui si ottiene il fit ottimale (riga rossa verticale).

In figura 2.9 è mostrato il modello di LASSO. Per un fattore di *shrinking* pari ad 1 non si hanno riduzioni o eliminazioni di parametri ed infatti i valori corrispondono al



Profiles of lasso coefficients, as the tuning parameter t is varied. Coefficients are plotted versus $s = t / \sum_1^p |\hat{\beta}_j|$. A vertical line is drawn at $s = 0.36$, the value chosen by cross-validation. Compare Figure 3.8 on page 65; the lasso profiles hit zero, while those for ridge do not. The profiles are piece-wise linear, and so are computed only at the points displayed; see Section 3.4.4 for details.

Figura 2.9

metodo di Ridge con $\lambda = 0$. Quando si aumenta lo *shrinking*, alcuni parametri vanno a diminuire e quando raggiungono il valore 0 vengono eliminati. Anche in questa figura è mostrato il valore ottimale del fattore di *shrinking* con una linea rossa verticale.

2.1 Richiamo di Nozioni Statistiche

Lavoriamo con due variabili $x \in X$ e $y \in Y$ con una serie di campionamenti di dati in cui si hanno le due variabili allo stesso momento (ad una x corrisponde una y). Normalmente le variabili x sono rilevate da una certa distribuzione di probabilità e sono soggette ad errore, e se esiste una funzione $y = f(x)$ che descrive il sistema, tipicamente i valori di y sono centrati attorno a $f(x)$ con l'aggiunta di un errore stocastico. Altre volte, invece, si ha una serie di dati (con la possibilità che siano correlati) estratti in maniera completamente stocastica. In generale si suppone di avere una serie di coppie di variabili ad ogni campionamento

$$D = \left\{ (x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1}) \right\} \quad (\text{Dominio})$$

con distribuzioni di probabilità $p(x)$ e $p(y)$. Nel caso continuo abbiamo

$$0 \leq p \leq 1, \quad x \in [a, b]$$

e

$$\int_a^b p(x) dx = 1;$$

nel caso discreto abbiamo un set $\{x_i\} = X$ con

$$\sum_{x_i \in X} p(x_i) = 1.$$

Avremo dei valori di aspettazione definiti dall'operatore dei momenti

$$\mathbb{E}[x^n] = \int_a^b x^n p(x) dx = \sum_{x_i \in X} x_i^n p(x_i).$$

Se $n = 1$ si ottiene la media $\mu_x = \mathbb{E}[x] = \langle x \rangle = \mu$, se $n = 0$ si ottiene la normalizzazione, per $n = 2$ se si considera il valore di aspettazione di $x - \mu_x$ si ottiene la varianza:

$$\begin{aligned} \text{Var}(x) = \sigma_x^2 = \sigma^2 &= \int_a^b (x - \mu_x)^2 p(x) dx = \sum_{x_i \in X} (x_i - \mu_x)^2 p(x_i) \\ &= \langle x^2 \rangle - 2\langle x \rangle \mu_x + \mu_x^2 = \langle x^2 \rangle - \mu_x^2. \end{aligned}$$

Quando si fanno dei campionamenti delle misure, la media e la varianza reali non si conoscono, ma è necessario avere media e varianza campionate:

$$\bar{\mu}_x = \frac{1}{n} \sum_{i=0}^{n-1} x_i \neq \mu_x$$

e

$$\bar{\sigma}_x^2 = \frac{1}{n-1} \sum_{i=0}^{n-1} (x_i - \bar{\mu}_x)^2 \neq \sigma_x^2.$$

Per $n \rightarrow \infty$ i valori campionati sono uguali a quelli reali, ma per un numero finito di campionamenti questo non è vero. La divisione per $n - 1$ nella varianza campionata deriva dal fatto che ci sono n gradi di libertà, ma da questi si estrae la media campionata e quindi ne rimangono solo $n - 1$ per stimare la varianza.

Un'altra funzione utile è la *covarianza*. Ogni volta che misuriamo le due variabili, che sono risultato di diversi campionamenti estratti dalla loro distribuzione di probabilità, abbiamo una certa probabilità di ottenere x ed y pari a $P(x, y)$. La covarianza è legata alla varianza, quindi a come si sparpagliano i valori campionati rispetto alla propria media, ed è definita come:

$$\begin{aligned} \text{cov}(x, y) &= \iint dx dy P(x, y) (x - \mu_x)(y - \mu_y) = \left(\iint dx dy P(x, y) xy \right) - \mu_x \mu_y \\ &= \mathbb{E}[xy] - \mu_x \mu_y = \langle xy \rangle - \langle x \rangle \mu_y - \mu_x \langle y \rangle + \mu_x \mu_y, \end{aligned}$$

dove assumiamo che $\sigma^2(x), \sigma^2(y) < +\infty$; quando si dovrà andare a calcolare a partire dai dati, otterremo la varianza campionata, ottenuta tramite le medie campionate.

Se le variabili sono indipendenti ed identicamente distribuite (*i.i.d.*), allora vale $P(x, y) = p(x)p(y)$ e quindi

$$\mathbb{E}[x, y] = \int dx p(x)x \int dy p(y)y = \mu_x \mu_y,$$

da cui emerge che $\text{cov}(x, y) = 0$. La covarianza è una misura di quanto due variabili siano correlate.

Si vede come la covarianza tra una variabile e se stessa sia pari alla varianza, per cui possiamo scrivere la matrice di covarianza (che è simmetrica)

$$C[x, y] = \begin{bmatrix} \text{Var}(x) & \text{cov}(x, y) \\ \text{cov}(x, y) & \text{Var}(y) \end{bmatrix}$$

La covarianza nello spazio delle variabili probabilistiche è un prodotto interno, infatti

- se $a, b \in \mathbb{R}$ e X, Y, Z sono variabili di stato, allora

$$\text{cov}(aX + bY, Z) = a \text{cov}(X, Z) + b \text{cov}(Y, Z);$$

- è simmetrica: $\text{cov}(X, Y) = \text{cov}(Y, X) \quad \forall X, Y$;
- è semidefinita positiva, infatti $\text{cov}(X, X) = \sigma^2(x) \geq 0 \quad \forall x$.

Siccome è un prodotto interno, vale la disuguaglianza di *Cauchy-Schwarz*:

$$|\text{cov}(X, Y)| \leq \sqrt{\text{cov}(X, X)} \sqrt{\text{cov}(Y, Y)} = \sqrt{\sigma^2(x) \sigma^2(y)}$$

Consideriamo le variabili x_1 e x_2 correlate (tendono a stare più o meno lungo una retta, figura 2.6); la varianza nella direzione "ampia" è maggiore di quella nella direzione "stretta". Nel caso delle correlazioni siamo interessati alla forma dell'ellissoide e quindi ciò che torna utile è normalizzare la matrice di covarianza rispetto alle varianze singole delle variabili. È per questo utile la matrice di correlazione

$$K[x, y] = \begin{bmatrix} \frac{\text{Var}(x)}{\text{Var}(x) \text{Var}(y)} & \frac{\text{cov}(x, y)}{\text{Var}(x) \text{Var}(y)} \\ \frac{\text{cov}(x, y)}{\text{Var}(x) \text{Var}(y)} & \frac{\text{Var}(y)}{\text{Var}(x) \text{Var}(y)} \end{bmatrix} = \begin{bmatrix} 1 & a \\ a & 1 \end{bmatrix},$$

per la disuguaglianza di Cauchy-Schwarz sappiamo che $|a| \leq 1$; nel caso di variabili *i.i.d.* vale

$$K[x, y] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

In generale per un set di variabili statistiche

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix}$$

vale

$$K(\vec{x}, \vec{x}) = \left[\text{diag}\{ \text{cov}(\vec{x}, \vec{x}) \} \right]^{-1} \text{cov}(\vec{x}, \vec{x}) \left[\text{diag}\{ \text{cov}(\vec{x}, \vec{x}) \} \right]^{-1}$$

Possiamo dare un esempio di matrici di covarianza nell'esempio di un sistema di regressione. Quando si fa una regressione OLS o Ridge con un sistema di p variabili e n campionamenti, si usa la matrice di design per cui in ogni riga si ha un set di campionamenti ed in ogni colonna si ha lo stesso predittore campionato n volte:

$$X = \begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} & \cdots & x_{0,p-1} \\ x_{1,0} & x_{1,1} & x_{1,2} & \cdots & x_{1,p-1} \\ x_{2,0} & x_{2,1} & x_{2,2} & \cdots & x_{2,p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{n-1,0} & x_{n-1,1} & x_{n-1,2} & \cdots & x_{n-1,p-1} \end{bmatrix}$$

Questa matrice si può scrivere come

$$\mathbf{X} = [\mathbf{x}_0 \quad \mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_{p-1}]$$

dove ogni colonna è la sequenza di campionamenti di una data variabile, oppure come

$$x_i^T = [x_{0,i} \quad x_{1,i} \quad x_{2,i} \quad \cdots \quad x_{n-1,i}]$$

dove ogni componente corrisponde ad ogni campionamento i

Si può estrarre la matrice di correlazione e di covarianza rispetto alle variabili in input dalla matrice di design X . La matrice di covarianza viene scritta come

$$C[x] = \begin{bmatrix} \text{Var}[x_0] & \text{cov}[x_0, x_1] & \text{cov}[x_0, x_2] & \cdots & \text{cov}[x_0, x_{p-1}] \\ \text{cov}[x_1, x_0] & \text{Var}[x_1] & \text{cov}[x_1, x_2] & \cdots & \text{cov}[x_1, x_{p-1}] \\ \text{cov}[x_2, x_0] & \text{cov}[x_2, x_1] & \text{Var}[x_2] & \cdots & \text{cov}[x_2, x_{p-1}] \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{cov}[x_{p-1}, x_0] & \text{cov}[x_{p-1}, x_1] & \text{cov}[x_{p-1}, x_2] & \cdots & \text{Var}[x_{p-1}] \end{bmatrix}$$

e la matrice di correlazione

$$K[x] = \begin{bmatrix} 1 & \text{corr}[x_0, x_1] & \text{corr}[x_0, x_2] & \cdots & \text{corr}[x_0, x_{p-1}] \\ \text{corr}[x_1, x_0] & 1 & \text{corr}[x_1, x_2] & \cdots & \text{corr}[x_1, x_{p-1}] \\ \text{corr}[x_2, x_0] & \text{corr}[x_2, x_1] & 1 & \cdots & \text{corr}[x_2, x_{p-1}] \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{corr}[x_{p-1}, x_0] & \text{corr}[x_{p-1}, x_1] & \text{corr}[x_{p-1}, x_2] & \cdots & 1 \end{bmatrix}$$

Quando si hanno variabili distribuite con media nulla, la covarianza diviene semplicemente il prodotto scalare dei vettori x e y . Avendo la matrice di design possiamo scrivere la matrice di covarianza come

$$C[x] = \frac{1}{n} X^T X = \mathbb{E}[X^T X]$$

se la media è campionata bisogna rinormalizzare la matrice di design sottraendo ad ogni colonna la media campionata e bisogna dividere per $n - 1$ anziché per n . Se per esempio consideriamo la matrice

$$x = \begin{bmatrix} x_{00} & x_{01} \\ x_{10} & x_{11} \end{bmatrix} = [\mathbf{x}_0 \quad \mathbf{x}_1],$$

possiamo valutare il valore di aspettazione

$$\mathbb{E}[X^T X] = \frac{1}{n} X^T X = \begin{bmatrix} x_{00}^2 + x_{01}^2 & x_{00}x_{10} + x_{01}x_{11} \\ x_{10}x_{00} + x_{11}x_{01} & x_{10}^2 + x_{11}^2 \end{bmatrix}$$

che è pari a

$$C[\mathbf{x}_0, \mathbf{x}_1] = \begin{bmatrix} \text{Var}[\mathbf{x}_0] & \text{cov}[\mathbf{x}_0, \mathbf{x}_1] \\ \text{cov}[\mathbf{x}_1, \mathbf{x}_0] & \text{Var}[\mathbf{x}_1] \end{bmatrix}$$

La matrice può essere usata per calcolare la varianza e le correlazioni tra tutte le variabili di input.

2.1.1 Probabilità

Date due variabili statistiche X e Y , il teorema della probabilità totale è

$$P(X \cup Y) = P(X) + P(Y) - P(X \cap Y)$$

e la probabilità di intersezione può essere scritta come

$$P(X \cap Y) = P(X, Y) = P(X|Y)P(Y) = P(Y|X)P(X).$$

La probabilità condizionata è per definizione

$$P(X|Y) = \frac{P(X, Y)}{P(Y)}$$

con la regola di somma

$$P(X) = \sum_{y_i \in Y} P(X|y_i)P(y_i).$$

Queste definizioni sono alla base del *teorema di Bayes*, per cui rielaborando la probabilità di intersezione si ottiene:

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

che generalizzando ad un insieme di più variabili X_i che partizionano lo spazio degli eventi Ω , ovvero dove

$$\begin{cases} X_i \cup X_j = \emptyset & \forall i \neq j, \\ \bigcup_{i=1}^n X_i = \Omega; \end{cases}$$

è

$$P(X_i|Y) = \frac{P(Y|X_i)P(X_i)}{\sum_j P(Y|X_j)P(X_j)} = \frac{P(Y|X_i)P(X_i)}{P(Y)};$$

si tende a parlare di probabilità *a priori* (come $P(X_i)$ e $P(Y)$), indipendenti da ciò che succede alle altre variabili, e di probabilità *a posteriori* (come $P(Y|X_i)$), che sono le probabilità condizionate dagli altri eventi. Il teorema di Bayes permette di calcolare la probabilità di una causa una volta che è accaduto l'evento.

2.2 Approfondimento Regressioni

Immaginiamo di prendere $p + 1$ variabili ad ogni misura: un output y e la lista di variabili di input x . Non è sempre così in un set di dati: quando si prendono i dati è possibile che si abbia una lista con differenti variabili (come nel dataset del Boston housing, che è un campionamento dei prezzi delle varie case, ma che non sono una funzione di qualche cosa, ma sono presentati vari aspetti della casa, come numero di stanze, la vicinanza a certe aree, ecc...). Quando si ha un set di dati non esiste

necessariamente un predittore ed un output, ma spesso ci interessiamo ad avere una variabile (che sarà y) che siamo in grado di predire in funzione delle altre (che saranno $\underline{x}$), quindi cerchiamo le correlazioni tra questa ed il resto del dataset. Abbiamo quindi $(y, x) \in \mathbb{R}^{p+1}$. Il set di dati è un insieme di punti di n campionamenti, ognuno di $p+1$ valori:

$$D = \{(y_0, \underline{x}_0), (y_1, \underline{x}_1), \dots, (y_{n-1}, \underline{x}_{n-1})\}$$

dove

$$\underline{x} = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_p \end{pmatrix}$$

e

$$\begin{cases} y \in \mathbb{R} & \text{problema di regressione} \\ y \in \{a_0, a_1, \dots, a_n\} & \text{problema di classificazione} \end{cases}$$

In un problema di regressione si cerca, se esiste, la relazione $y = f(\underline{x})$, che controlla la probabilità di ottenere in un campionamento $(y, \underline{x})$, ovvero la distribuzione di probabilità $P(y, x)$. Ciò che interessa non è la probabilità in sé, ma interessa capire se questa "nasconde" una relazione che lega y ad $\underline{x}$. Di solito si ha la funzione $f(\underline{x}) = y + \varepsilon$ con $\varepsilon = \mathcal{N}(\mu = 0, \sigma^2)$ ovvero un errore proveniente da una distribuzione normale con media nulla (errori stocastici), che è la distribuzione limite di errori che si sommano gli uni con gli altri (teorema del limite centrale).

Il problema è trovare la $f(\underline{x})$ che descrive al meglio la probabilità, con un numero di campionamenti finito.

Un modo per affrontare il problema è il metodo di *massima verosimiglianza* (MLE, *maximum likelihood estimation*). Assumiamo di avere un modello per la probabilità che dipende da un certo numero di parametri $\{\beta_i\}$, con $\beta \in \mathbb{R}^q$. Tra tutti i modelli che si possono applicare, l'idea è quella di prendere il modello con la maggiore probabilità di restituire il set di misure D , ovvero i campionamenti che già abbiamo. Quindi:

$$P(D|\vec{\beta}) = \prod_{i=0}^{n-1} p_m((y_i, \underline{x}_i)|\vec{\beta})$$

L'idea è prendere il set di parametri che identifica il modello migliore massimizzando la probabilità condizionata $P(D|\vec{\beta})$ nello spazio dei modelli possibili. Essendo la probabilità condizionata una produttoria ed essendo il logaritmo una funzione monotona, si può massimizzare il logaritmo della probabilità, in modo che si massimizzi una sommatoria:

$$\vec{\hat{\beta}} = \underset{\vec{\beta} \in \mathbb{R}^q}{\operatorname{argmax}} P(D|\vec{\beta}) = \underset{\vec{\beta} \in \mathbb{R}^q}{\operatorname{argmax}} \left\{ \sum_{i=0}^{n-1} \log p_m((y_i, \underline{x}_i)|\vec{\beta}) \right\}$$

Un altro modo è minimizzare

$$\vec{\hat{\beta}} = \underset{\vec{\beta} \in \mathbb{R}^q}{\operatorname{argmin}} C(\vec{\beta})$$

dove

$$C(\vec{\beta}) = - \sum_{i=0}^{n-1} \log p_m((y_i, \underline{x}_i)|\vec{\beta})$$

che è ciò che abbiamo fatto all'inizio. L'idea di minimizzazione della funzione di costo ha questa giustificazione statistica.

Come facciamo a trovare $p_m((y_i, x_i) | \vec{\beta})$? Per la scelta del modello bisogna considerare diverse cose. C'è un set di condizioni che la maggior parte delle volte viene soddisfatto. Assumiamo che

- ci sia una funzione vera, una relazione $y = f(\underline{x})$;
- che sia deterministica, ovvero fissato il valore di $\underline{x}$ non ci sia un errore su $\underline{x}$;
- ci sia un errore su y normalmente distribuito, quindi $\varepsilon \in \mathcal{N}(0, \sigma^2)$, con distribuzione normale

$$p(\varepsilon) = \frac{e^{-\varepsilon^2/2\sigma^2}}{\sqrt{2\pi\sigma^2}};$$

allora la y è distribuita normalmente attorno a $f(\underline{x})$:

$$y \in \mathcal{N}(f(\underline{x}), \sigma^2),$$

con

$$p(y|\underline{x}) = \frac{e^{-(y-f(\underline{x}))^2/2\sigma^2}}{\sqrt{2\pi\sigma^2}}$$

Si può dimostrare che effettivamente media e varianza corrispondono a ciò che ci aspettiamo:

$$\mathbb{E}[y] = \int d\varepsilon p(\varepsilon)y = \int d\varepsilon p(\varepsilon)[f(\underline{x}) + \varepsilon] = \mathbb{E}(f(\underline{x})) + \mathbb{E}(\varepsilon) = f(\underline{x}) = \langle y \rangle$$

e

$$\begin{aligned} \text{Var}(y) &= \mathbb{E}[(y - \langle y \rangle)^2] = \mathbb{E}(y^2) - (\mathbb{E}(y))^2 = \mathbb{E}[(f(\underline{x}))^2 + 2f(\underline{x})\varepsilon + \varepsilon^2] - (\mathbb{E}(y))^2 \\ &= (f(\underline{x}))^2 + 2f(\underline{x}) \cdot 0 + \sigma^2 - (f(\underline{x}))^2 = \sigma^2. \end{aligned}$$

Se la distribuzione dell'errore per qualche motivo non dovesse essere normale o se per qualche motivo non si può applicare il teorema del limite centrale ad y , la distribuzione sarebbe un po' diversa, ma i primi due momenti rimarrebbero pari a quelli di una distribuzione normale. Una volta che si ha la funzione normale, diventa possibile applicarla al metodo di massima verosimiglianza; definiamo una funzione di costo

$$C[f(\underline{x})] = - \sum_i \log p(y, \underline{x})$$

in cui la probabilità non è nient'altro che la distribuzione normale, per cui

$$C[f(\underline{x})] = \frac{n}{2} \log 2\pi + n \log \sigma + \frac{1}{2\sigma^2} \sum_{i=0}^{n-1} (y_i - f(x))^2.$$

Il metodo di massima verosimiglianza richiede la minimizzazione di una funzione di costo che è pari alla somma dei quadrati; la funzione più probabile è

$$\hat{f}(\underline{x}) = \underset{f(\underline{x})}{\operatorname{argmin}} \sum_{i=0}^{n-1} [y_i - f(x_i)]^2.$$

La soluzione migliore non è univoca, poiché esistono infiniti modelli che rendono nulla la funzione di costo. Se, ad esempio, abbiamo un modello con un numero di gradi di libertà maggiore dei punti che abbiamo ($q > n$), ci sono infinite funzioni che hanno abbastanza gradi di libertà da dare esattamente i valori campionati $\forall n$ e che fuori dal dataset di training si comportano nei modi più svariati. Si tratta, in questo caso di *overfitting*, che presenta errore nullo sul set di dati usati per il fit, ma che non ha controllo sui rimanenti.

Il metodo della massima verosimiglianza è basato sulla minimizzazione della funzione di costo ed un'applicazione particolare è quando partiamo dalle tre assunzioni di prima, per cui la funzione di costo è la somma dei quadrati.

Una volta scelta la funzione di costo, si possono stimare gli errori di previsione per un modello $f(\underline{x})$, prendendo l'errore medio di previsione

$$\begin{aligned} \text{Err}(f) &= \mathbb{E}[(Y - f(\underline{X}))^2] = \int [y - f(\underline{x})]^2 p(\underline{x}, y) dy d^p \underline{x} \\ &= \int d^p \underline{x} \int dy P(y|\underline{x}) P(\underline{x}) [y - f(\underline{x})]^2 = \int d^p \underline{x} P(\underline{x}) \int dy P(y|\underline{x}) [y - f(\underline{x})]^2 \\ &= \mathbb{E}_{\underline{X}}[\mathbb{E}_{Y|\underline{X}}[(Y - f(\underline{X}))^2|\underline{X}]] \end{aligned}$$

ovvero la media su tutto il set della funzione di costo (in questo caso lavoriamo sui minimi quadrati). Dà l'errore medio con cui la funzione $f(\underline{x})$ è in grado di prevedere y . L'errore medio di previsione è dato dall'errore su ognuno dei punti, mediato su tutti i punti possibili. Si può, quindi, minimizzare la media condizionata per ogni valore di $\underline{X} = \underline{x}_0$ separatamente. La funzione $f(x)$ può essere scritta come

$$f(x) = \underset{c}{\operatorname{argmin}} \mathbb{E}_{Y|\underline{X}}[(y - c)^2 | \underline{X} = \underline{x}_0] = \mathbb{E}[y | \underline{X} = \underline{x}_0]$$

Cosa succede se assumiamo una forma lineare della funzione? Abbiamo la possibilità di riderivare la regressione lineare già vista. Assumendo una funzione lineare $f(\underline{x}) \approx \underline{x}^T \vec{\beta}$ con $q = p$, l'errore medio di previsione è

$$\text{Err}(\underline{x}^T \vec{\beta}) = \int (y - \underline{x}^T \vec{\beta})^2 P(y, \underline{x}) d^p \underline{x} dy;$$

per ogni valore di $\underline{x}$ scegliamo i valori di $\vec{\beta}$ che rendono minimo l'errore:

$$\begin{aligned} \frac{\partial \text{Err}}{\partial \beta_i} &= \int 2(-2x_i)(y - \underline{x}^T \vec{\beta}) P(y, \underline{x}) d^p \underline{x} dy = 2 \left(\int x_i \underline{x}^T P(y, \underline{x}) d^p \underline{x} dy \right) \vec{\beta} - \\ &- 2 \int x_i y P(\underline{x}, y) d^p \underline{x} dy = 0 \quad \forall i = 0, 1, \dots, p-1; \end{aligned}$$

con

$$\frac{\partial \text{Err}}{\partial \vec{\beta}} = 2 \langle \underline{x} \underline{x}^T \rangle \vec{\beta} - 2 \langle \underline{x} y \rangle = 0$$

da cui si ottiene

$$\vec{\beta} = \langle \underline{x} \underline{x}^T \rangle^{-1} \langle \underline{x} y \rangle.$$

I valori di aspettazione si possono ottenere solo come medie campionate e si può dimostrare che approssimando con le medie campionate su D si ha

$$\begin{aligned} \langle \underline{x} \underline{x}^T \rangle_D &= X^T X \\ \langle \underline{x} y \rangle &= X^T \vec{y}, \end{aligned}$$

dove X è la matrice di design, quindi l'applicazione come media campionata non è altro che la regressione lineare già vista in precedenza (con p gradi di libertà). Restringendoci allo spazio dei modelli lineari, il più probabile è la soluzione che avevamo già trovato, che ora ha una giustificazione statistica.

Ritornando però alla formulazione più generale

$$f(x) = \underset{c}{\operatorname{argmin}} \mathbb{E}_{Y|X}[(y - c)^2 | X = \underline{x}_0] = \mathbb{E}[y | X = \underline{x}_0],$$

è ovvio che il modello non deve essere necessariamente lineare. Qual è il modello più semplice e che funziona meglio di tutti? Potremmo pensare di usare il modello in cui si prende la media su y di tutti i campionamenti in $\underline{x}_0$. Se avessimo un numero infinito di campionamenti, allora il modello sarebbe esatto:

$$f(\underline{x}) = \mathbb{E}[Y | X = \underline{x}].$$

Non si possono avere campionamenti esatti in ogni punto, quindi la soluzione più ovvia è prendere un piccolo intorno del punto e fare la media della funzione al suo interno. Questo è il metodo dei k vicini prossimi (*k-nearest neighbors*). Per ogni valore di $\underline{x}$ si cerca la media vera degli y condizionati a quel particolare set di valori di input. Se il valore è ottenibile allora la soluzione è esatta, ma dobbiamo stimare dal numero di campionamenti n in D , che è un numero finito. Difficilmente si campionano tanti punti per un singolo valore di $\underline{x}_0$, quindi si campionano i vicini prossimi:

$$\hat{f}_k(\underline{x}) = \frac{1}{k} \sum_{(y_i, \underline{x}_i) \in N_k(\underline{x})} y_i.$$

L'insieme dei vicini prossimi di $\underline{x}$ sono tutti i primi k punti messi in ordine di distanza da $\underline{x}_0$

$$N_k(\underline{x}_0) = \{(y_j, \underline{x}_j) \in D : d_{jx} \leq d_{kx} \quad j = 1, \dots, k \wedge d_{jx} > d_{kx} \quad \forall i > k\}$$

Si può scegliere $k = 1$ e si prende la media unicamente sul punto campionato e si disegna una funzione a scatti che ha un errore nullo per definizione. Se si vuole una funzione più continua, si deve avere una densità di punti abbastanza grande e si toglie la possibilità di considerare oscillazioni stocastiche o di fare un overfitting o di avere una funzione che è troppo discontinua. Più è grande k , meno gradi di libertà ci sono per gestire la funzione, che si può dimostrare essere n/k .

Nel grafico 2.10 ci sono un tot di punti arancioni e blu distribuiti in un quadrato e si cerca una funzione che sia in grado di predire se il colore di un punto. Il valore di y è quantizzato, quindi si parla di un problema di classificazione e non di regressione: gli input sono variabili continue, ma l'output è quantizzato. Uno dei modi per risolvere questo problema è trovare una funzione $\hat{Y}$ e assegnarle un valore 0 o 1 a seconda che sia blu o arancione, fare un fit e poi valutare la previsione della regressione:

$$\hat{G} = \begin{cases} \text{Arancione} & \text{se } \hat{Y} > 0.5, \\ \text{Blu} & \text{se } \hat{Y} < 0.5, \end{cases}$$

Dalla distribuzione dei punti possiamo avere due possibili casi: o i punti arancioni (analogamente i punti blu) sono distribuiti lungo un'unica distribuzione bivariata (gaussiana in 2D), o i punti sono distribuiti lungo 10 piccole gaussiane diverse. Se siamo nella situazione di due cluster grandi (arancione e blu) è facile dividere i punti

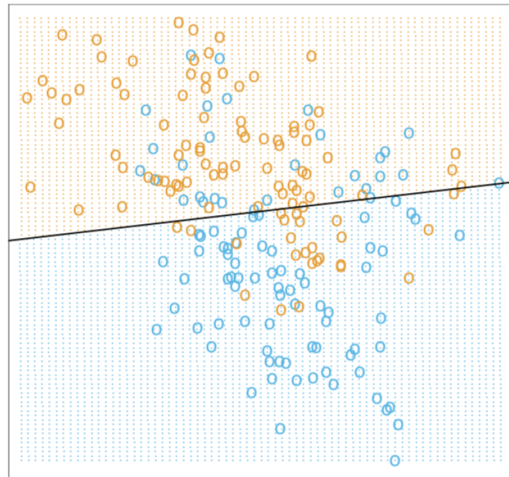
con una regressione lineare, per cui si prende la linea che divide le due distribuzioni. Se sono tanti cluster piccoli è meglio utilizzare il metodo dei nearest neighbors.

Nelle figura 2.11 sono mostrate due soluzioni analizzate con il metodo dei prossimi vicini, nel caso di $k = 15$ e $k = 1$. Nel secondo caso per definizione il modello è corretto per ogni punto utilizzato, ma i confini sono molto frammentati e discontinui: le previsioni sono esatte per i punti di training, ma si trova una funzione molto discontinua e che non è detto rappresenti la funzione iniziale. Il primo caso invece ha un contorno molto più continuo, anche se non azzecca tutti i punti.

L'approssimazione può migliorare o peggiorare a seconda di k e non è detto che la funzione con errore nullo $k = 1$ abbia previsioni corrette. In figura 2.12 sono mostrati gli errori dei diversi modelli di nearest neighbors sul dataset di training e sul dataset di test. I quadrati sono i risultati del fit lineare (che ha 3 parametri). La linea azzurra è l'errore ottenuto sul dataset usato per il training. Più si diminuisce k più l'errore scende fino ad arrivare a 0 per $k = n$. La linea gialla invece è l'errore che si ottiene sul dataset di test, sui dati non presi in considerazione per il training. Si ottiene che l'errore medio di previsione che si calcola diminuisce all'aumentare di gradi di libertà, fino a che il modello non diventa abbastanza plastico e con abbastanza gradi di libertà per adattarsi a tutti i dati campionati. Aggiungendo gradi di libertà l'errore di training continua a diminuire, ma non ci si accorge che si stanno introducendo cose che vanno fuori controllo e che peggiorano l'errore vero, ovvero l'errore sul dataset di test. È importante avere l'occhio per scegliere quei modelli che danno la possibilità, adattandosi al sistema, di ottenere gli errori minori.

Questo è un esempio del bilancio tra Bias e varianza, descritto qualitativamente nella figura 2.13. Più il modello è complesso più l'errore di training diminuisce, ma quando si prende l'errore di test di sample, questo segue quello di training fino ad un certo punto, ma poi comincia a risalire.

Se si calcola l'errore usando i minimi quadrati otteniamo, nel caso dei k-nearest



A classification example in two dimensions. The classes are coded as a binary variable (BLUE = 0, ORANGE = 1), and then fit by linear regression. The line is the decision boundary defined by $x^T \hat{\beta} = 0.5$. The orange shaded region denotes that part of input space classified as ORANGE, while the blue region is classified as BLUE.

Figura 2.10

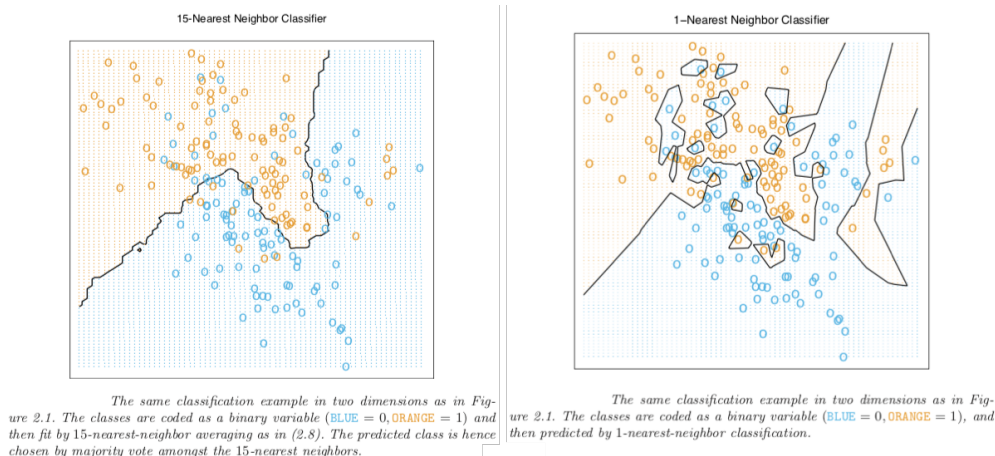


Figura 2.11

neighbors:

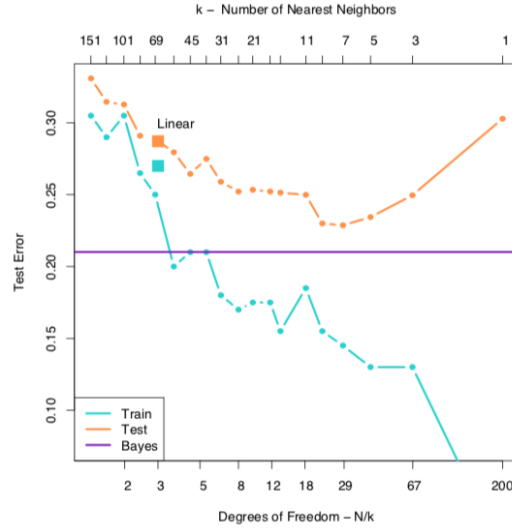
$$\begin{aligned}
 \text{EPE}_k(x_0) &= \mathbb{E}[(Y - \hat{f}_k(x_0))^2 | X = x_0] = \sigma^2 + [\text{Bias}^2(\hat{f}_k(x_0)) + \text{Var}(\hat{f}_k(x_0))] \\
 &= \sigma^2 + [f(x_0) - \frac{1}{k} \sum_{\ell=1}^k f(x_\ell)]^2 + \frac{\sigma^2}{k}
 \end{aligned}$$

è quasi una varianza tra la previsione del modello e il campionamento reale (per un certo x_0): oltre alla varianza c'è un Bias, che è il modo in cui il modello sbaglia sulla funzione reale del sistema. I tre elementi sono:

- l'errore statistico su y , che è intrinseco ai campionamenti, è un errore sperimentale;
- il Bias, che è una media tra le previsioni e il valore vero e misura quanto il modello sia inadatto a fittare completamente i dati. Più il modello è semplice, più fa fatica a riprodurre i dati e più il Bias è grande; più il modello è complesso, più riesce a fittare i dati e più il Bias rispetto ai dati campionati diminuisce;
- la varianza con cui vengono estratti i dati: più si fa un overfitting del sistema più si permette al sistema di comportarsi in maniera svariata al di fuori dei dati campionati.

Il Bias diminuisce e sostanzialmente segue la linea azzurra di figura 2.13, mentre la varianza cresce: ciò che si deve fare è trovare il compromesso tra i due fattori, ovvero il punto d'incontro.

Se si hanno molti dati e si ha un modello di nearest neighbors, si potrebbe pensare di essere nella situazione più conveniente, ma non è così in realtà. Dobbiamo avere una certa risoluzione, quindi in un volume abbastanza piccolo dobbiamo avere una densità consistente di punti. Quando siamo in tante dimensioni, per ottenere la stessa densità di punti che avremmo in uno spazio bidimensionale dovremmo avere un numero di punti sempre più grande (in maniera esponenziale). Possiamo fare degli esempi.



Misclassification curves for the simulation example used in Figures 2.1, 2.2 and 2.3. A single training sample of size 200 was used, and a test sample of size 10,000. The orange curves are test and the blue are training error for k -nearest-neighbor classification. The results for linear regression are the bigger orange and blue squares at three degrees of freedom. The purple line is the optimal Bayes error rate.

Figura 2.12

Esempio 1

Consideriamo un ipercubo di dimensioni p (figura 2.14 a sinistra) e prendiamone una piccola porzione cubica al suo interno. Vogliamo andare a vedere la dimensione necessaria per avere un certo numero di stati. Se Δv è la percentuale del volume del piccolo cubo, la lunghezza dei lati sarà $e_p(\Delta v) = \Delta v^{1/p}$. Possiamo fare qualche calcolo e vediamo che se si vuole un cubo che copre l'1% dello spazio delle fasi in 10 dimensioni, bisogna avere una copertura di ogni variabile del 63% dello spazio delle fasi, mentre se vogliamo che copra il 10% del volume totale, ogni dimensione deve coprire il 79% dello spazio delle fasi (vedi figura 2.14 a destra).

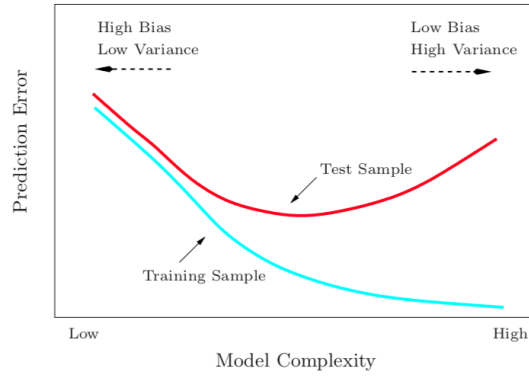
Immaginiamo di avere N punti campionati in uno spazio p -dimensionale; la distanza media dall'origine del punto più vicino all'origine è

$$d(p, N) = \left(1 - \frac{1}{2}^{1/N}\right)^{1/p},$$

se $N \rightarrow \infty$ la distanza è nulla, mentre se N è finito e p cresce, tutti i punti vanno a distanze abbastanza grandi dal centro e non si riesce a fare un'analisi corretta. Con $N = 500$, $p = 10$ la distanza media è circa la metà dello spazio totale ($d(p, N) \approx 0.52$).

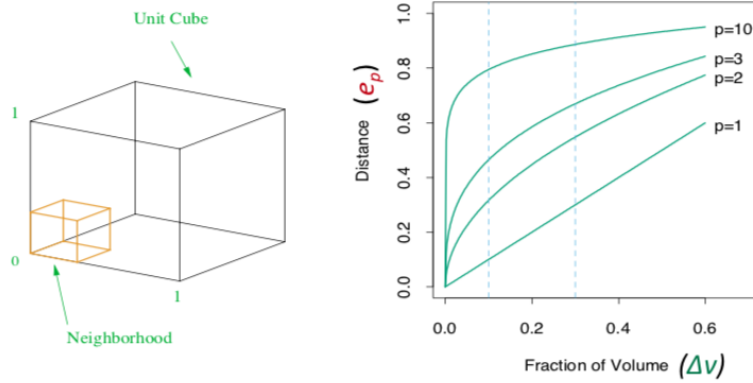
Esempio 2

Consideriamo la funzione in figura 2.15. Quando siamo in una dimensione il nearest neighbor è abbastanza vicino, mentre quando siamo anche solo in due dimensioni la distanza aumenta notevolmente.



Test and training error as a function of model complexity.

Figura 2.13



The curse of dimensionality is well illustrated by a subcubical neighborhood for uniform data in a unit cube. The figure on the right shows the side-length of the subcube needed to capture a fraction r of the volume of the data, for different dimensions p . In ten dimensions we need to cover 80% of the range of each coordinate to capture 10% of the data.

Figura 2.14

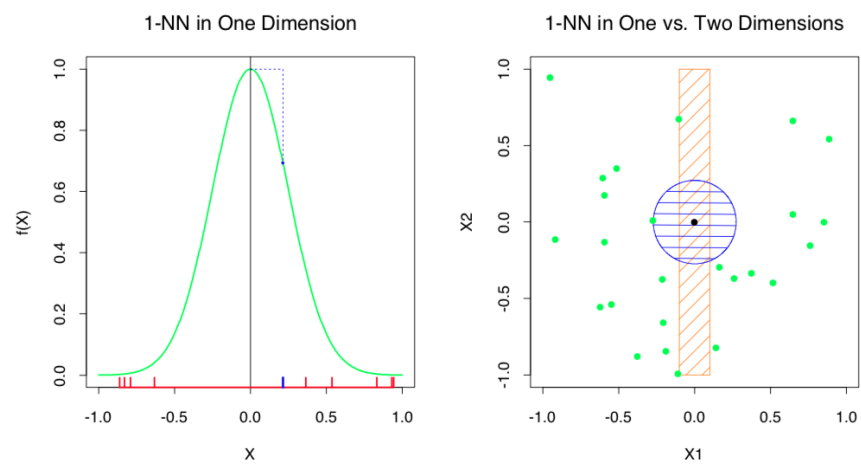


Figura 2.15

Capitolo 3

Selezione e Valutazione dei Modelli

3.1 Importanza dell'errore

Quello che succede normalmente è che si fitta il sistema con un modello (di training), per fare poi predizioni su dati non appartenenti al dataset di training ($x_i \notin S$). Quello che succede tipicamente quindi è che $y = f(\underline{x}) + \varepsilon$, dove le variabili X e Y sono le variabili stocastiche di tutto il dataset. Dopo di che la nostra predizione Y viene confrontata con il valore effettivamente misurato tramite una funzione di costo che ci dice quanto il valore predetto è differente dal valore misurato. Le funzioni di costo più comuni sono quella dei minimi quadrati e il valore assoluto degli scarti

$$L(Y, \hat{f}(X)) = \begin{cases} (Y - \hat{f}(X))^2; \\ |Y - \hat{f}(X)|. \end{cases}$$

si noti che la funzione costo dello scarto quadratico medio funzionerà particolarmente bene se la y è distribuita normalmente (ε è distribuita normalmente), quindi il modello con la maggiore verosimiglianza sarà quello che minimizza i quadrati.

In questo contesto è chiaro che è importante fare una stima degli errori di predizione che si commettono in ciascuna di queste fasi di utilizzo della rete neurale, quello più immediato è l'*errore di test*, o di *generalizzazione*:

$$\text{Err}_{\mathcal{T}} = E[L(Y, \hat{f}(X)) | \mathcal{T}] = \int d^p \underline{x} dy P(\underline{x}, y) (y - \hat{f}(\underline{x}; D))^2.$$

dove si sta utilizzando l'assunzione che x sia una variabile P-dimensionale. Questo è l'errore che si ottiene generalizzando il modello a dei dati che sono oltre il dataset di training $\mathcal{T}$; è il valore di aspettazione della funzione di costo per le variabili X e Y . Se disegniamo un grafico in cui poniamo sull'asse delle ascisse la complessità del modello (quindi in sostanza stiamo variando la complessità del modello - possiamo anche dire che stiamo variando il modello), otterremmo una delle sottili curve rosse in figura 3.1. Si osserva che Per modelli troppo semplici si ha un errore troppo grande; aumentando la complessità questo si riduce, ma dopo un certo punto risale. In sostanza, ogni volta che si prende un campionamento nuovo e si fa il fit trovando un nuovo modello $\hat{f}(\underline{x})$, l'errore di generalizzazione cambia; al variare di $\mathcal{T}$ esso si sparpaglia su un certo intervallo come si vede dalla distribuzione delle linee rosse più fini in figura 3.1).

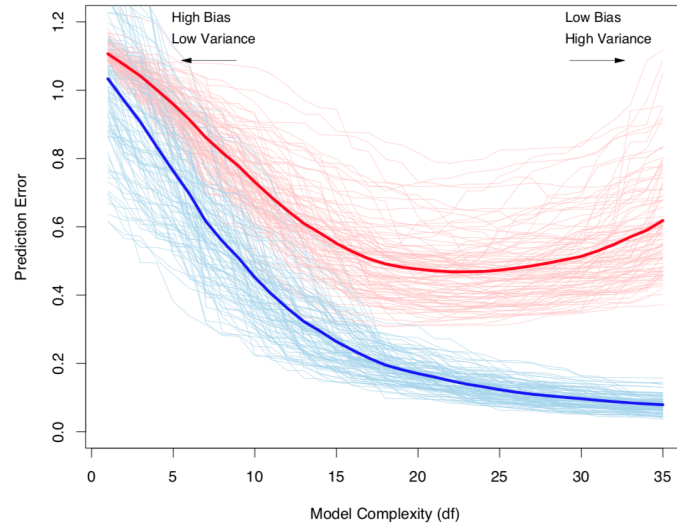


Figura 3.1: Andamento degli errori in funzione della complessità del modello

Possiamo quindi calcolare un errore più rappresentativo del modello prendendo una media di questi errori, ovvero l'*errore medio di previsione* o expected test error

$$\text{Err} = \mathbb{E}[L(Y, \hat{f}(X))] = \mathbb{E}[\text{Err}_{\mathcal{T}}] = \lim_{N_{\mathcal{T}} \rightarrow \infty} \frac{1}{N_{\mathcal{T}}} \sum_{i=1}^{N_{\mathcal{T}}} \text{Err}_{\mathcal{T}_i}.$$

Con questo errore si realizza sostanzialmente una media su tutti i possibili gradi di libertà del problema, tenendo conto anche della dipendenza del modello dal dataset di training. Una volta scelto il modello, minimizzando questo errore troviamo quale siano i gradi di libertà migliori e confrontando i modelli differenti capiamo quale sia quello più adeguato. Questo errore è rappresentato dalla linea rossa spessa in figura 3.1

L'ultimo errore interessante è l'*errore medio di training*, dato dalla media degli errori sui vari punti del dataset di training; è un errore il cui valore dipende dal dataset utilizzato. Si confrontano le predizioni del modello con i dati campionati utilizzati per determinare il modello stesso:

$$\overline{\text{Err}} = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}(x_i))$$

Il valore campionato non è necessariamente il valore di previsione del modello, perché il modello non è necessariamente la funzione vera $f(\underline{x})$ e perché comunque c'è un errore ε . Può essere poi calcolato come media su diversi dataset utilizzati per definire il modello, in figura 3.1 le linee blu più sottili sono i singoli errori di training e la linea blu più spessa, media delle linee blu sottili, è l'errore medio di training. È chiaro che più il modello è complesso, con gradi di libertà e più è malleabile, più l'errore medio di training diminuisce.

Tra le tecniche che consentono di ridurre l'errore e di ottenere un buon modello una delle cose più semplici da fare è, se si hanno abbastanza dati per fare una statistica sensata, dividere il set di dati in tre sottogruppi: set di training (50-60%), set di



Model selection: estimating the performance of different models in order to choose the best one.

Model assessment: having chosen a final model, estimating its prediction error (generalization error) on new data.

Figura 3.2: Esempio di divisione del dataset in training, validation, test

validazione (25-30%) e set di test (20-25%). La scelta delle percentuali e dei punti da prendere dipende da caso a caso ed è un'arte. Nella prima parte, di *model selection*, si utilizza il set di training per fittare il modello: si calcola l'errore di training sul set di training e lo si minimizza. Dopodiché si calcola l'errore di generalizzazione per il fit fatto testandolo sui dati del set di validazione:

$$\text{Err}_{\mathcal{T}} = \sum_{\underline{x}_i \in V} L(y_i, \hat{f}(\underline{x}_i; D)),$$

questo errore viene utilizzato per valutare e confrontare modelli diversi di cui poi si sceglie quello migliore, in questa fase si modifica il modello fino a che il validation error non viene minimizzato. Durante tutta questa procedura si estrae il modello migliore per studiare i dati e non si vanno a toccare i dati di test.

Una volta scelto il modello migliore si ricalcola l'errore di generalizzazione, ma sui dati del set di test, poiché il training e il validation non possono essere usati per avere una stima credibile sull'errore, poiché affetti da Bias (sono stati usati per scegliere il modello da utilizzare, non possono dare una stima vera dell'errore generale della predizione). Trovato il modello, si calcola l'errore sul dataset di test, che è l'ultimo step, il *model assessment*. Questa è la procedura che si segue normalmente e che è la migliore nel caso si abbiano tanti dati, cosa che in realtà succede poco frequentemente.

3.2 Tradeoff Bias Varianza

Consideriamo un set di dati

$$S = \{(y_i, \underline{x}_i) : i = 1, \dots, N\}$$

dove si assume che S sia estratto da una distribuzione $P(Y, \underline{X})$ non nota, si abbia una scelta per una funzione di costo $\ell(y, \hat{y})$ e si abbia un predittore $h(\underline{x}) \in C$ scelto da un insieme di possibili predittori (e.g. kNN, Linkage ecc).

Il predittore "finale" ottenuto utilizzando il set di training è

$$\hat{f}_{[S]} = \arg \min_{h \in C} \left\{ \frac{1}{N} \sum_{i=1}^N \ell(y_i, h(\underline{x}_i)) \right\} \quad (3.1)$$

dove la notazione $\hat{f}_{[S]}$ indica la dipendenza del predittore finale dal dataset di allenamento, inoltre il predittore dipenderà anche dalle possibili scelte che ho considerato per i predittori.

A questo punto ci sono almeno tre errori che è importante calcolare per avere una visione completa della performance di un modello:

1. errore medio di training

$$\text{Err}_{Tr} = \frac{1}{N} \sum_{i=1}^N \ell(y_i, \hat{f}(\underline{x}_i)), \quad (3.2)$$

questo errore dipende naturalmente dal dataset di training e dal modello che ho scelto.

2. errore di test o errore di generalizzazione

$$\text{Err}(S) = \mathbb{E}_{P(Y, X)}[\ell(Y, \hat{f}(X)) | S] = \ell_P(\hat{f}) \quad (3.3)$$

questo errore è noto anche come rischio statistico di $\hat{f}(\underline{x})$ e dipende dal dataset di training S ma soltanto tramite $\hat{f} = \hat{f}_{[S]}$

3. errore medio di previsione o errore di test medio

$$\text{Err} = \mathbb{E}_S[\text{Err}(S)] \quad (3.4)$$

quello che fa questo tipo di errore è sostanzialmente rimuovere la dipendenza da S

Ci sono due tipi di errori che si possono avere quando si fitta un modello: quanto il metodo sia buono per il tipo di problema e quante varianze ed errori ci sono sul dataset.

Ciò che cerchiamo è l'errore medio di previsione. Assumiamo che $Y = f(X) + \varepsilon$ con $E(\varepsilon) = 0$ e $\text{Var}(\varepsilon) = \sigma_\varepsilon^2$. Invece di fare una media su tanti punti differenti del training set $\mathcal{T}$

$$\mathcal{T} = \{(\underline{x}_0, y_0), \dots, (\underline{x}_n, y_n)\},$$

ci focalizziamo solo su un punto $\underline{x}_0$. Quanto la predizione $\hat{f}(\underline{x}; \mathcal{T})$ dipende dal set di training? Calcoliamo l'errore medio di previsione sui vari possibili training set nel punto $\underline{x}_0$ (mediamo sul training set e sui campionamenti di y):

$$\begin{aligned} \text{Err}(\underline{x}_0) &= \mathbb{E}[(Y - \hat{f}(\underline{x}_0))^2 | X = \underline{x}_0] \\ &= \sigma_\varepsilon^2 + [\mathbb{E}\hat{f}(\underline{x}_0) - f(\underline{x}_0)]^2 + \mathbb{E}[\hat{f}(\underline{x}_0) - \mathbb{E}\hat{f}(\underline{x}_0)]^2 \\ &= \sigma_\varepsilon^2 + \text{Bias}^2(\hat{f}(\underline{x}_0)) + \text{Var}(\hat{f}(\underline{x}_0)) \\ &= \text{Errore irriducibile} + \text{Bias}^2 + \text{Varianza} \end{aligned}$$

Questo errore si divide in tre parti: la varianza dei campionamenti, un errore al quadrato tra la differenza della media tra i vari dataset $\hat{f}(\underline{x}_0)$ rispetto alla funzione vera (Bias) e quanta è la varianza del modello al variare dei set di training.

Dimostrazione

Dimostriamo la formula della decomposizione Bias-Varianza.

$$\begin{aligned} \text{Err}(\underline{x}_0) &= \sum_{\mathcal{T}_i} \frac{1}{N_{\mathcal{T}}} \mathbb{E}_y(Y - \hat{f}(\underline{x}_0; \mathcal{T}_i))^2 \\ &= \mathbb{E}[(Y - \hat{f}(\underline{x}_0; \mathcal{T}) + \mathbb{E}[\hat{f}(\underline{x}_0)] - \mathbb{E}[\hat{f}(\underline{x}_0)])^2 | X = \underline{x}_0] \end{aligned}$$

dove abbiamo aggiunto e rimosso $\mathbb{E}[\hat{f}(\underline{x}_0)] = \mathbb{E}_{\mathcal{T}} \hat{f}(\underline{x}_0, \mathcal{T})$; continuando

$$\begin{aligned} \text{Err}(\underline{x}_0) &= \mathbb{E}[(f(\underline{x}_0) - \mathbb{E}\hat{f} + \varepsilon + \mathbb{E}\hat{f} - \hat{f}(\underline{x}_0))^2] \\ &= \mathbb{E}[(f(\underline{x}_0) - \mathbb{E}\hat{f})^2] + \mathbb{E}(\varepsilon^2) + \mathbb{E}[(\mathbb{E}\hat{f} - \hat{f}(\underline{x}_0))^2] + \\ &\quad + 2\mathbb{E}[\varepsilon(f(\underline{x}_0) - \mathbb{E}\hat{f}) + \varepsilon(\mathbb{E}\hat{f} - \hat{f}(\underline{x}_0)) + (f(\underline{x}_0) - \mathbb{E}\hat{f})(\mathbb{E}\hat{f} - \hat{f}(\underline{x}_0))] \end{aligned}$$

dove il primo ed il terzo termine hanno una media unicamente sul training set e non dipendono dal campionamento in sé, mentre il secondo elemento non dipende dal training set, ma unicamente dal campionamento. Tutti i termini misti si cancellano¹; otteniamo, quindi:

$$\begin{aligned} \text{Err}(\underline{x}_0) &= \sigma_\varepsilon^2 + (f(\underline{x}_0) - \mathbb{E}\hat{f})^2 + \mathbb{E}[(\mathbb{E}\hat{f} - \hat{f}(\underline{x}_0; \mathcal{T}))^2] \\ &= \sigma_\varepsilon^2 + \text{Bias}^2 + \text{Var}_{\mathcal{T}}(\hat{f}(\underline{x}_0)). \end{aligned}$$

Mediando su tutti i punti abbiamo:

$$\text{Err} = \mathbb{E}_x[\text{Err}(\underline{x}_0)] = \sigma_\varepsilon^2 + \overline{\text{Bias}^2} + \overline{\text{Var}(\hat{f}(\underline{x}_0))}$$

3.3 Decomposizione Bias-Varianza per Regressione Lineare

Nella regressione lineare abbiamo $\hat{y} = \underline{X}^T \beta = \hat{f}(\underline{x}, \beta(\mathcal{T}))$. Ciò che ci interessa è l'errore di previsione medio per un valore $\underline{x}_0$:

$$\text{Err}(\underline{x}_0) = \sigma_\varepsilon^2 + (f(\underline{x}_0) - \mathbb{E}_{\mathcal{T}} \hat{f}(\underline{x}_0))^2 + \text{Var}(\hat{f}(\underline{x}_0)),$$

La varianza si ottiene come

$$\text{Var}(\hat{f}) = \mathbb{E}_{\mathcal{T}, y}[(\hat{f}(\underline{x}_0) - \mathbb{E}\hat{f})^2]$$

e, se $\hat{f}(\underline{x}, \mathcal{T}) = \underline{X}^T \hat{\beta}(\mathcal{T}) \Rightarrow \mathbb{E}\hat{f}(\underline{x}) = \underline{X}^T \mathbb{E}(\hat{\beta})$, allora:

$$\begin{aligned} \text{Var}(\hat{f}) &= \mathbb{E}_{\mathcal{T}, y}[\underline{x}_0^T (\hat{\beta}(\mathcal{T}) - \mathbb{E}(\hat{\beta}))]^2 \\ &= \underline{x}_0^T \mathbb{E}[(\hat{\beta}(\mathcal{T}) - \mathbb{E}(\hat{\beta}))(\hat{\beta}(\mathcal{T}) - \mathbb{E}(\hat{\beta}))^T] \underline{x}_0 \\ &= \underline{x}_0^T \text{Var}(\hat{\beta}) \underline{x}_0. \end{aligned}$$

La media è anche sulle variabili y , infatti β dipende dal dataset di training, che dipende a sua volta da y . Sapendo che $\hat{\beta} = (X^T X)^{-1} X^T \underline{Y}$, possiamo calcolare

$$\text{Var}(\hat{\beta}) = \mathbb{E}[\{(X^T X)^{-1} X^T Y - \mathbb{E}(\hat{\beta})\} \{(X^T X)^{-1} X^T Y - \mathbb{E}(\hat{\beta})\}^T];$$

abbiamo che

$$\mathbb{E}(\hat{\beta}) = \mathbb{E}[(X^T X)^{-1} X^T Y] = (X^T X)^{-1} X^T \mathbb{E}(Y) = (X^T X)^{-1} X^T X \beta = \beta,$$

quindi, continuando:

$$\begin{aligned} \text{Var}(\hat{\beta}) &= \mathbb{E}[(X^T X)^{-1} X^T Y Y^T X (X^T X)^{-1} + \beta \beta^T - 2\beta \beta^T] \\ &= (X^T X)^{-1} X^T \mathbb{E}(Y Y^T) X (X^T X)^{-1} - \beta \beta^T. \end{aligned}$$

¹ $\mathbb{E}(\varepsilon) = 0$ e $\mathbb{E}(\mathbb{E}\hat{f} - \hat{f}) = 0$.

Se viene campionata la variabile $Y = X\beta$, otteniamo

$$E[YY^T] = X\beta\beta^T X^T + \sigma^2 I_{n \times n}$$

facendo il campionamento si ottiene la prima parte su $\mathcal{T}$ e la seconda parte sulla y vera (i termini misti si cancellano).

Ritornando alla varianza di β :

$$\begin{aligned} \text{Var}(\beta) &= (X^T X)^{-1} X^T \{X\beta\beta^T X^T + \sigma^2 I\} X (X^T X)^{-1} - \beta\beta^T \\ &= \beta\beta^T + \sigma_\varepsilon^2 (X^T X)^{-1} - \beta\beta^T = \sigma_\varepsilon^2 (X^T X)^{-1}, \end{aligned}$$

otteniamo la varianza di $\hat{f}$:

$$\text{Var}(\hat{f}(\underline{x}_0)) = \underline{x}_0 (X^T X)^{-1} \underline{x}_0 \sigma_\varepsilon^2$$

Abbiamo quindi tutto il necessario per calcolare l'errore di previsione nel punto $\underline{x}_0$. A questo punto, per calcolare l'errore di previsione medio, dobbiamo farne una media sulle $\underline{x}$.

L'errore di previsione medio si può stimare², come una media sui punti noti di training $\underline{x}_i$:

$$\text{Err} = \frac{1}{N} \sum_{i=1}^N \text{Err}(\underline{x}_i) = \sigma_\varepsilon^2 + \frac{1}{N} \sum_{i=1}^N (f(\underline{x}_i) - E\hat{f}(\underline{x}_i))^2 + \frac{p}{N} \sigma_\varepsilon^2$$

che è la decomposizione dell'errore in errore statistico, Bias e varianza in una regressione lineare. Il termine di varianza dipende dall'errore statistico dei campionamenti, ma anche dal numero di parametri: più punti si hanno del dataset minore sarà la varianza, ma allo stesso tempo, maggiore è il numero di parametri usati per il fit, maggiore sarà la varianza (linea rossa del grafico 3.3).

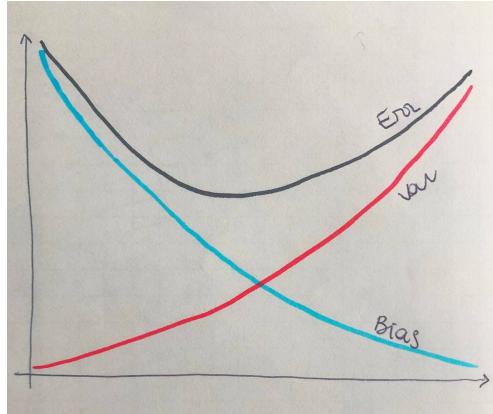


Figura 3.3

²Sapendo che $\sum_i \underline{x}_i^T (X^T X)^{-1} \underline{x}_i = \text{Tr}\{X(X^T X)^{-1} X^T\} = \text{Tr}\{I_{p \times p}\} = p$, poiché $X = \begin{pmatrix} \underline{x}_1^T \\ \vdots \\ \underline{x}_N^T \end{pmatrix}$

3.3.1 Caso del metodo dei prossimi vicini

Nel caso del metodo dei vicini prossimi, otteniamo una cosa di questo tipo

$$\text{Err}(x_0) = \sigma_\varepsilon^2 + \left[f(x_0) - \frac{1}{k} \sum_{\ell=1}^k f(x_\ell) \right]^2 + \frac{\sigma_\varepsilon^2}{k}$$

l'errore statistico è indipendente (il primo termine) dal modello e dal punto x_0 ; il Bias è la differenza tra il valore della funzione nel punto x_0 e il valore della funzione vicino ad ogni punto; la varianza è la varianza della funzione del modello: è proporzionale alla varianza del rumore (σ_ε^2) perché approssimiamo il valore della funzione con la media dei campionamenti dei valori più vicini, ma i campionamenti nei punti più vicini sono corretti dall'errore del rumore esterno, da cui quindi dipende la varianza della funzione. La varianza è inversamente proporzionale a k , infatti tanti più vicini prossimi si hanno, tanto più si fa una media e tanto più è difficile avere varianza alta.

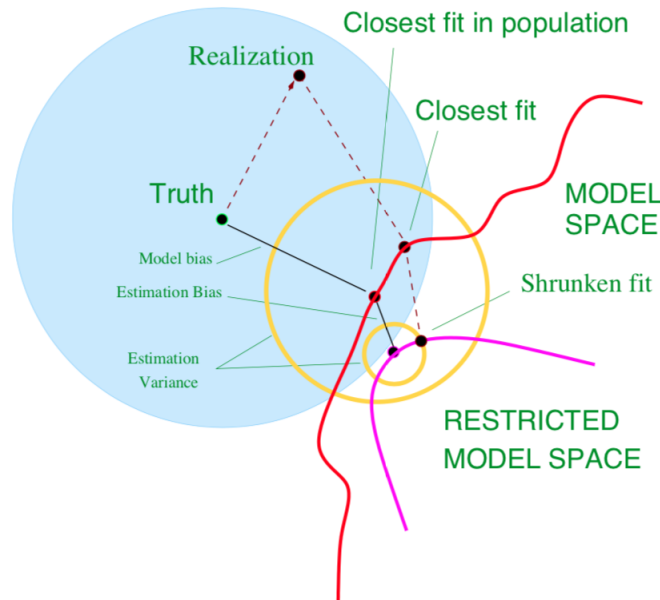


Figura 3.4

Nel grafico 3.4 si vede che quando si parte da un sistema con una certa previsione per un dato set di input x_i . Se ci si muove nello spazio dei modelli, i campionamenti a x_i non danno mai il valore vero, ma sono soggetti ad un errore ε (il campionamento si troverà all'interno della zona azzurra). Quando facciamo il fit di un modello, per cui scegliamo un particolare spazio dei modelli su cui lavorare, muovendoci nello spazio dei modelli cambiamo le predizioni che il modello può dare, e ne cerchiamo uno che è il più vicino possibile alla realizzazione del sistema.

Esiste la *closest fit in population*, che è il fit del modello che è il più vicino possibile alla distribuzione vera dei dati (linea rossa). Normalmente questo modello ha una varianza, per cui può variare nello spazio (lungo la linea gialla più estesa). Minore è la varianza, maggiormente deterministico sarà il modello, ma il risultato può essere differente dal valore vero per il Bias o perché il modello non riesce a fittare bene il

dataset. Quando il modello diventa abbastanza malleabile da poter fittare di più e quindi arrivare in posti più vicini, è più facile fare un fit che si avvicini alla realizzazione (campionamento) o al valore vero, ma la sua varianza tende a crescere e l'errore totale è comunque grande.

In genere quando si va a restringere lo spazio dei modelli, *restricted model space* (in genere Ridge e LASSO), si prende un modello che è un pochino differente dal modello non ristretto, ma si riesce a contenere la varianza del modello (ad esempio i β non esplodono a valori infiniti) e, anche se in media il modello non è molto vicino alla realtà, visto che l'errore dovuto alla varianza è piccolo si riesce comunque a stare nei pressi del valore vero (nella zona azzurra). Per questo motivo a volte è meglio avere un modello meno plastico ma che abbia poca varianza, in modo da contenere gli errori ed avere fit controllati.

Esempio

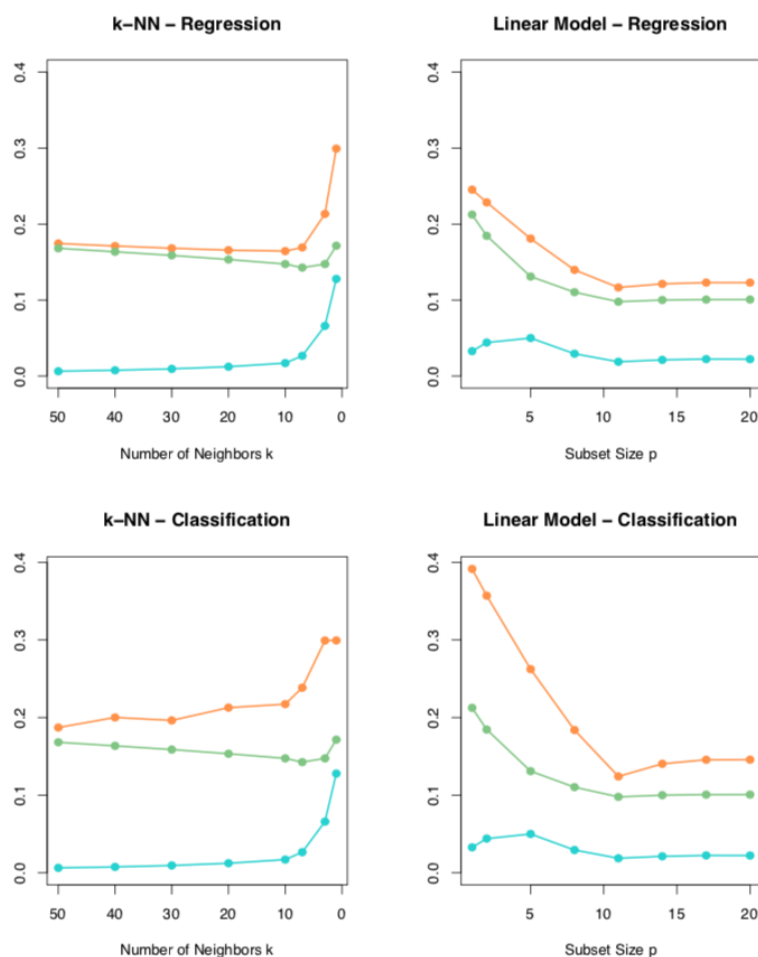


Figura 3.5

Consideriamo due sistemi differenti, mostrati in figura 3.5, in cui si hanno tot

dati di input. Abbiamo un problema di regressione ed un problema di classificazione esaminati tramite regressione lineare e tramite metodo dei prossimi vicini.

Il primo è un modello di classificazione con dimensione di variabili di input molto grande (500 o più):

$$Y = f(\underline{x}_i) = \begin{cases} 0 & \text{se } X_{i1} < \frac{1}{2} \\ 1 & \text{se } X_{i1} \geq \frac{1}{2} \end{cases}$$

Nel secondo modello si prende la media dei primi 10 punti di input e se il risultato è maggiore o minore di 5, si assegna un valore a Y :

$$Y = f(\underline{x}_i) = \begin{cases} 0 & \text{se } \sum_{j=1}^{10} X_{ij} < 5 \\ 1 & \text{se } \sum_{j=1}^{10} X_{ij} \geq 5. \end{cases}$$

Nell'implementazione del fitting usiamo due modelli differenti: il modello di regressione ed il modello di classificazione, che non è fatta con il metodo dei minimi quadrati.

Per il problema di regressione con i minimi quadrati, si ha un Bias (verde) che diminuisce con l'aumento dei gradi di libertà e una varianza che cresce.

Nel caso del problema di classificazione, il fit non è basato sui minimi quadrati: Bias e varianza (azzurro) nei due casi sono uguali, ma l'errore è differente nella classificazione, non è più la somma di Bias e varianza che avevamo ricavato assumendo il metodo dei minimi quadrati.

3.4 Metodi di Ricampionamento

I metodi di campionamento sono utili quando ci si trova nella situazione poco ottimale in cui si hanno dati che non sono sufficienti a fare una statistica precisa. Raramente si hanno abbastanza dati da poterne considerare solo una parte per il modello (dataset di training) e poi poter utilizzare gli altri per validation e test. Le tecniche di ricampionamento consentono di utilizzare l'intero dataset per fare training, validation e testing ma senza che questo influisca sul modello o impedisca un corretto funzionamento del modello.

Nel seguito descriviamo tre tecniche di ricampionamento diverse, di cui i primi due sono algoritmi che si possono applicare pressochè sempre, perché sono basati in un trattamento separato dei dati che non dipende dal modello: validazione incrociata (*cross-validation*), bootstrap e Jack-knife, l'ultimo metodo è una sorta di precursore dei primi due metodi ed è una sorta di via di mezzo tra le altre due tecniche.

3.4.1 Cross-validation

L'idea nel *k-fold cross validation* è quella di prendere tutto il set di dati di campionamento e dividerlo in K parti più o meno della stessa numerosità.

Un gruppo si tiene per la validazione, mentre gli altri sono utilizzati per il training. Si ottengono, quindi, K diversi training set. Si assegna ad ognuno degli N campionamenti uno dei K gruppi

$$\kappa : \{1, \dots, N\} \rightarrow \{1, \dots, K\}$$

Successivamente si fitta la funzione $\hat{f}^{-k}(x)$ sui $K - 1$ blocchi (tutti i blocchi meno il κ -esimo), quindi $\hat{f}^{-k}(x)$ è il modello ottimizzato sul set di punti che esclude il κ -esimo batch.

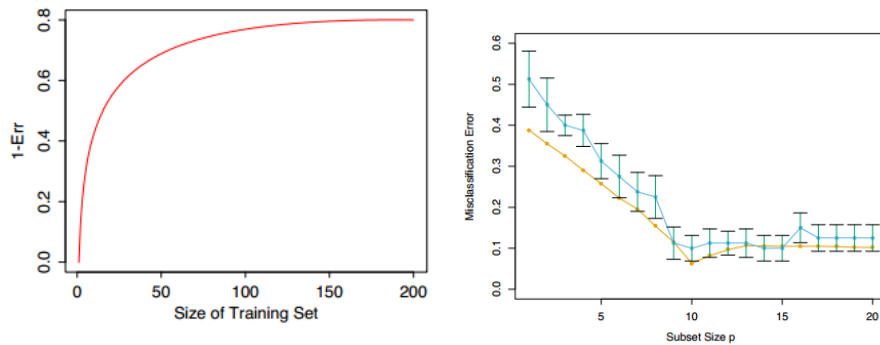
Una volta calcolato il fit, si valuta la media degli errori delle K funzioni calcolate (escludendo di volta in volta un batch diverso dal training)

$$CV(\hat{f}) = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}^{-\kappa(i)}(x_i))$$

e se si ha un modello che dipende anche dal parametro α che riguarda il numero di punti per ciascun batch, a questo punto l'errore è

$$CV(\hat{f}, \alpha) = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}^{-\kappa(i)}(x_i, \alpha))$$

Se si utilizzasse $K = N$, per cui tutti i punti eccetto uno fanno parte del dataset di training, si fitta il modello e poi si valuta l'errore sul dato rimosso.



(a) Ipotetica curva di apprendimento per un classificatore su un determinato compito: un grafico di 1_{Err} in funzione della dimensione del set di addestramento N . Con un dataset di 200 dati, la cross validazione a 5 fold utilizza set di training di dimensione 160, che si comportano molto simili all'intero set.

(b) La ripetizione dell'apprendimento N volte è onerosa in termini computazionali a causa della somiglianza tra i "set di addestramento" N . L'onere computazionale è anche considerevole, richiedendo N applicazioni dell'apprendimento.

In figura 3.6b si confrontano gli errori in rapporto al modello lineare nel problema di classificazione menzionato in precedenza (figura 3.5 in basso a destra). Ogni training parziale e la propria validazione hanno un proprio errore; più blocchi ci sono più la statistica sarà buona. Si vede che la curva azzurra segue molto bene l'errore di previsione; da questo si cerca il minimo, che corrisponde più o meno al minimo vero del modello e si riesce a fare la validazione del modello trovando il modello migliore.

Scegliere quanti blocchi fare non è ovvio. Più blocchi ci sono più la statistica è fine, ma c'è un *overread* di calcolo maggiore (bisogna rifare i training moltissime volte), mentre se si ha un numero minore si riesce a lavorare meglio ed in certi casi è più efficiente (di solito $K = 5, 6, 7$ è un buon numero, ma dipende anche da quanti dati si hanno). L'errore che si commette nella stima dell'errore (segmenti nell'asse verticale di figura 3.6b) dipende sia dal numero di punti N , sia dal numero di blocchi K .

Immaginiamo di stare facendo un'analisi medica di 5000-10000 geni differenti per un certo numero di persone, anche non eccessiva, con un output di "sano" o "malato".

Si ha un'enorme quantità di dati, per cui non tutti hanno influenza, ma per cui magari solo un centinaio sono importanti. Prima di fare il test, dobbiamo capire quali siano i parametri importanti: possiamo stilare una matrice di correlazione (o di covarianza) tra gli x_i e y . Tra tutti i dati, possiamo prendere quei 100 geni che sono davvero correlati con l'output. Riduciamo quindi i dati ed abbiamo un set di campionamenti con cui andiamo a fare la statistica.

Questo approccio non funziona, poiché nella selezione si è già fatta una sorta di training, quindi quando si dividono poi i dati nei k gruppi, essi saranno correlati, e quindi il modello è fittato su qualcosa che ha un Bias molto grande.

Ciò che bisogna fare invece è guardare dalla matrice solo le correlazioni tra le x_i , non considerando l'output y . Dalla matrice si scelgono gruppi di predittori non correlati tra di loro, da cui si prende un unico rappresentante.

Una volta scelti i geni (non avendo considerato y , abbiamo ridotto il dataset e possiamo fare il cross-validation. Funziona perché abbiamo un dataset con input non correlati tra di loro e per cui abbiamo ridotto i gradi di libertà, e dopo facciamo un fit tra dati di input e l'output. Se andiamo a prendere input che sono già correlati con l'output si perde la base del machine learning, perché si fa un fit con qualcosa che ha perso già parte delle sue opzioni.

Immaginiamo 5000 predittori distribuiti secondo una gaussiana, da cui se ne selezionano 100, con output 0 o 1, tutto quanto incorrelato. Siccome la Y non è correlata con alcun input, qualunque modello si trovi a partire dal fit avrà un errore sulla Y del 50%, poiché qualunque sia la predizione, ci sono due possibilità con la stessa percentuale di accadere. Applicando la procedura sbagliata, per cui si cercano correlazioni tra X e Y , l'errore in media è intorno al 30% e la distribuzione dei dati è un po' sfalsata (vedi figura 3.7 in alto).

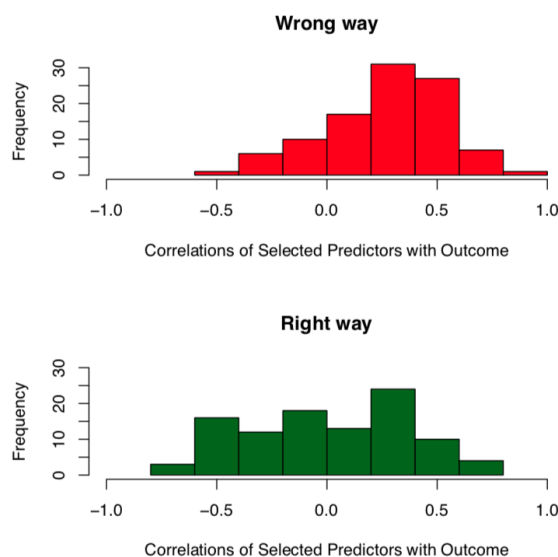


Figura 3.7

Se si fa la selezione di 100 input senza considerare la Y e poi si fa un cross validation, la distribuzione dei risultati è abbastanza uniforme e l'errore medio è effettivamente il 50% (vedi figura 3.7 in basso). Quando si fanno selezione di dati prima del fit, è

importante stare attenti a non introdurre un Bias guardando le correlazioni tra input e output.

Leave One Out Cross Validation

Il *LOOCV*, ovvero quando $K = N$, è quello che dà l'errore minore sulla predizione dell'errore, la varianza minore sull'errore previsto, ma dà un sacco di lavoro, perché bisogna fare N fit. Ci sono dei modelli in cui le predizioni sul set di input possono essere scritte come un prodotto di matrici

$$\hat{y} = Sy$$

dove S è una matrice $n \times n$ che dipende dai vettori di input x_i , ma non dagli output, si può vedere che la traccia di S è il numero effettivo di parametri che si stanno utilizzando (visto nella regressione lineare, nel metodo di Ridge) e si può dimostrare che per una classe di questi metodi l'errore quadratico medio calcolato sul LOOCV, può essere riscritto come l'errore quadratico medio su tutti i punti, riscalati per $1 - \#(\text{gradi di libertà})$:

$$\frac{1}{N} \sum_{i=1}^N [y_i - \hat{f}^{-i}(x_i)]^2 = \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i - \hat{f}(x_i)}{1 - S_{ii}} \right]^2$$

Si può stimare l'errore calcolato sul fit fatto con un punto in meno usando l'errore per il fit di tutti i dati e riscalando in questo modo. Questa approssimazione è la *generalized cross-validation*, in cui si stima l'errore in questo modo, con meno lavoro e computazioni più veloci:

$$\text{GCV}(\hat{f}) = \frac{1}{N} \sum_{i=1}^N \left[\frac{y_i - \hat{f}(x_i)}{1 - \text{Tr}(S)/N} \right]^2.$$

3.4.2 Bootstrap

Se abbiamo pochi campionamenti, le varianze crescono perché non abbiamo abbastanza dati su cui lavorare e dividere in K gruppi diventa complesso, poiché i dati non sono sufficienti per fare una statistica corretta. L'idea è di prendere a caso ed uniformemente N punti: alcuni dei dati compariranno più di una volta nell'estrazione.

A partire dal dataset si fanno B bootstrap sample di N punti casuali da cui si fitta il modello. Se abbiamo una variabile statistica con cui vogliamo lavorare, questa dipenderà dal campione di Bootstrap utilizzato (vedi figura 3.8). Se calcoliamo la media campionata

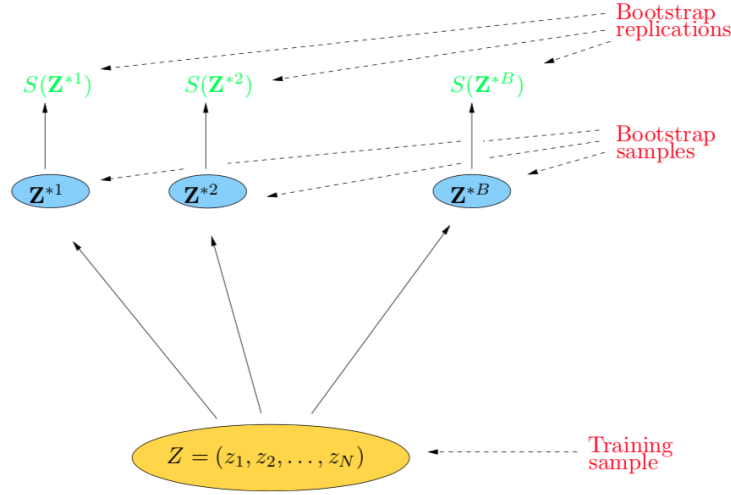
$$\bar{S}^* = \sum_b S(\mathbf{Z}^{*b})/B$$

nel limite di un numero infinito di campioni, questo sarà il valore corretto, e allo stesso modo la varianza sarà:

$$\widehat{\text{Var}}[S(Z)] = \frac{1}{B-1} \sum_{b=1}^B (S(Z^{*b}) - \bar{S}^*)^2,$$

dove si divide per $B-1$ perché si utilizza la media campionata, quindi i gradi di libertà diminuiscono di 1. L'errore di *bootstrap* per il fit viene calcolato come la media di $L(Y, \hat{f})$:

$$\widehat{\text{Err}}_{\text{boot}} = \frac{1}{B} \frac{1}{N} \sum_{b=1}^B \sum_{i=1}^N L(y_i, \hat{f}^{*b}(x_i)).$$



*Schematic of the bootstrap process. We wish to assess the statistical accuracy of a quantity $S(\mathbf{Z})$ computed from our dataset. B training sets $\mathbf{Z}^{*b}$, $b = 1, \dots, B$ each of size N are drawn with replacement from the original dataset. The quantity of interest $S(\mathbf{Z})$ is computed from each bootstrap training set, and the values $S(\mathbf{Z}^{*1}), \dots, S(\mathbf{Z}^{*B})$ are used to assess the statistical accuracy of $S(\mathbf{Z})$.*

Figura 3.8: Schema del processo bootstrap. Vogliamo valutare l'accuratezza statistica di una quantità $S(Z)$ calcolata dal nostro dataset. Vengono estratti con rimpiazzo dal dataset originale B set di addestramento Z^{*b} , $b = 1, \dots, B$ ciascuno di dimensione N . La quantità di interesse $S(Z)$ viene calcolata da ciascun set di addestramento bootstrap, e i valori $S(Z_1^*), \dots, S(Z_B^*)$ vengono utilizzati per valutare l'accuratezza statistica di $S(Z)$.

3.4.3 Jackknife

Il metodo *jackknife* è simile al LOOCV, ma funziona in maniera opposta. Si prende una serie di sottoinsiemi x^{-i} in cui si sommano tutti i campionamenti meno l' i -esimo:

$$x^{-i} = \{x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n\}$$

e si calcolano le medie dei vari sottoinsiemi

$$\bar{x}_i = \frac{1}{n-1} \sum_{j=1, j \neq i}^n x_j;$$

la media totale dei campionamenti sarà

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n \bar{x}_i \left(= \frac{1}{n} \sum_{i=1}^n x_i \right).$$

La stima della varianza sarà

$$\text{Var}(\bar{x}) = \frac{n-1}{n} \sum_{i=1}^n (\bar{x}_i - \bar{x})^2 = \frac{1}{n(n-1)} \sum_{i=1}^n (x_i - \bar{x})^2;$$

a differenza del cross validation, in cui si fanno n fit da cui è stato rimosso ad ognuno un punto e in cui si utilizza il punto rimosso per calcolare l'errore del fit e la media degli errori, il jackknife fa un set di n medie calcolate con $n - 1$ punti con cui trova la media generale e trova la varianza sui punti calcolati.

Capitolo 4

Metodi di Classificazione Lineare

Ci sono tre tipi di metodi di classificazione importati, che sono via via più complicati: la regressione lineare e la matrice di indice, l'analisi dei discriminanti (lineari e quadratici) e la regressione logistica, che è molto simile all'analisi dei determinanti, ma è più sofisticata.

I metodi di classificazione sono molto utili per un grande numero di motivi: per le svariate applicazioni vere e proprie di classificazione (ad esempio casi clinici, studi delle pubblicità sui social network, ecc...) e per la regressione logistica. Infatti essa è basata sull'uso del *sigmoide* (utile per le reti neurali) e non può essere risolta analiticamente o algebricamente, quindi bisogna iniziare a scovare metodi che si possono usare per minimizzare i modelli, indipendenti da formulazioni analitiche. La regressione logistica è un sistema ancora semplice che può essere studiato con pochi predittori, con pochi input, e quindi ci dà la possibilità di vedere più nel dettaglio metodi come la *discesa stocastica del gradiente* e capirli più nel profondo, prima che possano essere applicati a problemi molto più complessi.

4.1 Regressione Lineare e Matrice di Indice

Assumiamo di avere una serie di K classi che vogliamo classificare ed una serie di input $(\underline{x}_i)_j$ con $j = 1, \dots, p$ ed un output Y_k con $k = 1, \dots, K$. L'output è un vettore di K componenti, e per ogni vettore $\underline{y}_i$ si ha

$$(\underline{y}_i)_k = \begin{cases} 1 & \text{se } k \text{ è verificato} \\ 0 & \text{se } k \text{ non è verificato} \end{cases}$$

dove un'unica componente è pari a 1 e il restante delle componenti è 0 (esempio del codice postale: ogni simbolo verrà riconosciuto come solo una delle possibili lettere o numeri, non di più). Possiamo quindi fare una regressione lineare, in cui la matrice dei responsi, che prima era una colonna di valori

$$y = \begin{pmatrix} y_0 \\ \vdots \\ y_n \end{pmatrix}$$

è ora una matrice

$$\mathbf{Y} = \begin{pmatrix} \underline{y}_1^T \\ \vdots \\ \underline{y}_n^T \end{pmatrix}$$

dove ogni riga della matrice è il set di risultati sulle varie classificazioni. Per ogni colonna, il set di parametri β è ora una matrice $\hat{\mathbf{B}}$ di $(p+1) \times K$ componenti:

$$\hat{\mathbf{B}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y};$$

la predizione che si fa una volta che si ha $\hat{\mathbf{B}}$ sugli stessi punti del training set è:

$$\hat{\mathbf{Y}} = \mathbf{X} \hat{\mathbf{B}} = \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}.$$

Le due matrici $\mathbf{Y}$ e $\hat{\mathbf{Y}}$ non saranno uguali, in primo luogo perché le predizioni possono essere sbagliate, ma soprattutto perché le previsioni non saranno mai unicamente 0 e 1 (ma si può provare che la somma su tutti i K deve essere pari a 1), ma potremmo avere con $K = 4$, ad esempio:

$$\mathbf{Y} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

e come previsione potremmo avere:

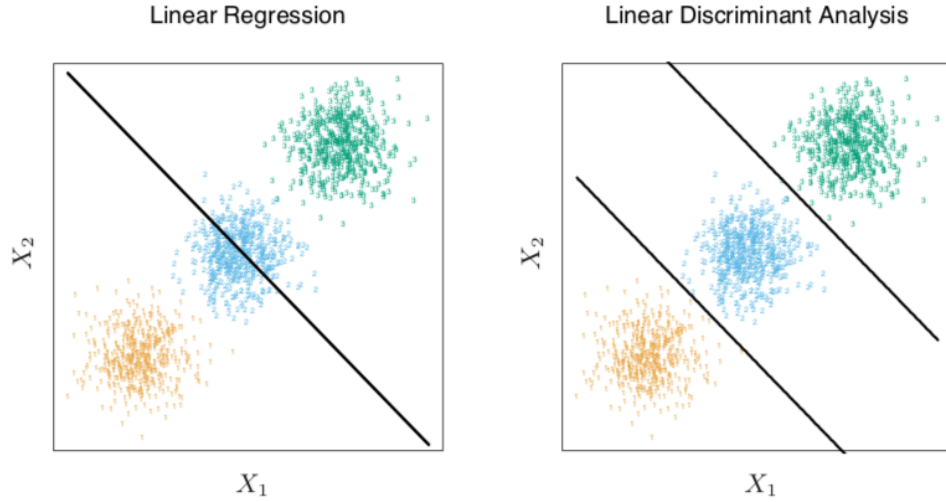
$$\hat{\mathbf{Y}} = \begin{pmatrix} 0.1 & 0.2 & 0.8 & -0.1 \\ 0 & 0.85 & 0.1 & 0.05 \\ -0.1 & 1.2 & -0.1 & 0 \\ 0.15 & 0.05 & 0.2 & 0.6 \\ 0.45 & 0.5 & 0 & 0.05 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}.$$

Dalla previsione delle probabilità si prendono, ad esempio, i valori più grandi per ogni riga, che sono le probabilità che il modello finisca in una classe piuttosto che in un'altra. Si cerca di stimare la probabilità che per quel dataset di input la classe sia una delle K possibili (in questo caso il modello avrebbe funzionato per la prima, la terza e la quarta riga).

Dato un input, si estrae un vettore K : $\hat{f}(x)^T = (1, x^T) \hat{\mathbf{B}}$; la previsione $\hat{G}(x)$ è quella per cui la funzione di risposta che si ottiene è maggiore: per un valore x_i la previsione è un set di K numeri $\hat{f}(x_i, \hat{\mathbf{B}})$, tra cui scegliamo quello massimo

$$\hat{G}(x) = \operatorname{argmax}_k \hat{f}_k(x).$$

Quando ci si trova con un numero di classi maggiore di 2, il fit lineare in molti casi non funziona. Infatti, se si ha la matrice $\hat{\mathbf{B}}$ come una combinazione lineare dei valori di input (ad esempio x_1 e x_2), ci saranno dei limiti in cui si passa da una classe ad un'altra. Siccome la funzione $\hat{f}(x)^T$ è lineare, i limiti di separazione tra le regioni sono delle linee rette (come in figura 4.1).



The data come from three classes in $\mathbb{R}^2$ and are easily separated by linear decision boundaries. The right plot shows the boundaries found by linear discriminant analysis. The left plot shows the boundaries found by linear regression of the indicator response variables. The middle class is completely masked (never dominates).

Figura 4.1

Se abbiamo tre classi o più, ma con un numero di variabili limitato (x_1 e x_2), facendo un fit si ottiene una combinazione lineare di due variabili, che è una retta. Si divide in due lo spazio dei modelli: da una parte la previsione per la classe 1 e dalla parte opposta quella della classe 3, ma la classe 2 non è mai scelta (metà sono contati come 1 e metà come 3). Si può risolvere solo se si hanno funzioni paraboliche, in cui l'intersezione con il piano dei predittori avviene più di una volta. Per avere una funzione del genere, abbiamo bisogno di mescolare potenze dei predittori, come x_1^2 , x_2^2 e x_1x_2 .

Quello che si vorrebbe avere è un modello un pochino più sofisticato, ma che ci dia una visione più approfondita del sistema. Un modo è lavorare con modelli che ci diano una probabilità di essere in una determinata classe o in un'altra, dato un certo input. La funzione $\hat{f}_k(\underline{x}) = (1, \underline{x}^T) \hat{\mathbf{B}}$ può essere interpretata come la probabilità $p(k)$ che la classe corretta per il valore di input sia la classe k . Ciò che ci può interessare può essere la distribuzione di probabilità: dato l'input $\underline{x}$, qual è la probabilità che il campionamento dia la classe k . Non è necessariamente la stessa cosa di prima, poiché si può avere tante volte lo stesso input $\underline{x}$ associato a k diversi.

4.2 Analisi del Discriminante

4.2.1 Discriminante Lineare

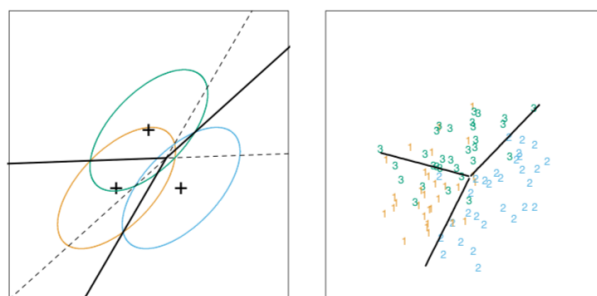
Dato il vettore di input dei predittori, qual è la probabilità che un campionamento risulti in una classe $G(\underline{x}) = 1, 2, \dots, K$? Possiamo usare il teorema di Bayes:

$$P(A, B_i) = \frac{P(B_i, A)P(A)}{\sum_i P(B_i|A)P(B_i)};$$

scriviamo la probabilità di avere una classe k a partire da un input $\underline{x}$:

$$P(G = k | \underline{X} = \underline{x}) = \frac{f_k(\underline{x})\pi_k}{\sum_{\ell=1}^K f_\ell(\underline{x})\pi_\ell},$$

dove π_k è la probabilità a priori della classe k . Questa espressione ci dice che avere la funzione $f_k(\underline{x})$ (cosa che stimavamo con la regressione lineare appena fatta) ci informa sulla probabilità vera che stiamo cercando.



The left panel shows three Gaussian distributions, with the same covariance and different means. Included are the contours of constant density enclosing 95% of the probability in each case. The Bayes decision boundaries between each pair of classes are shown (broken straight lines), and the Bayes decision boundaries separating all three classes are the thicker solid lines (a subset of the former). On the right we see a sample of 30 drawn from each Gaussian distribution, and the fitted LDA decision boundaries.

Figura 4.2

In questa analisi si prende una parametrizzazione della funzione $f_k(\underline{x})$, ovvero: una volta che la classe è k , qual è la probabilità che questa venga da $\underline{x}$? Se, ad esempio, $k = k_i$, allora $\underline{x}$ sarà distribuita attorno ad un certo valore $\underline{\mu}_k$; se consideriamo l'esempio del diabete, stiamo cercando la probabilità che se troviamo una certa quantità di zucchero nel sangue (l'analogo di $\underline{\mu}_k$) il paziente sarà diabetico (non lo è necessariamente, potrebbe essersi mangiato una torta prima del test). Quindi modelliamo la funzione $f_k(\underline{x})$, che è ciò che ci dà la probabilità di avere un predittore attorno a $\underline{\mu}_k$, assumendo che la funzione sia una gaussiana multivariata

$$f_k(\underline{x}) = \frac{1}{(2\pi)^{p/2} |\Sigma_k|^{1/2}} e^{-\frac{1}{2}(\underline{x}-\underline{\mu}_k)^T \Sigma_k^{-1} (\underline{x}-\underline{\mu}_k)}$$

ovvero una gaussiana in cui la varianza è la diagonale della matrice di correlazione: $\Sigma_k = \text{Diag}(\text{Var}) = \text{Diag}(\sigma_k^2)$. Se le direzioni sono correlate, la distribuzione dei

valori assume la forma di un ellissoide (in 2 dimensioni) o di un iperellissoide (in più dimensioni).

Ciò che vogliamo fare è usare questa funzione di probabilità, che indica come si distribuiscono le $\underline{x}$ che appartengono alla classe k , per predire la probabilità che a partire da un predittore $\underline{x}$ io abbia una classe k_1 piuttosto che un'altra classe k_2 . Non ci interessano le probabilità precise $P(k_1|\underline{x})$ e $P(k_2|\underline{x})$, ma ci interessa il loro rapporto:

$$\log \frac{P(G = k|\underline{X} = \underline{x})}{P(G = \ell|\underline{X} = \underline{x})}.$$

A priori, stiamo utilizzando una funzione multivariata normale, quindi stiamo assumendo che le varie variabili siano correlate dalla matrice di correlazione $|\Sigma_k|$. È la distribuzione più semplice che si può immaginare di utilizzare, fissando media e varianza.

Basandoci sulla distribuzione multivariata, usiamo il teorema di Bayes per trovare il rapporto tra le due probabilità (che ci rimuove la normalizzazione):

$$\begin{aligned} \log \frac{P(G = k|\underline{X} = \underline{x})}{P(G = \ell|\underline{X} = \underline{x})} &= \log \frac{f_k(\underline{x})}{f_\ell(\underline{x})} + \log \frac{\pi_k}{\pi_\ell} \\ &= \log \frac{\pi_k}{\pi_\ell} - \frac{1}{2}(\underline{x} - \underline{\mu}_k)^T \Sigma_k^{-1} (\underline{x} - \underline{\mu}_k) + \frac{1}{2}(\underline{x} - \underline{\mu}_\ell)^T \Sigma_\ell^{-1} (\underline{x} - \underline{\mu}_\ell) \end{aligned}$$

se questo rapporto è minore di 1 la classe ℓ è più probabile, mentre in caso contrario sarà più probabile la classe k . Il metodo dell'analisi dei discriminanti lineari si ottiene quando si assume che la matrice di correlazione sia costante ed indipendente dalla classe k :

$$\Sigma_k = \Sigma \quad \forall k;$$

se le variabili di input sono correlate tra di loro, ma scorrelate dalla classe, si assume che la matrice di correlazione tra di esse, non dipenda dalla classe k .

Assumendo questo, il rapporto tra le probabilità non è altro che una funzione lineare, poiché i termini quadratici si cancellano nella relazione $(\underline{x}^T \Sigma^{-1} \underline{x})$ e rimangono i termini misti:

$$\log \frac{P(G = k|\underline{X} = \underline{x})}{P(G = \ell|\underline{X} = \underline{x})} = \log \frac{\pi_k}{\pi_\ell} - \frac{1}{2}(\underline{\mu}_k + \underline{\mu}_\ell)^T \Sigma^{-1} (\underline{\mu}_k - \underline{\mu}_\ell) + \underline{x}^T \Sigma^{-1} (\underline{\mu}_k - \underline{\mu}_\ell).$$

Il rapporto tra le due probabilità non è nient'altro che una funzione che può essere fittata con una funzione lineare

$$\log \frac{P(G = k|\underline{X} = \underline{x})}{P(G = \ell|\underline{X} = \underline{x})} = \beta_{k\ell} + \underline{x}^T \mathbf{B}$$

che giustifica l'uso di un metodo lineare per farne la regressione, che non è necessariamente quello che funziona meglio.

Nel calcolo della distribuzione di probabilità, non si hanno le matrici di correlazione e le medie in sé, perché non si conoscono le funzioni $f_k(\underline{x})$, ma si può ricercare tra le funzioni discriminanti

$$\delta_k(\underline{x}) = \underline{x}^T \Sigma^{-1} \underline{\mu}_k - \frac{1}{2} \underline{\mu}_k^T \Sigma^{-1} \underline{\mu}_k + \log \pi_k$$

qual è quella più grande.

In pratica, il dataset di training è

$$Y, \underline{X} = \begin{pmatrix} y_0 = k & x_{00} & x_{01} & \cdots & x_{0p} \\ y_1 = \ell & x_{10} & x_{11} & \cdots & x_{1p} \\ y_2 = q & x_{20} & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{N-1} = r & x_{N-1,0} & x_{N-1,1} & \cdots & x_{N-1,p} \end{pmatrix}$$

da ogni campionamento si può stimare con facilità la probabilità della k -esima classe come

$$\hat{\pi}_k = \frac{N_k}{N};$$

si può stimare la media dei valori $\underline{x}_i$ come

$$\hat{\underline{\mu}}_k = \sum_{g_i=k} \frac{x_i}{N_k};$$

e si può calcolare

$$\hat{\underline{\Sigma}} = \frac{1}{N - K} \sum_{k=1}^K \sum_{g_i=k} (x_i - \hat{\underline{\mu}}_k)(x_i - \hat{\underline{\mu}}_k)^T.$$

4.2.2 Discriminante Quadratico

Se abbiamo un set di dati molto ampio, avremo tanti casi in cui si verifica un certo valore k o ℓ , quindi si potrebbero fare matrici di covarianza separate per ogni k ; nella regressione lineare, però, vogliamo una matrice di covarianza unica, quindi la stimiamo facendone la media su tutte le classi k possibili.

È possibile calcolare quindi la funzione di discriminazione, usando le medie e la matrice di covarianza, da cui si ottiene un modello lineare, di cui abbiamo un esempio in figura 4.3. In questo caso, si è riuscita a creare una separazione lineare tra le tre classi diverse.

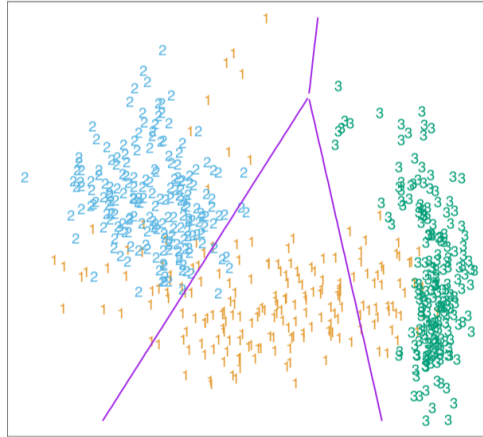


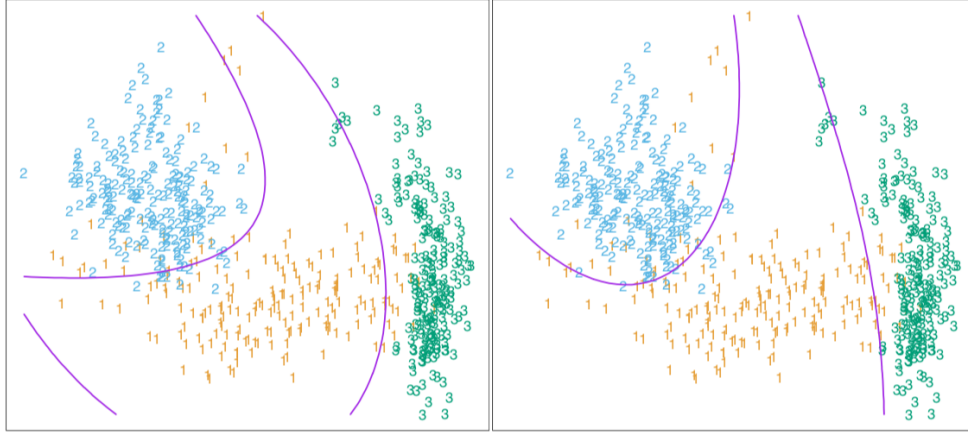
Figura 4.3

L'altro approccio è quello di non limitarsi a calcolare una media delle matrici di covarianza di ogni k , ma di considerare le K matrici di covarianza separatamente. In

questo caso, ci si ritrova a lavorare con una matrice Σ_k campionata separatamente, e da cui bisognerà calcolare il discriminante:

$$\delta_k(\underline{x}) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (\underline{x} - \underline{\mu}_k)^T \Sigma_k^{-1} (\underline{x} - \underline{\mu}_k) + \log \pi_k.$$

In figura 4.4 è mostrata la divisione dei dati tramite utilizzo di discriminanti quadratici, che danno la possibilità di fittare meglio i dati rispetto che tramite discriminanti lineari (figura 4.3).



Two methods for fitting quadratic boundaries. The left plot shows the quadratic decision boundaries for the data in Figure 4.1 (obtained using LDA in the five-dimensional space $X_1, X_2, X_1X_2, X_1^2, X_2^2$). The right plot shows the quadratic decision boundaries found by QDA. The differences are small, as is usually the case.

Figura 4.4

L'analisi del discriminante lineare è molto legata alla regressione lineare, infatti per $K = 2$ nella regressione lineare l'idea di base era che la funzione che associa con più probabilità la classe k alla variabile $\underline{x}_i$

$$G_k(\underline{x}_i) = \hat{y}_i = \hat{\beta}_0 + \hat{\underline{\beta}}^T \underline{x}_i$$

sia basata su una regressione lineare a variabili non discrete. Nella regressione lineare si associa alle $\hat{y}$ un certo valore e si scelgono i β in modo che si minimizzi

$$\hat{\beta}_0, \hat{\underline{\beta}} = \operatorname{argmin}_{\hat{\beta}_0, \hat{\underline{\beta}}} \sum_i [y_i - \hat{y}_i(\beta)]^2.$$

Questo approccio è equivalente ad un'analisi del discriminante lineare, dove al posto di aver scelto una classe k che massimizza il discriminante lineare, si sceglie la classe che massimizza il discriminante lineare per la probabilità della classe stessa (quante volte c'è nei campionamenti):

$$\hat{G}(x) = \operatorname{argmax}_k \left\{ N_k S_k^{LDA}(x) \right\}.$$

Non sono realmente uguali, poiché le scelte non sono uguali, ma proporzionali tra di loro, ma se le classi sono egualmente o similmente probabili, i due modi di fare i fit sono sostanzialmente equivalenti. Se $K \geq 2$, questa equivalenza non vale più.

In figura 4.4 sono mostrati due modi diversi di approcciarsi al problema. Sulla destra vediamo un'applicazione del discriminante quadratico (nelle dimensioni X_1 e X_2). Sulla sinistra, invece, si utilizza l'analisi del discriminante lineare, ma su cinque dimensioni: $X_1, X_2, X_1^2, X_2^2, X_1X_2$. I risultati sono simili, ma andando a vedere l'errore, questo è leggermente minore per l'analisi quadratica, ma fare LDA è normalmente "meno costoso". In generale il conteggio di parametri è

$$n_{\text{par}}(\text{LDA}) = (K-1)(1+p), \quad n_{\text{par}}(\text{QDA}) = (K-1)(1+p(p+3)/2).$$

Perché LDA e QDA funzionano così bene?

Spesso nei fit questi metodi funzionano molto bene e si potrebbe pensare che adottando una $f_k(\underline{x})$ gaussiana multivariata si abbia l'approssimazione migliore (a parità dei primi due momenti): questo è vero in alcuni casi, ma non in generale. Inoltre, abbiamo assunto anche che le matrici di covarianza siano indipendenti ed uguali tra di loro, che non è sempre vero! In realtà si sta introducendo un Bias, perché si fanno assunzioni anche abbastanza forti su f_k e Σ_k che spesso sono corrette, ma che comunque introducono un Bias. I metodi LDA e QDA sono molto stabili, ed in quanto tali, non è facile ottenere fit diversi, le previsioni sono poco dipendenti dal modo in cui viene fatto il fit e dai dati di training. Quindi essi sì, introducono un Bias, ma sono costruiti in modo tale da non lasciare spazio ad overfitting.

4.3 Regressione Logistica

La regressione logistica è simile alla regressione lineare, per cui avevamo:

$$y_i = f(\underline{x}_i) + \varepsilon_i$$

con errore che normalmente si assume distribuito secondo una distribuzione normale. Nel caso della regressione logistica, abbiamo una funzione y_i , che è una regressione sulle probabilità di avere una certa classe k , con errore statistico tipicamente estratto da una distribuzione multinomiale (binomiale nel caso binario):

$$y_i = p(\underline{x}_i) + \tilde{\varepsilon}_i.$$

Nella regressione logistica, si cercano per ogni k probabilità di tipo esponenziale

$$p_k(\underline{x}_i) = p_k(y_i|\underline{x}_i, \underline{\beta}) = \frac{\exp\{\beta_0 + \underline{\beta}^T \underline{x}_i\}}{1 + \exp\{\beta_0 + \underline{\beta}^T \underline{x}_i\}} = p_i.$$

Come mai la probabilità ha questa forma? La risposta deriva dai principi su cui la regressione lineare è costruita.

Consideriamo un set di K classi, con p predittori. Vogliamo utilizzare una parametrizzazione lineare, quindi avremo

$$\underline{\beta}_i = \begin{pmatrix} \beta_0 \\ \underline{\beta} \end{pmatrix}_i = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}_i \implies \mathbf{B} = (\underline{\beta}_0, \underline{\beta}_1, \dots, \underline{\beta}_{K-1}).$$

Ci interessa lavorare con i rapporti tra le probabilità ed assicurarci che le probabilità siano normalizzate correttamente

$$\sum_{k=1}^K p_k(\underline{x}) = 1, \quad \forall \underline{x} \in \mathbb{R}^p.$$

Per la regressione logistica assumiamo che

$$\begin{aligned} \log \frac{P(G=1|\underline{X}=\underline{x})}{P(G=K|\underline{X}=\underline{x})} &= \beta_{10} + \underline{\beta}_1^T \underline{x} \\ \log \frac{P(G=2|\underline{X}=\underline{x})}{P(G=K|\underline{X}=\underline{x})} &= \beta_{20} + \underline{\beta}_2^T \underline{x} \\ &\vdots \\ \log \frac{P(G=K-1|\underline{X}=\underline{x})}{P(G=K|\underline{X}=\underline{x})} &= \beta_{K-1,0} + \underline{\beta}_{K-1}^T \underline{x} \end{aligned}$$

Non è necessario scrivere i rapporti di probabilità tra tutte le classi, ma bastano i $K-1$ rapporti appena scritti, infatti se volessimo valutare il rapporto tra le probabilità di due classi $k, \ell \neq K$, basterebbe

$$\begin{aligned} \log \frac{P(G=k|\underline{X}=\underline{x})}{P(G=\ell|\underline{X}=\underline{x})} &= \log \frac{P(G=k|\underline{X}=\underline{x})}{P(G=K|\underline{X}=\underline{x})} - \log \frac{P(G=\ell|\underline{X}=\underline{x})}{P(G=K|\underline{X}=\underline{x})} \\ &= (\beta_{k0} - \beta_{\ell 0}) + (\underline{\beta}_k - \underline{\beta}_\ell)^T \underline{x}. \end{aligned}$$

La funzione di probabilità può essere estratta in maniera abbastanza veloce, infatti, se chiamiamo $p_k = P(G=k|\underline{X}=\underline{x})$ avremo che

$$\frac{1}{p_K} = \sum_{\ell=1}^K \frac{p_\ell}{p_K} = \sum_{\ell} \exp \left\{ \log \frac{p_\ell}{p_K} \right\} = 1 + \sum_{\ell=1}^{K-1} e^{\beta_{\ell 0} + \underline{\beta}_\ell^T \underline{x}},$$

da cui si estrae facilmente la normalizzazione (utilizzando $p_1(\underline{x}) = p_1(\underline{x})p_K(\underline{x})/p_K(\underline{x})$):

$$\begin{aligned} P(G=1|\underline{X}=\underline{x}) &= \frac{\exp \{ \beta_{10} + \underline{\beta}_1^T \underline{x} \}}{1 + \sum_{\ell=1}^{K-1} \exp \{ \beta_{\ell 0} + \underline{\beta}_\ell^T \underline{x} \}} \\ P(G=k|\underline{X}=\underline{x}) &= \frac{\exp \{ \beta_{k0} + \underline{\beta}_k^T \underline{x} \}}{1 + \sum_{\ell=1}^{K-1} \exp \{ \beta_{\ell 0} + \underline{\beta}_\ell^T \underline{x} \}} \\ P(G=K|\underline{X}=\underline{x}) &= \frac{1}{1 + \sum_{\ell=1}^{K-1} \exp \{ \beta_{\ell 0} + \underline{\beta}_\ell^T \underline{x} \}}. \end{aligned}$$

Come possiamo ottimizzare questo tipo di regressione? Il metodo è abbastanza complesso: innanzitutto dobbiamo vedere un principio, in cui riconosciamo una probabilità più realistica e poi si massimizza questa, ovvero applichiamo il metodo della *massima verosimiglianza*.

Immaginiamo di avere 2 parametri, per cui $y_i = 0, 1$ ed il dominio è

$$\mathcal{D}_{\text{train}} = \{ (y_i, x_i) : x_i \in \mathbb{R}^p; y_i \in \mathbb{R}^2 \},$$

e solo due parametri $\underline{\beta}$, avremo

$$P(y_i = 1|\underline{x}_i, \hat{\beta}) = \frac{\exp(\beta_0 + \beta_1^T \underline{x}_i)}{1 + \exp(\beta_0 + \beta_1^T \underline{x}_i)}$$

$$P(y_i = 0|\underline{x}_i, \hat{\beta}) = 1 - P(y_i = 1|\underline{x}_i, \hat{\beta}).$$

Cerchiamo ora il principio della *Maximum Likelihood Estimation* (o MLE), ovvero cerchiamo la probabilità $p(\mathcal{D}|\hat{\beta})$ di avere un certo dominio $\mathcal{D}$ una volta scelto il parametro β :

$$\hat{\beta} = \underset{\beta}{\operatorname{argmax}} p(\mathcal{D}|\beta),$$

dove¹

$$p(\mathcal{D}|\hat{\beta}) = \prod_{i=1}^n [p(y_i = 1|\underline{x}_i, \hat{\beta})]^{y_i} [1 - p(y_i = 1|\underline{x}_i, \hat{\beta})]^{1-y_i};$$

la funzione di costo sarà una funzione che si massimizza insieme alla probabilità, ovvero:

$$C(\hat{\beta}) = \sum_{i=1}^n (y_i \log p(y_i = 1|\underline{x}_i, \hat{\beta}) + (1 - y_i) \log[1 - p(y_i = 1|\underline{x}_i, \hat{\beta})])$$

$$= \sum_{i=1}^n (y_i(\beta_0 + \beta_1^T \underline{x}_i) - \log(1 + \exp(\beta_0 + \beta_1^T \underline{x}_i)));$$

analogamente, la si può definire in modo che si minimizzi quando la probabilità è massima, aggiungendo un segno meno davanti.

Alla funzione di costo si possono aggiungere termini in λ

$$C(\hat{\beta}) = \sum_{i=1}^n (y_i(\beta_0 + \beta_1^T \underline{x}_i) - \log(1 + \exp(\beta_0 + \beta_1^T \underline{x}_i))) + \lambda \sum_{p=0}^p \|\beta_p\|^m$$

con $m = 1, 2, \dots$, che per $m = 1$ ed $m = 2$ corrispondono al metodo di Lasso ed il metodo di Ridge.

La soluzione della funzione di costo (da minimizzare) nel caso di $p = 1$ e $K = 2$, si ottiene

$$\frac{\partial C(\hat{\beta})}{\partial \beta_0} = - \sum_{i=1}^n \left(y_i - \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right),$$

$$\frac{\partial C(\hat{\beta})}{\partial \beta_1} = - \sum_{i=1}^n \left(y_i x_i - x_i \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)} \right).$$

Nel caso più generale, invece, possiamo scrivere:

$$\frac{\partial C(\hat{\beta})}{\partial \hat{\beta}} = -\hat{\mathbf{X}}^T(\hat{\mathbf{y}} - \hat{\mathbf{p}}) \in \mathbb{R}^p,$$

dove $\hat{\mathbf{X}}$ è una matrice di $n \times p$ componenti, che contiene i valori $\underline{x}_i$.

¹Stiamo assumendo che la misura ottenuta di ogni campionamento sia quella più probabile.

L'altro termine alla base dei metodi per minimizzare le funzioni è la derivata seconda, anche chiamata matrice hessiana

$$\begin{aligned}\left(\frac{\partial^2 C(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}^T}\right)_{\ell j} &= H(\hat{\beta})_{\ell j} = \frac{\partial}{\partial \hat{\beta}_j} \left(\frac{\partial C}{\partial \hat{\beta}_\ell} \right) \\ &= -\frac{\partial}{\partial \hat{\beta}_j} \left(\sum_i \underline{x}_i (y_i - p_i) \right) = + \sum_i \frac{\partial}{\partial \hat{\beta}_j} (\underline{x}_i)_\ell p_i \\ &= - \sum_i (\underline{x}_i)_\ell \frac{(1 + \exp\{\dots\}) \exp\{\dots\} (\underline{x}_i)_j - (\exp\{\dots\})^2 (\underline{x}_i)_j}{(1 + \exp\{\dots\})^2}\end{aligned}$$

il numeratore della frazione non è altro che

$$\text{num} = \exp\{\dots\} (\underline{x}_i)_j [1 + \exp\{\dots\} - \exp\{\dots\}] = \exp\{\dots\} (\underline{x}_i)_j$$

e continuando nel conto otteniamo

$$\begin{aligned}\left(\frac{\partial^2 C(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}^T}\right)_{\ell j} &= \sum_i (\underline{x}_i)_\ell \frac{\exp\{\dots\}}{1 + \exp\{\dots\}} \frac{1}{1 + \exp\{\dots\}} (\underline{x}_i)_j \\ &= \sum_i (\underline{x}_i)_\ell p_i (1 - p_i) (\underline{x}_i)_j;\end{aligned}$$

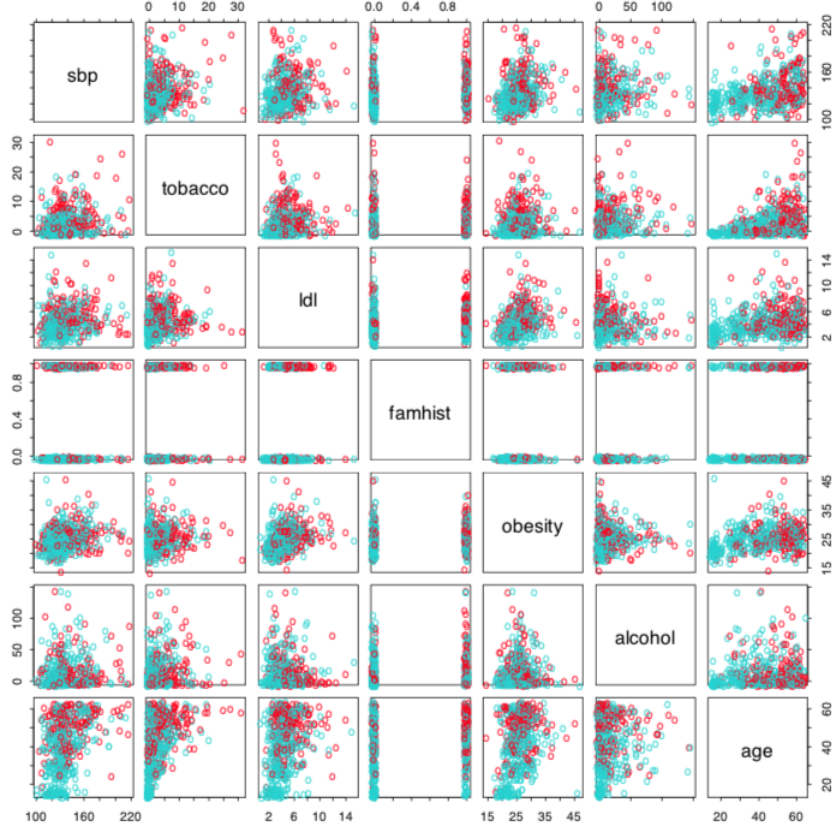
possiamo esprimere l'ultima sommatoria tramite una matrice diagonale in cui ci sono i prodotti delle probabilità di avere la classe 1 o la classe 2, ovvero, se definiamo una matrice diagonale $\hat{W}$ di elementi $p(y_i|x_i, \hat{\beta})(1 - p(y_i|x_i, \hat{\beta}))$ possiamo scrivere:

$$\frac{\partial^2 C(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}^T} = \hat{X}^T \hat{W} \hat{X}.$$

4.3.1 Esempio

Immaginiamo di fare una regressione logistica per un insieme di dati di 160 persone in Sudafrica con 302 controlli che hanno avuto un infarto oppure no. In figura 4.5 è mostrata la matrice di correlazione per 7 parametri scelti (i punti rossi hanno avuto un infarto, mentre i punti azzurri no). Da questo problema, si fa una regressione logistica con due classi (infarto o non infarto).

In figura 4.6 sono gli errori su ogni parametro e lo Z score. Lo Z score è una misura dell'importanza del predittore, misura quanto cambia la probabilità toccando uno dei parametri; uno Z score di circa 2 è pari ad una significatività al 5%. Si vede che per pressione sanguigna e obesità, si vede un fattore Z piccolo o anche negativo, il che implica che siano meno importanti di altri predittori. In realtà sono sì fattori importanti, ma sono molto correlati ad altre cose, quindi il modello ci dà poco peso. Eliminando i predittori con uno Z score piccolo si ottiene un modello in cui ci sono solo i predittori in figura 4.7, ma che sono sufficienti a fare un modello abbastanza accurato.



*A scatterplot matrix of the South African heart disease data. Each plot shows a pair of risk factors, and the cases and controls are color coded (red is a case). The variable family history of heart disease (**famhist**) is binary (yes or no).*

Figura 4.5

Un altro modo per evitare overfitting è tramite metodi di regolarizzazione, come Ridge e LASSO:

$$\begin{aligned}
 C'(\hat{\beta}) &\leftarrow C(\hat{\beta}) + \lambda \sqrt{\hat{\beta}^T \hat{\beta}} & C'' &\leftarrow C(\hat{\beta}) + \lambda \hat{\beta}^T \hat{\beta} \\
 \frac{\partial C'(\hat{\beta})}{\partial \hat{\beta}} &\leftarrow \frac{\partial C(\hat{\beta})}{\partial \hat{\beta}} + \lambda \frac{\hat{\beta}}{|\hat{\beta}|} & \frac{\partial C''(\hat{\beta})}{\partial \hat{\beta}} &\leftarrow \frac{\partial C(\hat{\beta})}{\partial \hat{\beta}} + 2\lambda \hat{\beta} \\
 \frac{\partial^2 C'(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}^T} &\leftarrow H(\hat{\beta}) + \lambda \frac{(I|\hat{\beta}|^2 - \hat{\beta}\hat{\beta}^T)}{|\hat{\beta}|^3} & \frac{\partial^2 C''(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}^T} &\leftarrow H(\hat{\beta}) + 2\lambda I
 \end{aligned}$$

Se si applica il metodo LASSO a questo esempio, si ottiene il grafico in figura 4.8: all'aumentare della norma (quindi con λ piccolo) si ottiene la regressione logistica normale, con tutti i predittori, mentre al diminuire della norma, spariscono alcuni parametri.

Results from a logistic regression fit to the South African heart disease data.

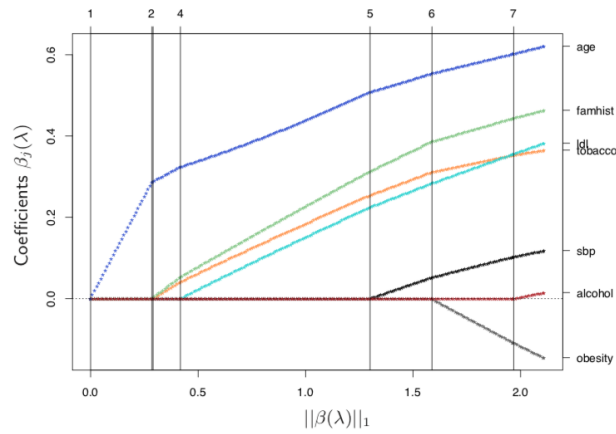
	Coefficient	Std. Error	Z Score
(Intercept)	-4.130	0.964	-4.285
sbp	0.006	0.006	1.023
tobacco	0.080	0.026	3.034
ldl	0.185	0.057	3.219
famhist	0.939	0.225	4.178
obesity	-0.035	0.029	-1.187
alcohol	0.001	0.004	0.136
age	0.043	0.010	4.184

Figura 4.6

Results from stepwise logistic regression fit to South African heart disease data.

	Coefficient	Std. Error	Z score
(Intercept)	-4.204	0.498	-8.45
tobacco	0.081	0.026	3.16
ldl	0.168	0.054	3.09
famhist	0.924	0.223	4.14
age	0.044	0.010	4.52

Figura 4.7



L_1 regularized logistic regression coefficients for the South African heart disease data, plotted as a function of the L_1 norm. The variables were all standardized to have unit variance. The profiles are computed exactly at each of the plotted points.

Figura 4.8

Capitolo 5

Discesa del Gradiente e Metodi di Minimizzazione

Ci troviamo in una situazione in cui abbiamo una funzione di costo $C(\hat{\beta})$ e vogliamo trovare il valore

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} C(\beta).$$

Nella regressione lineare o nei discriminanti lineari e quadratici, ci sono delle espressioni analitiche che ci portano ad ottenere ciò che cerchiamo. Nella regressione logistica non c'è una formula che possiamo utilizzare, abbiamo unicamente la derivata prima e seconda

$$g(\hat{\beta}) = \frac{\partial C(\hat{\beta})}{\partial \hat{\beta}} \qquad H(\hat{\beta}) = \frac{\partial^2 C(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}^T}.$$

Il punto più generale da cui poter partire per trovare metodi di minimizzazione è annullare la derivata prima di $C(\hat{\beta})$. Un primo modo è usare il metodo di Newton-Raphson, per cui si disegna la funzione $f(\beta)$ tale che:

$$\begin{aligned} f(\beta) &= c_1 + c_2 \beta \\ f'(\beta) &= c_2 = g'(\beta) \\ f(\beta_0) &= c_1 + c_2 \beta_0 = g(\beta_0); \end{aligned}$$

se si impone che $f(\beta) = 0$, si ottiene $\beta = -c_1/c_2$. Si ottiene poi che l' i -esima iterazione è

$$\beta_{i+1} = \beta_i - \frac{g(\beta_i)}{g'(\beta_i)}$$

e per infinite iterazioni si arriva alla soluzione. Per stimare i passi successivi, ci serve la derivata della funzione di costo, ovvero l'hessiana. Se sviluppiamo la funzione di costo fino a potenze di 2, ovvero fino a termini lineari in g , e trascuriamo i termini successivi, possiamo utilizzare il metodo di Newton-Raphson. Se la funzione fosse davvero quadratica si potrebbe risolvere esattamente, ma nel caso generale, possiamo solo supporre che nelle vicinanze di un minimo la funzione sia "abbastanza simile" ad una funzione quadratica.

Assumiamo di trovarci in un punto β_n e di voler trovare il modo di andare più direttamente possibile nel valore di minimo $\hat{\beta}$, possiamo sviluppare con Taylor attorno

a β_n :

$$\begin{aligned} C(\beta) &= C(\beta_n) + g^T(\beta_n)(\beta - \beta_n) + \frac{1}{2}(\beta - \beta_n)^T H(\beta_n)(\beta - \beta_n) + O((\beta - \beta_n)^3) \\ &= C(\beta_n) - g^T(\beta_n)\beta_n + \frac{1}{2}\beta_n^T H \beta_n + g^T(\beta_n)\beta - \beta_n^T H(\beta_n)\beta + \frac{1}{2}\beta^T H(\beta_n)\beta \\ &= \tilde{C} + \tilde{b}^T \beta + \frac{1}{2}\beta^T A \beta \end{aligned}$$

con $\tilde{b} = g(\beta_n) - \beta_n^T A$ e $A = H$. Prendendo la derivata rispetto a β otteniamo

$$\begin{aligned} \frac{\partial C}{\partial \beta} &= \underline{b} + \mathbf{A}\underline{\beta} = 0 \\ \underline{\beta}_{n+1} &= -\mathbf{A}^{-1}\underline{b} = -\mathbf{H}^{-1}(g - \mathbf{H}\underline{\beta}_n) \\ &= \underline{\beta}_n - (\mathbf{H}(\underline{\beta}_n))^{-1}g(\underline{\beta}_n) \end{aligned}$$

che è analoga alla situazione in 1 dimensione trattata appena qui sopra.

In molti casi, soprattutto con molti parametri, la matrice hessiana non è facile da invertire; l'inversione di matrice ha un tempo di calcolo dell'ordine di $(\dim(\mathbf{A}))^3$. Per cercare di evitare tempi lunghi, si può provare ad approssimare la matrice hessiana con γI , dove γ è il cosiddetto passo: la minimizzazione non va direttamente verso il minimo, ma va verso il gradiente più veloce, con una velocità scelta dal parametro γ .

Sostanzialmente, dal punto in cui siamo nello spazio dei parametri $\underline{\beta}_n$ si prende il gradiente della funzione di costo e ci si sposta in una direzione che dipende dal gradiente e dalla matrice hessiana. Se ad esempio fossimo in due dimensioni e la nostra funzione fosse quadratica, vicino al minimo avremmo nello spazio $\beta_1 - \beta_2$ delle ellissi concentriche: partendo da un punto β_n su un'ellisse, rimuovendo il prodotto tra gradiente ed hessiana, ci troveremmo, solitamente, su un'ellissi più vicina al punto di minimo. Se la funzione è effettivamente quadratica, la soluzione in un colpo porta al minimo ($\beta_1 \rightarrow \hat{\beta}$). Nel caso la funzione non fosse esattamente quadratica, bisogna fare un certo numero di iterazioni prima di arrivare al punto di minimo. Il problema di questo approccio è il tempo di computazione ad alte dimensioni.

Regressione Logistica

Nel caso generale abbiamo

$$\underline{\beta}_{\text{new}} = \underline{\beta}_{\text{old}} - \left(\frac{\partial^2 C(\underline{\beta})}{\partial \underline{\beta} \partial \underline{\beta}^T} \right)^{-1}_{\underline{\beta}_{\text{old}}} \frac{\partial C(\underline{\beta})}{\partial \underline{\beta}} \Big|_{\underline{\beta}_{\text{old}}}.$$

Per la regressione logistica avevamo calcolato la funzione di costo, presa con il principio di MLE. Avevamo calcolato:

$$\begin{aligned} g(\underline{\beta}) &= -\mathbf{X}^T(\underline{y} - \underline{p}(\underline{\beta})) \\ \mathbf{H}(\underline{\beta}) &= \mathbf{X}^T \mathbf{W} \mathbf{X} \end{aligned}$$

dove $\dim \mathbf{W} = n$ e $\dim \mathbf{H} = p + 1$.

Applicando il metodo di Newton-Raphson otteniamo

$$\begin{aligned} \underline{\beta}_{\text{new}} &= \underline{\beta}_{\text{old}} + (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T (\underline{y} - \underline{p}) \\ &= (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} (\mathbf{X} \underline{\beta}_{\text{old}} + \mathbf{W}^{-1}(\underline{y} - \underline{p})) \\ &= (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \underline{z}, \end{aligned}$$

dove $\underline{z} = \mathbf{X}\underline{\beta}_{\text{old}} + \mathbf{W}^{-1}(\underline{y} - \underline{p})$. Scegliere $\underline{\beta}_{\text{new}}$ a partire da questa formula è equivalente a scegliere $\underline{\beta}_{\text{new}}$ come minimizzazione di

$$\underline{\beta}_{\text{new}} \leftarrow \underset{\underline{\beta}}{\operatorname{argmin}} (\underline{z} - \mathbf{X}\underline{\beta})^T \mathbf{W} (\underline{z} - \mathbf{X}\underline{\beta}),$$

dove $\underline{z}$ e $\mathbf{W}$ dipendono in teoria da $\underline{\beta}$. Fissando un $\underline{\beta}_n$, si calcolano $\underline{z}$ e $\mathbf{W}$ e si valuta la funzione: il $\underline{\beta}_{n+1}$ che la minimizza è la soluzione del metodo di Newton. Infatti, visto che $\mathbf{W}$ è diagonale, possiamo scrivere questa funzione come

$$(\underline{z} - \mathbf{X}\underline{\beta})^T \mathbf{W} (\underline{z} - \mathbf{X}\underline{\beta}) = \sum_i p_i (1 - p_i) [z_i - (\mathbf{X}\underline{\beta})_i]^2$$

e derivando rispetto a $\underline{\beta}$, mantenendo $\underline{z}$ e $\mathbf{W}$ costanti, avremo:

$$\begin{aligned} \frac{\partial}{\partial \underline{\beta}} [(\underline{z} - \mathbf{X}\underline{\beta})^T \mathbf{W} (\underline{z} - \mathbf{X}\underline{\beta})] &= \sum_i W_{ii} (\underline{z} - (\underline{z} - \mathbf{X}\underline{\beta})^T \mathbf{W} (\underline{z} - \mathbf{X}\underline{\beta}) (\mathbf{X}^T)_i = 0 \\ \mathbf{X}^T \mathbf{W} \underline{z} - \mathbf{X}^T \mathbf{W} \mathbf{X} \underline{\beta} &= 0 \end{aligned}$$

che è la stessa soluzione estratta prima; possiamo scrivere

$$z_i = \mathbf{W}(\underline{\beta}_n) x_i + \frac{y_i - p_i(\underline{\beta}_n)}{p_i(\underline{\beta}_n) [1 - p_i(\underline{\beta}_n)]}.$$

Iterativamente si minimizza la funzione, ricalcolando i pesi $\underline{z}$ e $\mathbf{W}$ ogni volta, si parla del metodo di *iteratively reweighted least squares* (o IRLS).

L'algoritmo è:

- inizializzazione di $\underline{\beta} \in \mathbb{R}^p$;
- inizializzazione di $\beta_0 = \log \bar{y} - \log(1 - \bar{y}) = \text{cost}^1$, con $\bar{y} = \frac{1}{n} \sum_i y_i$;
- si ripete fino alla convergenza:
 - $\eta_i = \beta_0 + \underline{\beta}^T \underline{x}_i$;
 - $p_i = \exp p(\eta_i) / [1 + \exp(\eta_i)]$;
 - $w_i = p_i (1 - p_i)$;
 - $z_i = \eta_i + \frac{y_i - p_i}{w_i}$;
 - $\mathbf{W} = \text{diag}(\mathbf{W}_{1:n})$;
 - $\underline{\beta} = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \underline{z}$.

Nel caso della distribuzione logistica la matrice hessiana, è simmetrica, infatti

$$\mathbf{H} = \mathbf{H}^T = (\mathbf{X}^T \mathbf{W} \mathbf{X})^T = \mathbf{X}^T \mathbf{W} \mathbf{X},$$

ed è definita positiva, infatti se le probabilità sono $0 < p_i < 1 \forall i = 1, \dots, n$ (diverse da 0, che succede sempre, dalla definizione di p_i nell'algoritmo), abbiamo che $\forall \underline{v} \in \mathbb{R}^p \neq \underline{0}$:

$$\underline{v}^T \mathbf{H} \underline{v} = (\mathbf{X} \underline{v})^T \mathbf{W} \mathbf{X} \underline{v} = \sum_i w_i (\mathbf{X} \underline{v})_i^2 > 0$$

che vuol dire che $\mathbf{H}$ è definita positiva e come tale ha autovalori positivi.

¹Ricordiamo che la matrice $\mathbf{X}$ ha come prima colonna una colonna di soli 1, che ci permette di scrivere β_0 in questo modo.

5.1 Problemi del Metodo di Newton-Raphson

Il primo problema è che il metodo può convergere ad un massimo, e non ad un minimo (vedi figura 5.1). Applicando il metodo di Newton-Raphson, l'algoritmo approssima

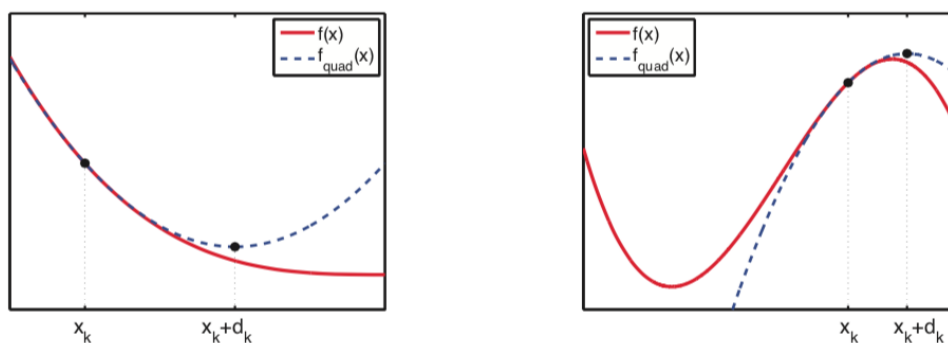


Figura 5.1

la funzione con una funzione quadratica e ne prende il minimo, cercando un punto in cui la derivata sia nulla; ma se ci troviamo vicino ad un massimo locale, il metodo ci avvicina a quest'ultimo. Invece che avvicinarci ad un massimo, questo metodo può anche bypassare il minimo ed il massimo approssimando la funzione ad una funzione quadratica quando non lo è (vedi figura 5.2). Il punto minimo della parabola che

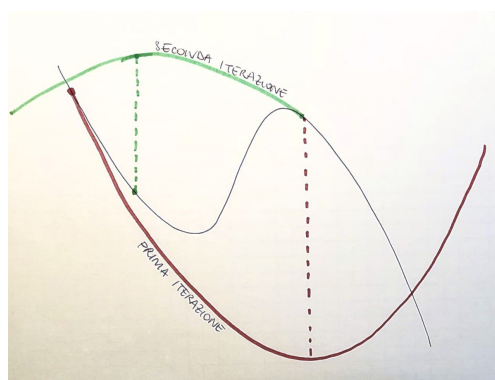


Figura 5.2

ha trovato come approssimazione si troverà oltre il minimo ed il massimo e potrebbe trovarsi in un punto molto ripido: cercando di fittare, esso troverà un'altra parabola, che lo riporterà vicino al punto di partenza, non facendo convergere l'algoritmo.

Un altro problema già citato è il costo computazionale dell'inversione di $\mathbf{H}$ che è $\sim O(p^3)$.

L'ultimo problema è che $\mathbf{H}$ non è necessariamente invertibile e l'algoritmo si blocca.

Nei casi in cui l'algoritmo può essere applicato, siamo sicuri che funzioni per funzioni strettamente convesse, per cui si fa:

- inizializzazione di β_0 ;

- per $n = 0, 1, 2, \dots$ fino a convergenza:
 - calcola $\underline{g}_n = \nabla_{\underline{\beta}} C(\underline{\beta}_n)$;
 - calcola $\underline{\mathbf{H}}_n = \nabla_{\underline{\beta}} \nabla_{\underline{\beta}}^T C(\underline{\beta}_n)$
 - risolve $\underline{\mathbf{H}}_n(\Delta \underline{\beta}_n) = -\underline{g}_n$ per $\Delta \underline{\beta}_n$;
 - ottimizza il passo η_n lungo la direzione di $\Delta \underline{\beta}_n$ (line search);
 - $\underline{\beta}_{n+1} = \underline{\beta}_n + \eta_n(\Delta \underline{\beta}_n)$, l'aggiunta del fattore η_n (o γ_n) di *learning rate* serve per ottimizzare l'algoritmo.

Ci sono alcune modifiche dell'algoritmo che possono essere usate per risolverne i problemi appena citati. Tra questi ne trattiamo due.

Discesa Rapida (Steepest Discent) In questo metodo, l'aggiornamento del vettore $\underline{\beta}$ avviene

$$\underline{\beta}_{n+1} = \underline{\beta}_n - \gamma_n \underline{g}(\underline{\beta}_n)$$

se l'hessiana è troppo complessa da invertire, si può utilizzare semplicemente un fattore γ_n . Il gradiente non è nient'altro che un vettore che punta nella direzione di crescita della funzione, quindi mettendoci un meno davanti siamo sicuri di muoverci nella direzione del minimo. Scegliere il parametro migliore del learning rate è molto complesso: per valori troppo piccoli la convergenza è troppo lenta, mentre per valori troppo alti ci sono oscillazioni o divergenze (vedi figura 5.3).

Un metodo abbastanza utile per trovare un valore di γ sensato è la minimizzazione rettilinea, o *line minimization* o *line search*. Si parte da un punto e si va nella direzione del gradiente: invece, di scegliere a caso un valore di γ , si cerca in quella direzione il punto in cui si minimizza la funzione di costo e si sceglie il γ corrispondente. Si sceglie la direzione, che solitamente è

$$\underline{d} = -\frac{\nabla_{\underline{\beta}} C}{|\underline{g}_n|}$$

e si minimizza rispetto a γ :

$$C(\underline{\beta}_n + \gamma \underline{d}) \sim C(\underline{\beta}_n) + \gamma \underline{g}^T(\underline{\beta}_n) \cdot \underline{d} + O(\gamma^2);$$

tipicamente come si vede nel terzo (e quarto) grafico in figura 5.3, si parte da un punto $\underline{\beta}_n$ e si continua fino a quando non si ottiene un minimo, ma per costruzione si è in un punto in cui il gradiente è ortogonale, quindi $\underline{d}_{n+1} \perp \underline{d}_n$, ci si muove sull'iperpiano perpendicolare alla direzione da cui si è arrivati.

Altri modi per ottenere un'ottimizzazione degli algoritmi e per ottenere una convergenza migliore è quello di aggiungere un termine di momento:

$$\underline{\beta}_{n+1} = \underline{\beta}_n - \gamma_n \underline{g}_n + \mu_n (\underline{\beta}_n - \underline{\beta}_{n-1});$$

oltre a muoversi nella direzione del gradiente più veloce, si tiene anche un pochino di memoria della direzione da cui si arrivava prima. Un altro metodo per l'ottimizzazione di γ è lavorare con un'espansione in γ , sempre con l'idea di ottimizzare il passo. Assumiamo

$$\underline{\beta} = \underline{\beta}_n - \gamma \underline{g}(\underline{\beta}_n),$$

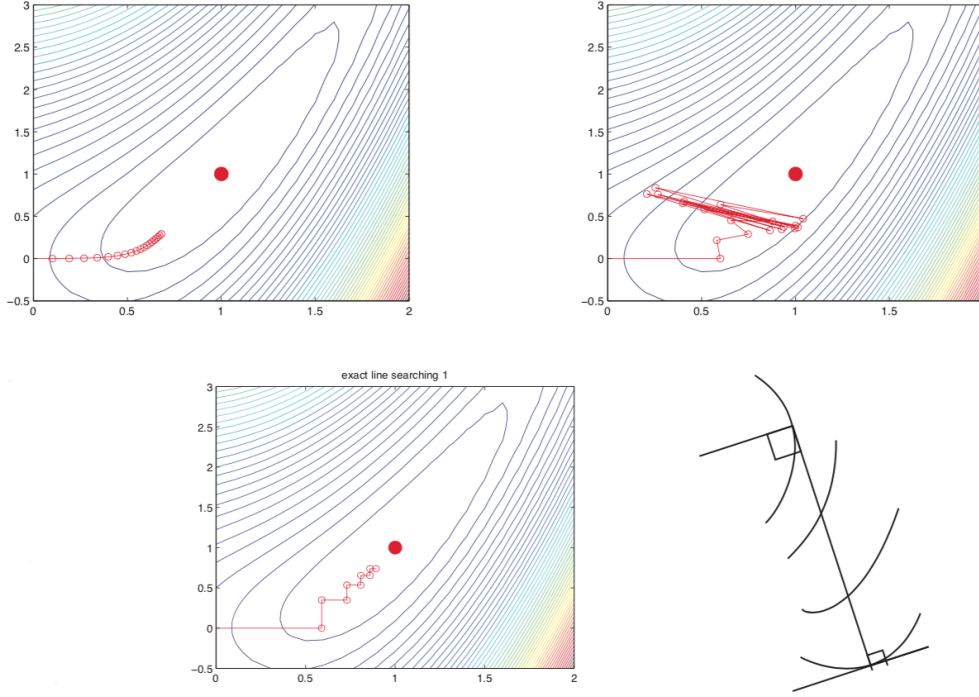


Figura 5.3: Funzione a due variabili. Nel primo grafico abbiamo $\gamma = 0.1$ costante, ma pian piano che la funzione si appiattisce, il passo è sempre più piccolo e la convergenza avviene dopo moltissimo tempo; nel secondo grafico abbiamo $\gamma = 0.6$, vediamo che già nel primo passo abbiamo superato la valle (se fossimo stati nella direzione più "lunga" sarebbe andato bene) e a tutte le iterazioni si sorpassa la valle e non si giunge alla soluzione; il terzo grafico corrisponde alla *line search*.++

espandendo la funzione di costo rispetto a γ otteniamo:

$$\begin{aligned} C(\underline{\beta}) &= C(\underline{\beta}_n) + (\underline{\beta} - \underline{\beta}_n)^T \underline{g}_n + \frac{1}{2}(\underline{\beta} - \underline{\beta}_n)^T \mathbf{H}_n (\underline{\beta} - \underline{\beta}_n) + \dots \\ &= C(\underline{\beta}_n) - \gamma \underline{g}_n^T \underline{g}_n + \frac{\gamma^2}{2} \underline{g}_n^T \mathbf{H}_n \underline{g}_n + O(\gamma^3). \end{aligned}$$

Se trascuriamo i termini non quadratici, possiamo scegliere la γ migliore annullando la derivata:

$$\left. \frac{\partial C}{\partial \gamma} \right|_{\gamma^*} = 0 \quad \Rightarrow \quad -\|\underline{g}_n\|_2 + \gamma \underline{g}_n^T \mathbf{H}_n \underline{g}_n = 0,$$

ovvero

$$\gamma^* = \frac{\underline{g}_n^T(\underline{\beta}_n) \underline{g}_n(\underline{\beta}_n)}{\underline{g}_n^T(\underline{\beta}_n) \mathbf{H}_n(\underline{\beta}_n) \underline{g}_n(\underline{\beta}_n)}$$

e ci si deve sempre calcolare la matrice hessiana ed il suo valore di aspettazione rispetto a $\underline{g}$, che ha un costo computazionale proporzionale a p^2 , che è un po' meglio. Se fossimo in una dimensione, il patto non è altro che $1/h(\beta)$, dove $h(\beta)$ è l'autovalore della matrice hessiana di dimensione 1.

Si può dimostrare che il valore ottimale di γ deve soddisfare

$$\gamma^{\text{opt}} < \frac{2}{\lambda_{\max}}, \quad \text{dove } \lambda_{\max} = \max_{\lambda} \left\{ \lambda : \exists \underline{v} : \mathbf{H}\underline{v} = \lambda \underline{v} \right\}.$$

Non dimostriamo questa condizione, ma la si può immaginare: assumiamo di avere in una dimensione una funzione di costo che segua l'andamento di figura 5.4. Il valore

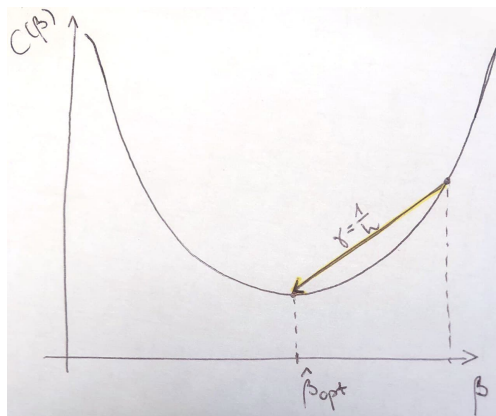


Figura 5.4

ottimale $\hat{\beta}$ è

$$\hat{\beta}_{\text{opt}} = \beta - \frac{g(\beta)}{h(\beta)} = \beta - \gamma g(\beta);$$

se la funzione è quadratica², γ è tale da portarci in un colpo solo al valore ottimale. Se $\gamma \ll 1/h$ si fanno dei passettini molto piccoli e pian pianino si arriva alla soluzione; se $2/h > \gamma > 1/h$ si sorpassa il minimo, ma ad ogni iterazione ci si avvicina sempre di più ad esso. Se $\gamma > 2/h$ si sorpassa il minimo, ma si giunge su un punto della parabola ad un'altezza maggiore di quella iniziale, quindi ad ogni iterazione ci si allontana sempre di più dal valore ottimale, l'algoritmo diverge. Se ci si trova in una situazione in cui si hanno più parametri bisogna cercare di non trovarsi nell'ultima situazione appena menzionata, ed è chiaro che l'autovalore massimo dell'hessiana sarà quello che darà una maggior pendenza della parabola (ovvero la direzione più "schiacciata" in figura).

Nel caso di una funzione non quadratica, ci sono dei termini non lineari, ma alla fine quando ci si avvicina al minimo, si può sempre approssimare ad una funzione quadratica e la condizione su γ va mantenuta.

5.2 Metodi Iterativi

L'idea è risolvere l'equazione

$$\underline{b} = \mathbf{A}\underline{x},$$

dove nel nostro caso $\underline{b} = \mathbf{H}(\underline{\beta}_n)\underline{\beta}_n = \underline{g}(\underline{\beta}_n)$, $\mathbf{A} = \mathbf{H}(\underline{\beta}_n)$ e $\underline{x} = \underline{\beta}$; questo metodo si utilizza quando non si vuole calcolare l'inverso di $\mathbf{A}$.

Se $\underline{x}^{(n)}$ fosse la soluzione vera, avremo che il residuo

$$\underline{r}^{(n)} \equiv \underline{b} - \mathbf{A}\underline{x}^{(n)} = 0;$$

²Quindi valgono $g(\beta) = dC/d\beta = c_1 + 2c_2\beta$ e $h(\beta) = d^2C/d\beta^2 = 2c_2 = \text{cost.}$

se $\underline{x}^{(n)}$ non fosse la soluzione, allora il residuo non sarebbe nullo. L'idea è di partire da $\underline{x}^{(0)}$ e di iterare il processo fino ad arrivare al valore vero $\underline{x}^{(n)}$ (con residuo nullo). Si parte dal punto $\underline{x}^{(0)}$ con residuo $\underline{r}^{(0)} = \underline{b} - \mathbf{A}\underline{x}^{(0)}$; al passo $n + 1$ avremo:

$$\begin{aligned}\underline{r}^{(n+1)} &= \underline{b} - \mathbf{A}\underline{x}^{(n+1)} = \underline{b} - \mathbf{A}(\underline{x}^{(n)} + \alpha^{(n)}\underline{r}^{(n)}) \\ &= \underline{b} - \mathbf{A}\underline{x}^{(n)} - \alpha^{(n)}\mathbf{A}\underline{r}^{(n)} = 0\end{aligned}$$

dove abbiamo assunto che il passo $(n + 1)$ -esimo è pari al passo $\underline{x}^{(n)}$ più il suo residuo moltiplicato per una costante $\alpha^{(n)}$, scelta in modo da limitare il più possibile l'errore al passo successivo, ovvero rendere $\underline{r}^{(n+1)} \sim 0$. Andando avanti con il conto otteniamo

$$\underline{b} - \mathbf{A}\underline{x}^{(n)} - \alpha^{(n)}\mathbf{A}\underline{r}^{(n)} = \underline{r}^{(n+1)} = \alpha^{(n)}\mathbf{A}\underline{r}^{(n)} \sim 0,$$

moltiplicando per $(\underline{r}^{(n)})^T$ si ottiene la soluzione

$$\alpha^{(n)} = \frac{(\underline{r}^{(n)})^T \underline{r}^{(n)}}{(\underline{r}^{(n)})^T \mathbf{A}\underline{r}^{(n)}}.$$

Si parte da $\underline{x}^{(0)}$ e ad ogni passo si ha

$$\underline{x}^{(n+1)} = \underline{x}^{(n)} + \alpha^{(n)}\underline{r}^{(n)},$$

finché $\|\underline{r}^{(n+1)} - \underline{r}^{(n)}\|_2 \leq \varepsilon$, dove $\varepsilon = 10^{-6} - 10^{-10}$.

5.2.1 Esempio regressione lineare

Nella regressione lineare abbiamo

$$\begin{aligned}\hat{f} &= \mathbf{X}\underline{\beta} \\ C(\underline{\beta}) &= \sum_i (y_i - \hat{y}_i)^2 = (\underline{y} - \mathbf{X}\underline{\beta})^T (\underline{y} - \mathbf{X}\underline{\beta})\end{aligned}$$

da cui si calcolano

$$\begin{aligned}\underline{g}(\underline{\beta}) &= \nabla_{\underline{\beta}} C \propto \mathbf{X}^T (\underline{\beta} - \underline{y}) \\ \mathbf{H} &= \frac{\partial^2 C}{\partial \underline{\beta} \partial \underline{\beta}^T} \propto \mathbf{X}\mathbf{X}^T\end{aligned}$$

Essendo C una forma quadratica, la soluzione dell'hessiana non è nient'altro che la soluzione. C'è unicamente un'iterazione. Se per qualche motivo dovessimo usare una discesa del gradiente (se per esempio non possiamo invertire $\mathbf{X}$), ipotizzando $n \approx 10^5$, $p \approx 10^3$, il calcolo di $\mathbf{X}^T \mathbf{X}$ richiede un conto di $p^2 \cdot n \sim 10^{11}$ floating point ≈ 2 min per CPU da 2 GHz (è una sottostima); se fossero necessarie 100 interazioni, il tempo impiegato sarebbe già consistente, nonostante sia uno dei modelli più banali che si possono applicare. Se avessimo più dati su cui lavorare e modelli più complessi, anche la regressione lineare sarebbe troppo costosa in termini di tempo.

Bisogna trovare, quindi, altri metodi di calcolo, che possono essere metodi stocastici, ovvero non calcolare la discesa del gradiente per tutti i casi possibili, ma solo per un certo sottoinsieme delle misure.

Riportiamo nuovamente le limitazioni dei metodi di discesa del gradiente:

- è facile che trovi minimi locali;

- è molto sensibile alle condizioni iniziali, bisogna avere già un'idea di quale possa essere $\underline{\beta}$;
- il calcolo dei gradienti è computazionalmente molto costoso per grandi dataset;
- è molto sensibile alla scelta del *learning rate*, che non è una scelta facile, poiché dipende molto dal punto della funzione in cui ci si trova;
- sono necessari tempi molto lunghi per allontanarsi dai punti di sella;
- il sistema tratta tutte le dimensioni dello spazio in maniera uniforme, sia quelle più importanti che quelle meno importanti.

Per trattare sistemi complessi, con set di dati molto voluminosi, è necessario trovare un metodo più efficace.

5.3 Metodi Stocastici

La funzione di costo, nella maggior parte dei casi, è scritta come la somma di una serie di funzioni di costo relative ad ogni dato misurato. Se abbiamo un *batch*, un set di campionamenti

$$\mathcal{D} = \{(y_i, \underline{x}_i), i = 1, \dots, N\}$$

la funzione di costo è la media su tutti i punti del campionamento di una certa funzione per il punto in particolare:

$$C(\underline{\beta}) = \frac{1}{N} \sum_{i=1}^N \tilde{C}(\underline{\beta}, y_i, \underline{x}_i) = \mathbb{E}_{\mathcal{D}} [\tilde{C}(\underline{\beta}, y_i, \underline{x}_i)]$$

Per esempio, il metodo di massima verosimiglianza e il metodo dei minimi quadrati hanno le funzioni di costo:

$$\begin{aligned} \tilde{C}(\underline{\beta}, y_i, \underline{x}_i) &= -\log p(y_i | \underline{x}_i, \underline{\beta}) \\ \tilde{C}(\underline{\beta}, y_i, \underline{x}_i) &= [y_i - \hat{y}_i(\underline{x}_i | \underline{\beta})]^2. \end{aligned}$$

Visto che è molto costoso valutare i gradienti della funzione di costo, perché non prenderne solo una parte? In questo modo si minimizzano solo alcune delle funzioni di costo parziali. Se prendessimo un insieme

$$D' = D + D$$

con $2N$ dati, ovvero un set di campionamento doppio, e se facessimo la minimizzazione su D' , il costo di computazione non cambierebbe, poiché non stiamo usando un dataset più ricco del precedente, ogni informazione che abbiamo ora, l'avevamo prima, il minimo è lo stesso. L'idea del metodo Montecarlo è che non ci serve avere tutti i dati per fare una buona stima del gradiente. Se nello spazio $\underline{x} - \underline{y}$, abbiamo i campionamenti di figura 5.5, per cui abbiamo un set di dati che non è esattamente raddoppiato, ma in cui per ogni punto c'è un punto molto vicino, possiamo rimuovere la metà dei dati. Se avessimo dei dati sparsi ed alcuni vicini tra di loro, si possono togliere alcuni dei dati vicini, senza pregiudicare la buona uscita del modello.

Normalmente i punti sono sparsi un po' dappertutto e il metodo ne prende una sola parte. Anche senza conoscere il gradiente esatto di tutti i punti possiamo riuscire



Figura 5.5

a portare il gradiente totale verso il minimo e la stima può essere sufficientemente buona.

Invece del gradiente della funzione di costo, si ha una media campionata del gradiente della funzione di costo, più un errore, che se è abbastanza piccolo può essere trascurato:

$$\nabla_{\underline{\beta}} C \longrightarrow \nabla_{\underline{\beta}} \hat{C} + \sqrt{\sigma^2}.$$

Questi metodi possono essere usati in situazioni in cui o il file di dati è troppo lungo e si può caricare in memoria solo un pezzo alla volta, oppure se si vuole fare un'ottimizzazione dinamica del sistema (esempio se Facebook studia le abitudini dei consumatori man mano che essi cliccano certe pubblicità o scrivono determinati post, man mano che si usa il social network avrà più e più dati), quando si cerca una miglioria del modello in maniera graduale, per cui il set di dati cresce di volta in volta.

Ad ogni campionamento k si minimizza $\tilde{C}_k(\underline{\beta}, y_k, \underline{x}_k)$ e si ottiene

$$\underline{\beta}_k = \underset{\underline{\beta}}{\operatorname{argmin}} \tilde{C}_k(\underline{\beta}, y_k, \underline{x}_k)$$

che è quello che minimizza il nuovo contributo alla funzione di costo. Si cerca sostanzialmente la media dei $\underline{\beta}_k$. Se si è in grado di risolvere il valore di beta migliore si otterrà il parametro $\underline{\beta}$, ma altrimenti si può fare una discesa del gradiente stocastica (SDG):

$$\underline{\beta}_{k+1} = \underset{\underline{\beta} = \underline{\beta}_k - \eta_k \cdot \nabla \tilde{C}_k}{\operatorname{argmin}} \tilde{C}(\underline{\beta}, y_k, \underline{x}_k)$$

ogni misura sulla funzione di costo ci dà un'idea dell'errore che stiamo ottenendo nel set di parametri $\underline{\beta}$ rispetto alla funzione ottimale. Il set di parametri migliore sarà la media:

$$\begin{aligned} \langle \underline{\beta}_k \rangle &= \frac{1}{k} \sum_{t=1}^k \underline{\beta}_t = \frac{1}{k} \left(\sum_{t=1}^{k-1} \underline{\beta}_t + \underline{\beta}_k \right) = \frac{k-1}{k} \langle \underline{\beta}_{k-1} \rangle + \frac{1}{k} \underline{\beta}_k \\ &= \langle \underline{\beta}_{k-1} \rangle + \frac{1}{k} (\langle \underline{\beta}_{k-1} \rangle - \underline{\beta}_k) \end{aligned}$$

Come abbiamo già detto, l'idea di base è che non è necessario usare tutti i campionamenti per avere una buona stima del gradiente. Immaginiamo di avere un batch di N dati

$$\mathcal{D} = \{(y_i, \underline{x}_i), i = 1, \dots, N\}$$

e di prenderne un sottoinsieme, un mini-batch

$$B_j \subset \mathcal{D}$$

di dimensione $n_B = \dim B_j \approx 100$. I dati che non vengono compresi nel mini-batch, non saranno buttati, ma verranno inseriti in altri mini-batch, in modo da avere $\{B_j : j = 1, \dots, K\}$ dove $K \approx N/n_B$. Il set di dati viene suddiviso in K sottoinsiemi B_j che verranno a mano a mano utilizzati per ogni adattamento dei parametri: ogni utilizzo di tutti i sottoinsiemi viene chiamato *epoca*.

L'algoritmo SGD mini-batch è organizzato come

- inizializzare $\underline{\beta}_0, \eta_0, n = 0$;
- ripetere (τ/K epoche) fino a convergenza:
 - rimescolare l'ordine in $\mathcal{D}$ casualmente (i dati non devono essere in sequenza, per evitare che i dati siano correlati tra di loro, ed ogni campionamento va preso un'unica volta);
 - per $j = 0, \dots, K - 1$:

$$B_j = \{z_{i(\ell)} : \ell = jn_B + 1, \dots, jn_B + n_B\}$$

$$\underline{g}_j = \sum_{\ell \in B_j} \nabla_{\underline{\beta}} \tilde{C}(\underline{\beta}_n, z_\ell)$$

$$\underline{\beta}_{n+1} = \underline{\beta}_n - \eta_n \underline{g}_j(\underline{\beta}_n)$$

$$\eta_{n+1} \leftarrow \eta_n \quad (\text{si aggiorna la velocità di apprendimento})$$

$$n \longrightarrow n + 1.$$

Ogni volta si calcola il gradiente basandosi solo su un certo numero di campionamenti, assumendo che questo dia una buona approssimazione; si ottiene una media dei gradienti e si fa una statistica su tutto l'insieme: non si fa una media su 10000000 (N) contributi al gradiente per fare un passo, ma ogni passo si basa su 100 contributi (n_B), e quindi si fanno 10000 passi nello spazio delle fasi. Pian piano si hanno 10000 opportunità di avvicinarsi al minimo piuttosto che una sola: anche se il campionamento e la conoscenza del gradiente non sono perfetti, nella media si avvicineranno molto ad esserlo.

Uno dei problemi è la scelta della velocità di apprendimento.

Ci sono dei casi limite. Se la dimensione del mini-batch è $n_B = N$, si valuta il gradiente su tutti i dati, che è quindi il metodo di discesa del gradiente standard. Se i mini-batch hanno dimensione $n_B = 1$, ogni stima del gradiente è basata su un campionamento solo, quindi si ottiene che per ogni dato si calcola il β migliore e ci si muove in quella direzione. La soluzione intermedia è il *mini-batch SGD*: non ci si affida al campionamento di ogni dato, ma neanche alla risposta di un unico dato.

È utile il mini-batch, poiché si lavora con matrici, ma matrici trattabili, che si possono tranquillamente caricare su una GPU (*Graphical Processing Unit*) e per ogni ciclo calcolare il mini-batch, si ha un trattamento dei dati senza perdere in velocità (il processore grafico tratta i 100×100 dati nello stesso tempo in cui una CPU ne tratta 100).

In una situazione di minimo locale, se consideriamo tutti i dati, la discesa è molto veloce verso il minimo locale; se facciamo però una statistica sul gradiente, uscire dal minimo locale è molto più facile, poiché i gradienti non sono fatti alla perfezione ed

è facile che questi portino a superare il minimo locale. Ci sono due nuovi parametri da gestire, poiché abbiamo il numero di mini-batch da associare (quindi il numero di dati nei sottoinsiemi) ed il numero di epoche, il numero di volte in cui l'algoritmo va ripetuto.

La velocità di apprendimento è un campo di ricerca a sé stante, in cui si stanno ancora sviluppando delle idee. Normalmente è bene che $\eta < 2/|\lambda_{\max}|$, ma bisogna cercare di capire quale sia il valore migliore, che dipende dal modello, dalla curvatura, ecc... Una strategia che si può usare per aggiornare η è di partire con un valore grande (se siamo distanti dal minimo ci vogliamo muovere velocemente), ma che poi si riduca in modo da evitare oscillazioni intorno al minimo:

$$\eta_k = (1 - \alpha)\eta_0 + \alpha\eta_\tau$$

dove α cambia linearmente con il numero di epoche e dove solitamente $\eta_\tau = \eta_0/100$.

Esempio

Consideriamo un set di dati regolati da $y = 4 + 3x_0 + (\text{errore gaussiano})$, con la regressione lineare otteniamo dei valori di β che sono vicini (vedi figura 5.6). In figura

Regressione Lineare (LSE) :

```
X_b = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

```
>>> theta_best
array([[ 4.21509616],
       [ 2.77011339]])
```

Figura 5.6

Standard SGD:

```
eta = 0.1 # learning rate
n_iterations = 1000
m = 100

theta = np.random.randn(2,1) # random initialization

for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
    theta = theta - eta * gradients

>>> theta
array([[ 4.21509616],
       [ 2.77011339]])
```

Figura 5.7

5.7 si usa il metodo della discesa del gradiente standard che è lo stesso risultato, perché le 1000 iterazioni sono sufficienti ad arrivare nel minimo vero. La regressione lineare ci porta al minimo in un colpo solo (invertendo la matrice), mentre con lo standard GD, ci si arriva dopo 1000 iterazioni e con $\eta = 0.1$. In figura 5.8 è mostrato ciò che accade quando si cambia il learning rate. Con un learning rate piccolo, l'avvicinamento al minimo è estremamente lento, dopo 1000 iterazioni non è ancora arrivato. Se il learning rate è buono, ci si avvicina al risultato corretto dopo poche iterazioni e dopodiché si aggiusta. Se il learning rate è troppo elevato, si salta avanti e indietro, oltrepassando il minimo.

Nel caso di SGD con dimensione di mini-batch pari a 1 si ottengono i risultati in figura 5.9, che non è la soluzione migliore per i 100 punti totali (che sarebbe la

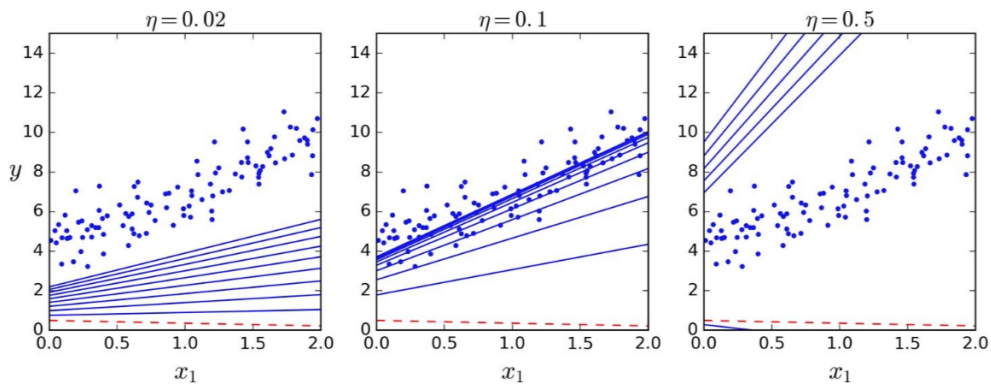


Figura 5.8

Stochastic GD:

```
from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(n_iter=50, penalty=None, eta=0.1)
sgd_reg.fit(X, y.ravel())

>>> sgd_reg.intercept_, sgd_reg.coef_
(array([ 4.18380366]), array([ 2.74205299]))
```

Figura 5.9

regressione lineare), ma comunque ci si avvicina molto. In figura 5.10 è mostrato il processo di SGD, in cui i passi sono molto irregolari, ma si arriva ad una soluzione non perfetta, ma adeguata. In figura 5.11 è mostrato un grafico nello spazio dei parametri

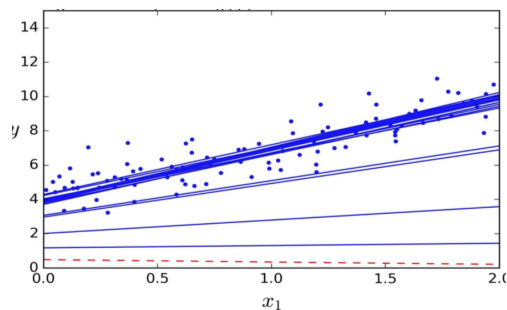


Figura 5.10

dei 3 diversi metodi utilizzati: il caso migliore è il mini-batch, per cui si arriva al risultato anche in meno di 1000 iterazioni, il movimento erratico ci dà la possibilità di uscire da un minimo locale, ma non è portato all'estremo come nel caso stocastico (dimensione dei mini-batch pari a 1).

Una volta scelto il set, è necessario determinare il learning rate. Ci sono alcuni metodi per il calcolo del learning rate più efficaci.

Momentum Based GD Immaginiamo di essere in una situazione del tipo in figura 5.12. Se utilizziamo il metodo standard (la direzione è sempre ortogonale al contorno)

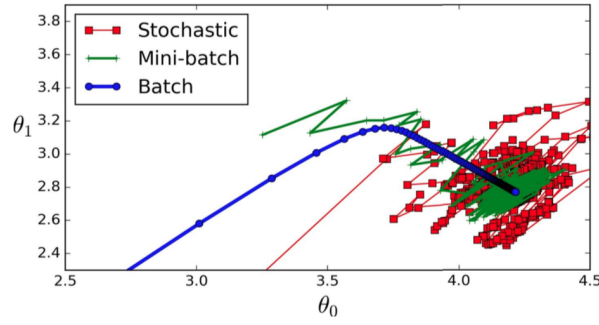


Figura 5.11

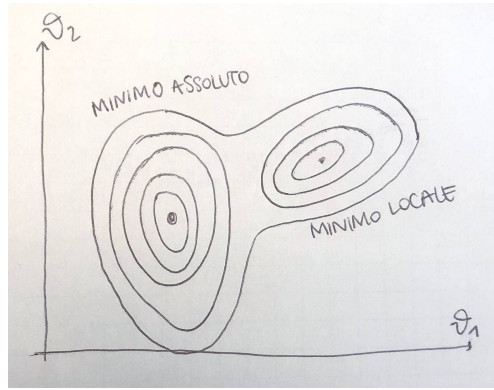


Figura 5.12

è molto facile cadere nel minimo locale. L'idea di questo metodo è di dare una certa inerzia al cammino nello spazio dei parametri, in modo che si abbia una sorta di memoria della direzione precedente, aggiungendo una componente lungo tale direzione:

$$\begin{aligned}\underline{v}_t &= \gamma \underline{v}_{t-1} + \eta_t \nabla_{\underline{\theta}} E(\underline{\theta}_t) \\ \underline{\theta}_{t+1} &= \underline{\theta}_t - \underline{v}_t\end{aligned}$$

dove $\underline{\theta} \equiv \underline{\beta}$ (nel campo delle reti neurali è più diffuso l'utilizzo di $\underline{\theta}$) e $E(\underline{\theta}) \equiv C(\underline{\beta})$. La velocità viene aggiornata di volta in volta con una parte data dal passo precedente e una parte data dal gradiente. Equivalentemente possiamo scrivere:

$$\Delta \underline{\theta}_{t+1} = \gamma \Delta \underline{\theta}_t \eta_t \nabla_{\underline{\theta}} E(\underline{\theta}_t),$$

dove definiamo $\Delta \underline{\theta}_t = \underline{\theta}_t - \underline{\theta}_{t-1}$. Questo tipo di approccio, si basa sull'idea di considerare non solo l'evoluzione nel tempo nello spazio dei parametri, ma di considerarne anche la velocità. Ci sono analogie con la dinamica classica, in cui, se $\underline{w}$ è la posizione, avremo

$$m \frac{d^2 \underline{w}}{dt^2} + \mu \frac{d \underline{w}}{dt} = -\nabla_{\underline{w}} E(\underline{w}),$$

dove abbiamo il gradiente che sarà legato all'accelerazione e ad un attrito. Possiamo discretizzare questa equazione come

$$m \frac{w_{t+\Delta t} - 2w_t + w_{t-\Delta t}}{(\Delta t)^2} + \mu \frac{w_{t+\Delta t} - w_t}{\Delta t} = -\nabla_{\underline{w}} E(\underline{w});$$

che, riarrangiando i termini porta a:

$$\Delta \underline{w}_{t+\Delta t} = -\frac{(\Delta t)^2}{m + \mu \Delta t} \nabla_{\underline{w}} E(\underline{w}) + \frac{m}{m + \mu \Delta t} \Delta \underline{w}_t$$

che è l'equazione di prima. Il parametro μ di attrito fa sì che la velocità diminuisca man mano che ci si avvicini al punto minimo, evitando di oltrepassarlo. Si può scrivere che

$$\gamma = \frac{m}{m + \mu \Delta t}, \quad \eta = \frac{(\Delta t)^2}{m + \mu \Delta t}.$$

Root Mean Squared Propagation (RMS prop) Il parametro $\underline{\theta}$ si scrive come

$$\underline{\theta}_{t+1} = \underline{\theta}_t - \mathbf{H}^{-1} \underline{g}(\underline{\theta}_t)$$

e questo metodo si basa sull'imporre la matrice hessiana come matrice diagonale, dal momento che spesso non è facilmente invertibile. L'idea è quindi di approssimare l'hessiana come

$$\begin{aligned} \mathbf{H} &= \frac{\partial^2 C}{\partial \underline{\theta} \partial \underline{\theta}^T} = \frac{\partial \underline{g}}{\partial \underline{\theta}^T} \\ &\approx \text{diag}(\underline{g}^T \underline{g}). \end{aligned}$$

Man mano che si aggiungono dati al sistema, si aggiunge il termine $\underline{g}_t^2$:

$$\begin{aligned} \underline{g}_t &= \nabla_{\underline{\theta}} E(\underline{\theta}) \\ \underline{s}_t &= \beta \underline{s}_{t-1} + (1 + \beta) \underline{g}_t^2 \\ \underline{\theta}_{t+1} &= \underline{\theta}_t - \eta_t \frac{\underline{g}_t}{\sqrt{\underline{s}_t + \varepsilon}}, \end{aligned}$$

dove β è un parametro che controlla la crescita del gradiente e $\underline{s}$ è la somma dei quadrati del gradiente: si mantiene la somma precedente, quindi l'idea della curvatura precedente, e si aggiunge il nuovo gradiente, che è il contributo alla curvatura dell'ultima iterazione. Si divide poi il learning rate per la radice quadrata della somma dei quadrati più un termine molto piccolo, in modo da evitare problemi nel caso di $\underline{s} = 0$.

Nel metodo di Newton avevamo

$$E(\underline{\theta}_n) = f(\underline{\theta}_n - \eta \underline{g}(\underline{\theta}_n)) = f(\underline{\theta}_n) - \eta \underline{g}^T \underline{g} + \frac{1}{2} \eta^2 \underline{g}^T \mathbf{H} \underline{g} + O(\eta^3)$$

e si cercava di annullare la somma tra secondo e terzo termine: questo porta ad avere un bilanciamento tra la curvatura del gradiente e la sua intensità. Se la curvatura è molto grande (la derivata seconda è elevata) ed il learning rate è troppo grande, il metodo diverge (si sorpassa la valle), mentre se la curvatura non è pronunciata ma il learning rate è piccolo, il metodo non arriva alla soluzione (si muove troppo lentamente).

Andando a stimare il termine $\underline{g}^T \underline{g}$ di volta in volta, si può prendere un learning rate più adeguato alla situazione.

ADAM (Adaptive Momentum) In questo metodo in base al gradiente si tiene memoria della velocità e si tiene conto della curvatura ($\underline{g}^2$):

$$\begin{aligned}\underline{g}_t &= \nabla_{\underline{\theta}} E(\underline{\theta}) \\ \underline{m}_t &= \beta_1 \underline{m}_{t-1} + (1 - \beta_1) \underline{g}_t \\ \underline{s}_t &= \beta_2 \underline{s}_{t-1} + (1 - \beta_2) \underline{g}_t^2 \\ \underline{m}_t &= \frac{\underline{m}_t}{1 - \beta_1^t} \\ \underline{s}_t &= \frac{\underline{s}_t}{1 - \beta_2^t} \\ \underline{\theta}_{t+1} &= \underline{\theta}_t - \eta_t \frac{\underline{m}_t}{\sqrt{\underline{s}_t} - \varepsilon}\end{aligned}$$

dove si è fatta una normalizzazione di $\underline{m}_t$ e $\underline{s}_t$ basata sui valori di β_1 e β_2 scelti (tipicamente 0.9 e 0.99 rispettivamente). Non si aggiusta solo η , ma si aggiusta pian piano anche il gradiente.

In generale, a prescindere dal metodo di ottimizzazione utilizzato (come quelli appena citati, ci sono alcuni consigli pratici:

- rimescolare in modo random i dati dei mini-batch creati per evitare di avere dati correlati;
- trasformare gli input, riscalarli in modo che i valori di $\{\underline{x}_i\}_{i=1,\dots,p}$ vadano da 0 a 1 (analogamente per le y);
- monitorare l'*out-of-sample performance*, si consiglia di controllare il validation dataset tutte le volte che si fa la minimizzazione;
- l'uso di *adaptive methods* non sempre funziona nelle generalizzazioni; non ci sono generalizzazioni matematiche che ci dicono quando usare un metodo piuttosto che un altro, attualmente è una branca molto attiva.

Capitolo 6

Reti Neurali

Le reti neurali artificiali sono metodi computazionali che posso ‘imparare’ ad eseguire compiti, a volte anche molto differenti tra loro, confrontandosi con una serie di esempi. Tipicamente questo avviene senza la necessità di programmare il modello secondo regole specifiche per il problema in questione. L’idea che ha dato origine alle reti neurali è fare una mimica dei neuroni nel cervello umano. Il neurone è una cellula, basata su un certo numero di dendriti, che sono delle connessioni di input, da cui si ricevono gli impulsi dagli altri neuroni. Quando arrivano i segnali elettrici dall’esterno, questi si sommano e creano una differenza di potenziale all’interno del neurone; quando si supera una certa soglia c’è una scarica elettrica che corre lungo l’assone (uno dei bracci) che si collega ai dendriti di altri neuroni.

Se si ha un set di variabili di input $\{x_i\}$, una stima della carica contenuta nel neurone è una loro combinazione lineare

$$z = (w_1 \quad w_2 \quad \cdots \quad w_N) \cdot \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} + \text{bias}$$

e l’output (ciò che viene comunicato ai neuroni successivi) sarà

$$f(z) = y$$

che può essere, per esempio, una Theta di Heavyside. In realtà i sistemi biologici sono molto più complessi, la relazione non è lineare e i neuroni comunque hanno un tempo di carica ed un tempo di scarica (funziona come un condensatore).

Le reti neurali che ci interessano sono dei modelli che possiamo usare per analizzare i dati, ideati a partire dagli anni '60. Negli ultimi vent’anni, grazie allo sviluppo di computer più efficienti e di memorie più capienti, questi modelli sono diventati applicabili e funzionanti.

Vediamo ora i tipi di reti neurali.

Feed-Forward Neural Networks (FFNN)

In questa rete neurale si parte da un set di input $\{x_i\}$ e si va ad eccitare una serie di nodi (neuroni), chiamati *hidden layers*. Ogni strato eccita lo strato successivo, fino ad

arrivare allo strato finale, in cui gli output vengono caricati sulla variabile y :

$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow y$$

Il motivo di questo nome è che le informazioni si muovono solo in una direzione, attraverso gli strati.

Convolutional Neural Networks

È una variazione di FFNN: certe parti delle reti (degli strati) vengono trattati in maniera locale, non si ha connessione tra tutti i nodi di uno strato e quello successivo, ma ci sono delle strutture locali. Tipicamente viene usata per riconoscere immagini poiché è basata sulla struttura dei neuroni della parte di corteccia dedicata alla visione.

Recurrent Neural Networks

In queste reti neurali l'informazione non viaggia solo in una direzione, ma può essere riportata a neuroni precedenti (si ha sempre un input e l'output y). Spesso vengono usate per riconoscimento vocale, testi scritti e per estrarre informazioni da serie temporali (utile in tutti quei casi in cui si trattano sistemi che hanno in qualche modo a che fare con una memoria).

6.1 Percettroni

Il primo tipo di rete neurale che affrontiamo sono le FFNN in particolare partendo dal singolo neurone, passando per i neuroni multistrato e arrivando poi a strutture di reti neurali più complesse.

6.1.1 Singolo Percettrone

Uno dei motivi per cui i percettroni sono utili è perché c'è lo *universal approximation theorem*, che dice che sotto determinate condizioni, una rete di percettroni riesce ad approssimare qualsiasi funzione continua.

Questo modello è l'elemento fondamentale per creare reti che hanno la possibilità di trattare problemi anche molto complessi. Se però gli strati cominciano a diventare un grande numero, conviene utilizzare altri modelli e reti più sofisticate.

Il singolo percettrone può essere schematizzato come in figura Fig. (6.1); ci sono più valori di input $\{x_i\}$ ed una variabile di output y . Il percettrone (o in generale il singolo neurone) realizza una combinazione lineare degli input:

$$z = w_1x_1 + w_2x_2 + \dots + b = \underline{w}^T \underline{x} + b$$

che viene poi combinata tramite una funzione di attivazione che inserisce nel modello un elemento di non linearità:

$$y = f(z).$$

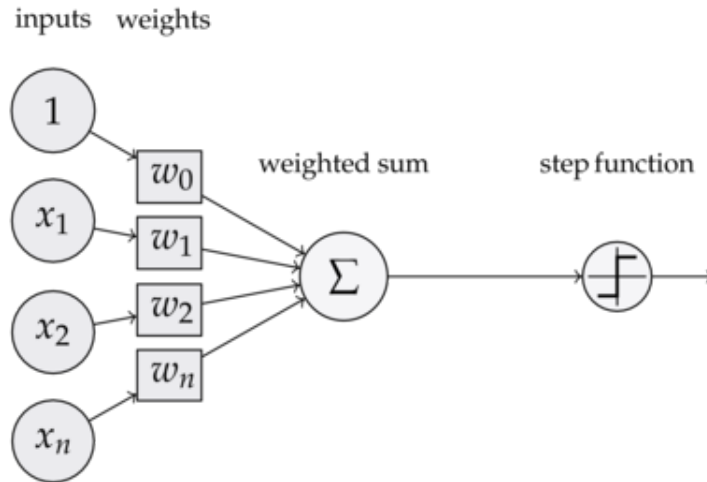


Figura 6.1: L'immagine rappresenta la struttura di un singolo percettrone con diversi input, una certa funzione di attivazione e l'intervento di un bias prima dell'uscita dell'informazione

dove f è la funzione di attivazione. I parametri del modello sono quindi $\underline{w}, b, f(\cdot)$. Tra le funzioni di attivazione, alcune possibili sono

$$\begin{array}{ll} \text{Sigmoide } \frac{1}{1 + e^{-x}} & \text{Step } f(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases} \\ \text{Softmax } \frac{e^x}{1 + \sum e^x} & \text{ReLU } f(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases} \end{array}$$

Importanza dell'utilizzo di reti con più neuroni Vediamo ora un esempio in cui si osserva la non efficacia del singolo neurone nel ricostruire alcune operazioni logiche fondamentali come ad esempio lo XOR. In logica l'or esclusivo (o XOR) funziona come in tabella: Se lo dovessimo rappresentare su un piano cartesiano lo possiamo

x_1	x_2	$t = x_1 \wedge x_2$
0	0	0
0	1	1
1	0	1
1	1	0

rappresentare come in figura Fig. 6.2.

Utilizzando il singolo percettrone, con la Theta di Heavyside come funzione di attivazione (quindi $\hat{y} = \Theta(w_1x_1 + w_2x_w + b)$), con $b = 1/2$ si ha

$$\begin{array}{ll} \hat{y} = 1 & \text{se } z > 1/2; \\ \hat{y} = 0 & \text{se } z < 1/2. \end{array}$$

Se applichiamo il percettrone non stiamo facendo altro che fare la regressione lineare classica per i problemi di classificazione. Questo modello però non può funzionare

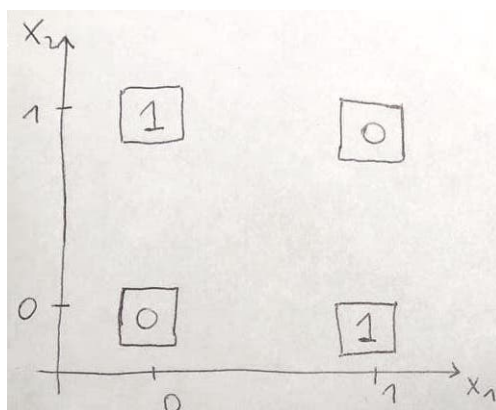


Figura 6.2: Nell'immagine è rappresentata l'azione dello XOR (all'interno dei quadrati) in funzione dei valori di input (discreti)

se abbiamo più di 2 classificazioni possibili, come avevamo già visto nella regressione lineare per problemi di classificazioni. Quindi per la funzione XOR non può funzionare, come vediamo se proviamo a fare i conti per il percettore (o con la regressione lineare):

$$z = (x_1^1 \quad x_2^1) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} + b = (1 \quad x_1^2 \quad x_2^2) \underline{\beta}$$

dove

$$\underline{\beta} = \begin{pmatrix} b \\ w_1 \\ w_2 \end{pmatrix}$$

che ha output

$$\begin{aligned} \hat{y} &= 1 && \text{se } z > 1/2; \\ \hat{y} &= 0 && \text{se } z < 1/2. \end{aligned}$$

La matrice di design e l'output sono

$$\mathbf{X} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

Calcolando

$$\underline{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y$$

otteniamo

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 2 & 1 \\ 2 & 1 & 2 \end{pmatrix}, \quad (\mathbf{X}^T \mathbf{X})^{-1} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & 1 & 0 \\ -\frac{1}{2} & 0 & 1 \end{pmatrix}$$

e quindi,

$$\underline{\beta} = \begin{pmatrix} 1/2 \\ 0 \\ 0 \end{pmatrix}$$

e quindi con $b = 1/2, w_1 = w_2 = 0$ si ha $z = 1/2 \quad \forall x_1, x_2$ e non è possibile ottenere l'output corretto, non è possibile dire se il risultato sarà 0 oppure 1. Questo sistema infatti trova un confine lineare tra le due classi del sistema esattamente come la regressione lineare e, come si può immaginare dalla figura 6.2, non è possibile scegliere un confine lineare che possa dare luogo ai risultati corretti. Se aggiungiamo uno

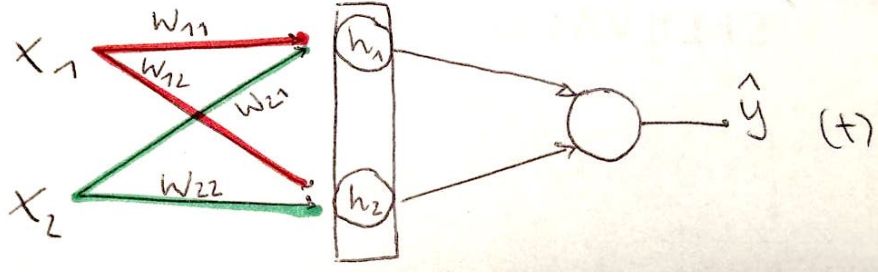


Figura 6.3: Immagine della rete neurale utilizzata per ottenere l'output di uno XOR

strato latente, chiamato così perché non è legato né ai valori di input, né ai valori di output e l'utente non ne conosce i componenti, possiamo ottenere la soluzione corretta. Costruiamo la rete neurale come in figura 6.3, nello strato latente abbiamo due perceptroni. Definiamo le due variabili di input z_1 e z_2 dei due neuroni dello strato latente:

$$\begin{aligned} z_1 &= w_{11}x_1 + w_{12}x_2 + b_1 & h_1 &= f^{(1)}(z_1) \\ z_2 &= w_{21}x_1 + w_{22}x_2 + b_2 & h_2 &= f^{(1)}(z_2). \end{aligned}$$

Avremo poi uno strato finale di output, che sarà una combinazione di h_1 e h_2 (il cerchio in figura 6.3):

$$\begin{aligned} f^{(1)}(\mathbf{W}^T \underline{x}_i^T + \underline{b}^T) &\rightarrow \underline{h}^T = \begin{pmatrix} h_1 \\ h_2 \end{pmatrix}^T \\ f^{(2)}(\underline{w}^T \underline{h}) &\rightarrow \hat{y} \end{aligned}$$

dove $\mathbf{W}$ è la matrice dei pesi w_{ij} (relativi al passaggio dell'informazione dallo strato di input allo strato latente). La soluzione per questo problema si trova scrivendo la matrice

$$\mathbf{X} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix} \quad \mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \quad \underline{b} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \quad \underline{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Come funzione di attivazione possiamo utilizzare una Relu per entrambi i nodi:

$$f^{(1)} = f^{(2)} = \text{ReLU} = \max\{0, z\}.$$

Quindi risolvendo otteniamo

$$\mathbf{W}^T \mathbf{X} + \underline{b} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 2 & 2 \end{pmatrix} + \begin{pmatrix} 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} = (z_1 \quad z_2)$$

Calcoliamo quindi

$$\underline{h}^T = \max\{0, \mathbf{W}^T \mathbf{X} + \underline{b}^T\} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

$$\underline{h}^T \underline{w} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

$$\hat{y} = \max(0, \underline{w}^T \underline{h}) = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \equiv t$$

In questo caso, inserendo un layer nascosto, si riesce a riprodurre la funzione di partenza. La situazione è descritta graficamente come in figura 6.4.

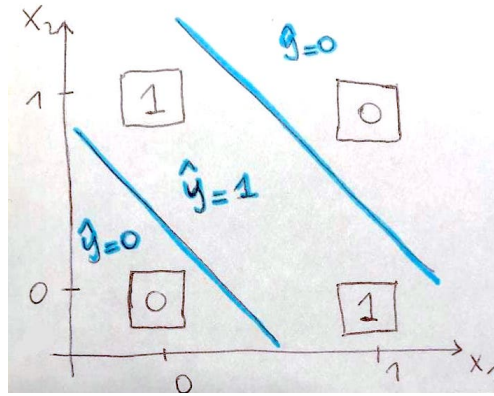


Figura 6.4: Rappresentazione grafica dell'output previsto dalla rete neurale per le diverse zone del grafico e dell'output reale di uno XOR in funzione degli input.

6.1.2 Funzioni di attivazione

Finora abbiamo visto alcuni esempi di funzioni di attivazione, come ad esempio la funzione gradino di Heaviside. Esistono diverse possibili scelte per le funzioni di attivazione, tra le più comuni troviamo:

- sigmoide:

$$\sigma(z) = \frac{e^z}{1 + e^z};$$

- tangente iperbolica:

$$\tanh(z) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2z) - 1;$$

- softmax:

$$f(z_i^{(L)}) = \frac{e^{z_i}}{\sum_{j=1}^q e^{z_j}}, \quad \text{con } \sum_i f(z_i^{(\ell)}) = \sum_i p_i = 1$$

La sigmoide è una funzione di attivazione particolarmente utile perché permette di approssimare in maniera più o meno blanda la funzione gradino, con il vantaggio però di inserire un elemento di non linearità. La tangente iperbolica è di fatto il risultato dell'applicazione di una traslazione e di riscaldamento della funzione di attivazione sigmoide (si può dimostrare algebricamente) ed è quindi un approccio simile, con uno scalino che andrà tra -1 e +1 (in genere è però più conveniente il sigmoide - anche per questioni di ottimizzazione della rete).

La funzione di attivazione soft-max è spesso utilizzata nel caso di regressione logistica di più classi: imponendo che il rapporto tra le probabilità di essere in una classe piuttosto che in un'altra sia una funzione lineare ed imponendo la somma delle probabilità ad 1, si ottiene infatti proprio la funzione softmax. Si noti che nel caso di classificazione binaria è equivalente alla funzione di attivazione sigmoide.

Se si ha una rete neurale con delle variabili x e si desidera fare una regressione logistica, spesso si vogliono trattare le classi in maniera differente, per cui negli strati latenti si usano funzioni di attivazione differenti, ma nello strato di output si va ad usare il softmax in maniera tale che l'output risulti in qualche modo normalizzato e che venga messa in evidenza la predizione più probabile. Ad esempio, se volessimo riconoscere i numeri scritti da 0 a 9, abbiamo 10 classi esclusive, vogliamo come output le dieci cifre decimali, con una probabilità la cui somma è 1; per questo usiamo la funzione softmax. Se abbiamo una classificazione in cui le classi non sono esaustive (es: maschio o femmina, maggiorenne o minorenne, ecc...), ad ogni variabile di uscita si ha una classificazione binaria indipendente dalle altre ed è meglio usare il sigmoide.

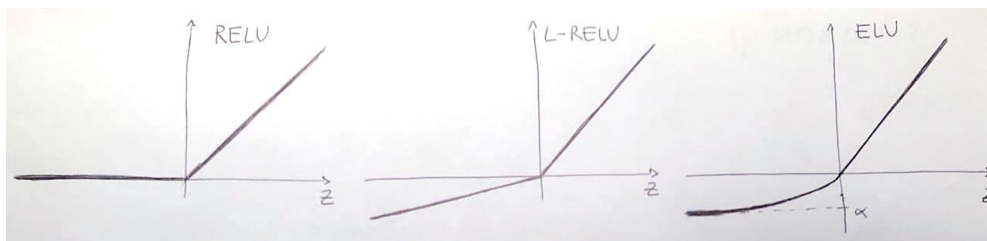


Figura 6.5: Esempi di funzioni di attivazione che agiscono rettificando l'input

Ci sono altre funzioni di attivazione che, a differenza da quelle appena viste che si riconducono a funzioni a gradino e si basano su una rettificazione della variabile di input (figura primo, secondo e terzo grafico in figura 6.5,rispettivamente):

- Rectified Linear Unit (ReLU), che è una funzione lineare che mescola input e output:

$$f(z) = \max\{0, z\},$$

la pendenza della ReLU non fa differenza, poiché nello strato successivo ci sarà il coefficiente di combinazione lineare che può essere riscalato (equivale a cambiare la pendenza), quindi di norma si prende il rapporto a 1: questa funzione introduce una non linearità all'origine, che però può essere esagerata, poiché rimuove qualunque input appena negativo;

- Leaky ReLU, che è un rettificatore, ma che tiene leggermente in conto i valori negativi:

$$f(z, \alpha) = \max\{0, z\} + \alpha \min\{0, z\},$$

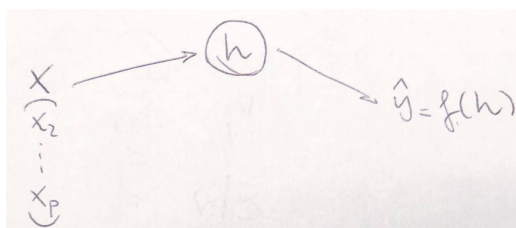
con $\alpha \sim 0.01$;

- Exponential Linear Unit (ELU):

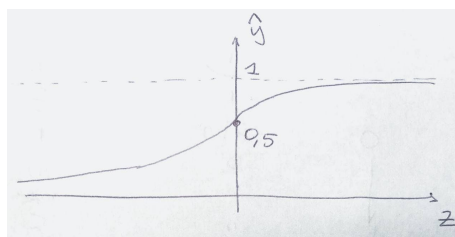
$$f(x) = \begin{cases} z & z \geq 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$$

se $\alpha \neq 1$ la funzione non è continua in 0, e può essere un problema nel calcolo delle derivate.

6.2 Teorema di approssimazione universale



(a) Rappresentazione del nodo su cui ci concentriamo



(b) Funzione di attivazione sigmoide

Immaginiamo di avere una rete neurale e che sia costruita come in figura 6.6a, ovvero con un nodo di input x (possono essercene di più ma ci interessiamo solo al primo), un nodo latente h , la funzione di output $\hat{y} = f(h)$ e una sigmoide come funzione di attivazione

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

mostrata in figura 6.6b. Cosa succede se cambiamo i parametri del modello? Noi abbiamo x come input, la funzione $z = wx + b$ e $\hat{y} = \sigma(wx + b)$. Se prendessimo $w = b = 0$, indipendentemente da x , $\hat{y} = \text{cost}$, avremmo il primo grafico in figura 6.7. Se invece prendiamo $b = 0$ e $w = 0.01$, lavoriamo nell'intorno lineare del sigmoide, mostrato nel secondo grafico in figura 6.7.

Se $b = 0$ e $w = 4$ il range preso in considerazione cresce, e analogamente per $w = -4$, quello che vedremmo negli ultimi due casi è rappresentato negli ultimi due grafici in figura.

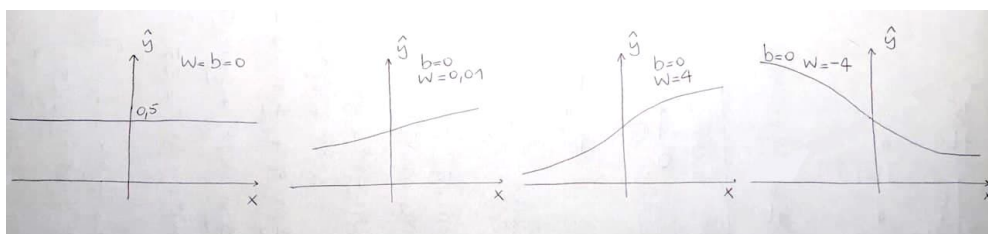


Figura 6.7: Variazione dell'output in funzione dell'input con funzione di attivazione sigmoide

Nel limite di pesi estremamente grandi, che riescono ad andare da z molto negativi a z molto positivi; se $w \gg 1$, avremo l'andamento che del primo grafico in figura Fig. 6.8, ovvero un andamento simile a quello della funzione di Heavyside.

Tuttavia, il punto di salto può cambiare in relazione al bias b : se utilizziamo ad esempio $y = \sigma(z) = \sigma(w(x - x_0)) = \sigma(wx - wx_0)$ vediamo che il punto di salto è in x_0 (secondo grafico in figura Fig. 6.8).

Immaginiamo di avere due layer h_1 e h_2 : la nostra $\hat{y}$ sarà

$$\hat{y} = f(w_1^o h_1 + w_2^o h_2)$$

dove i w_i^o ci servono a calibrare l'altezza della step-function. La funzione di uscita è una combinazione di due percettroni latenti. Possiamo fare una funzione di percettrone

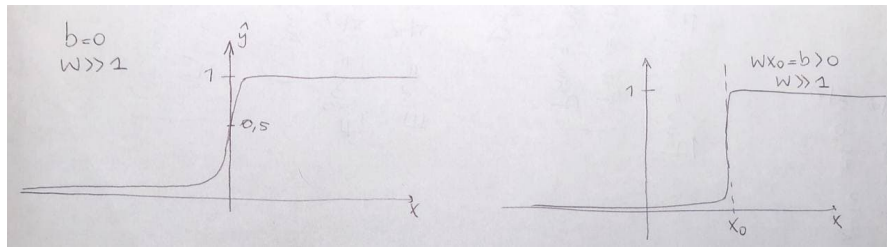


Figura 6.8: Approssimazione della funzione a gradino tramite neurone e funzione sigmoide

positivo ed uno negativo ($w_1^o < 0$ e $w_2^o > 0$ o viceversa), e così avremo una funzione come in figura Fig. 6.9 dove il valore in uscita è un certo numero A . Se abbiamo una combinazione di più neuroni, possiamo avere più gradini, il cui risultato sarà una funzione complicata quanto si vuole, purché si abbiano un numero sufficiente di neuroni; questa è l'idea su cui si basa il teorema di approssimazione universale, che dice che una rete con uno strato di neuroni latente può approssimare una qualsiasi funzione a patto che ci siano abbastanza nodi.

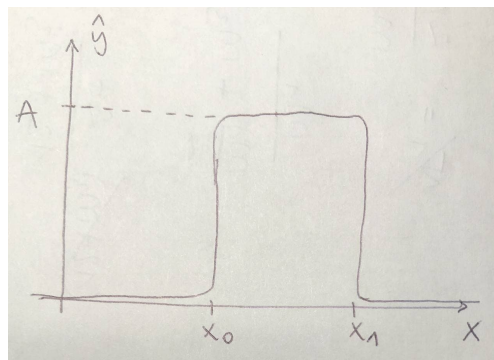


Figura 6.9: Esempio di una approssimazione possibile - teorema di approssimazione universale

6.3 Feed Forward Neural Network

Generalmente un FFNN viene creata come combinazione di neuroni, quindi si tratta in un certo senso di multilayer perceptron (dove in questo caso il termine perceptrone è utilizzato in maniera leggermente impropria - la funzione di attivazione può anche essere diversa dalla step function tipica del perceptrone). Nella forma più generale abbiamo un set di variabili di input $x_1, \dots, x_p$ ed una serie di variabili di output $y_1, \dots, y_q$, con una serie di strati latenti h_m^ℓ dove ℓ indicizza i layer ed $m(\ell)$ indica il numero di neuroni per layer che in generale può variare.

Il primo strato latente sarà una combinazione lineare dei dati di input con un bias e gli strati successivi saranno una combinazione lineare delle variabili dello strato precedente, con bias $b^{(\ell)}$. Per lo strato latente varrà $h^{(\ell)}$, con $z_j^{(\ell)} = \sum_i w_{ij}^{(\ell)} h_i^{(\ell-1)} + b_j^{(\ell)}$, dove $j = 1, \dots, m(\ell)$; avremo $h_j^{(\ell)} = f(z_j^{(\ell)})$, con $h_j^{(0)} \equiv x_j$ e $h_j^{(L)} \equiv a_j^{(L)} = y_j$. In generale, i parametri che definiscono una rete neurale sono:

- funzione di attivazione, che introduce effetti non lineari nella nostra approssimazione (in genere è uguale per tutti gli strati);
- funzione di attivazione in uscita (può essere differente);
- l'architettura che include il numero di input p , il numero di outputs q , il numero L di layers, il numero $m(\ell = 1), \dots, m(\ell = L - 1)$ di nodi per layer, e i pesi e i bias di ciascun nodo della rete $w_{ij}^{(\ell)}, b_j^{(\ell)}$, in generale tutti questi parametri vengono indicati con il simbolo θ ;
- le possibili tecniche di regolarizzazione che vengono implementate (come ad esempio Ridge, Lasso, ecc...);
- le funzioni di costo da ottimizzare

Funzione di attivazione Le possibili funzioni di attivazione che vengono utilizzate per i nodi della rete neurale sono elencate nella sezione 6.1.2.

Funzione di attivazione di output Spesso è possibile che lo strato di output abbia una funzione di attivazione differente da quella degli altri nodi. In particolare nel caso di classificazioni binarie in cui le variabili di uscita non sono mutualmente esclusive si utilizza la funzione di attivazione sigmoide, mentre se sono esclusive si tende ad usare il softmax per via della normalizzazione.

Funzione costo Di funzioni di costo, normalmente si utilizzano la MSE (regressione), per cui $C \approx \sum_i (y_i - \hat{y}_i)^2$, oppure la cross-entropy (regressione logistica), per cui $C \approx -\log(\prod_i p_i^{y_i})$.

Architettura L'architettura dipende dal numero $L - 1$ di strati latenti (ovvero dalla *profondità*) della rete e dal numero di nodi di ogni strato, ovvero p per lo strato in ingresso (numero di nodi in ingresso pari al numero di input), q per lo strato in uscita (pari ai possibili output) e $m(\ell = 1), \dots, m(\ell = L - 1)$ per gli strati intermedi (ampiezza). Le versioni tradizionali del teorema di approssimazione universale riguardano la larghezza, per cui si ipotizza di poter avere infiniti nodi; è stato dimostrato che se si prende una larghezza limitata pari a $m(\ell) = p + 4$, aumentando fino all'infinito la profondità si riesce ad approssimare una funzione qualunque.

Regolarizzazione I due tipi più diffusi sono il metodo Ridge e LASSO, per cui si aggiunge alla funzione di costo la norma dei parametri, per cui:

$$C \rightarrow C + \frac{\lambda}{2} \underline{\theta}^T \underline{\theta} \text{ (Ridge)}$$
$$C \rightarrow C + \lambda ||\underline{\theta}||_1 \text{ (LASSO)}$$

Training Il training sostanzialmente si fa utilizzando il metodo SGD, che è l'algoritmo grazie al quale le reti neurali sono diventate importanti. Ci sono diversi metodi che si possono utilizzare, come standard, momentum SGD, Adegrad, RMS pro, ADAM, ecc...

C'è il problema che la nostra rete neurale è molto complessa, e se si fa la discesa del gradiente, dobbiamo calcolare $\nabla_{\underline{\theta}} C(\underline{\theta})$, che non è un conto banale, nonostante sappiamo di poter scomporre $C = \sum_i \bar{C}_i(\underline{x}_i, \underline{\theta})$ e nonostante stiamo lavorando con funzioni lineari. La strategia da utilizzare è la *back-propagation*

6.4 Back-Propagation Algorithm

L'algoritmo di back-propagation è un algoritmo di ottimizzazione che viene utilizzato in maniera estesa nell'allenamento di reti neurali, soprattutto nel caso di multilayer perceptron, FFNN, DNN e anche CNN. Per l'ottimizzazione delle reti ricorsive e ricorrenti invece viene utilizzata una variante dell'algoritmo di back-propagation nota come BPTT o *Back Propagation Through Time*.

Step fondamentali dell'utilizzo di BPP L'algoritmo di BPP viene utilizzato in fase di ottimizzazione della rete neurale, in particolare è responsabile per il calcolo dei gradienti della funzione di loss rispetto ai pesi della rete.

1. Forward Pass: Durante la fase di forward pass, l'input della rete neurale viene propagato attraverso gli strati della rete per ottenere una previsione.
2. Calcolo della Loss (Errore): La previsione ottenuta durante il forward pass viene confrontata con l'output desiderato, e viene calcolata la loss, che rappresenta la discrepanza tra la previsione della rete e la risposta attesa.
3. Backward Pass (Back-propagation): In questa fase, l'algoritmo di back-propagation viene utilizzato per calcolare i gradienti della loss rispetto ai pesi della rete. Questo coinvolge l'applicazione della regola della catena per calcolare come la loss varia rispetto ai pesi di ciascun strato della rete.
4. Aggiornamento dei Pesi: Una volta calcolati i gradienti, l'algoritmo di ottimizzazione (solitamente SGD o una sua variante) viene utilizzato per aggiornare i pesi della rete in modo che la loss sia ridotta. L'importanza dell'algoritmo di back-propagation è evidente in questa fase, poiché fornisce i gradienti necessari per guidare l'aggiornamento dei pesi in direzione della minimizzazione della loss.

L'algoritmo di back-propagation è fondamentale perché consente di calcolare in modo efficiente i gradienti della loss rispetto ai pesi della rete, rendendo possibile l'aggiornamento dei pesi in maniera appropriata durante l'addestramento. Questo è cruciale per far apprendere alla rete neurale dai dati e migliorare le sue prestazioni nel tempo.

6.4.1 Dimostrazione

Questo algoritmo deriva direttamente dall'applicazione ricorsiva della regola di derivazione di funzione di funzione. In questo caso, i conti e la dimostrazione che vedremo noi riguardano reti FFNN con $L - 1$ strati latenti indicati e caratterizzati come segue:

$$\underline{h}^{(\ell)} = f([\mathbf{W}^{(\ell)}]^T \underline{h}^{(\ell-1)} + \underline{b}^{(\ell)}) = f(\underline{z}^{(\ell)})$$

con $m(\ell)$ nodi, dove la funzione di attivazione f non è una funzione lineare (altrimenti parleremmo di regressione lineare).

Lo strato finale è

$$\underline{\hat{y}} = a([\mathbf{W}^{(L)}]^T \underline{h}^{(L-1)} + \underline{b}^{(L)}) = a(\underline{z}^{(L)})$$

dove a è una funzione di attivazione che può essere differente da f .

Il set di dati di apprendimento è

$$\mathcal{D} = \{(t_i, \underline{x}_i) : t_i \in \mathbb{R}^q, \underline{x}_i \in \mathbb{R}^p, i = 1, \dots\};$$

la funzione di costo sarà (con funzione di costo anche diversa per ogni campionamento)¹:

$$C(\{\underline{\theta}\}) = \sum_i \tilde{C}(t_i, \underline{y}_i);$$

dove

$$\{\underline{\theta}\} = \{\mathbf{W}^{(\ell)}, \underline{b}^{(\ell)} : \ell = 1, \dots, L, \mathbf{W}^{(\ell)} \in \mathbb{R}^{m^{(\ell-1)} \times m^{(\ell)}}, \underline{b}^{(\ell)} \in \mathbb{R}^{m^{(\ell)}}\}.$$

Abbiamo quindi:

$$\begin{aligned} h_i^{(\ell)} &= f(z_i^{(\ell)}) & z_i^{(1)} &= \sum_{j=1}^p w_{ji}^{(1)} x_j + b_i^{(1)} \\ \hat{y}_k &= a(z_k^{(L)}) & z_k^{(\ell)} &= \sum_i^{m^{(\ell-1)}} w_{ik}^{(\ell)} h_i^{(\ell-1)} + b_k^{(\ell)} \end{aligned}$$

e per ognuno dei parametri vogliamo farne l'update

$$\theta_r^{(n+1)} \longleftarrow \theta_r^{(n)} - \eta \frac{\partial \tilde{C}}{\partial \theta_r};$$

abbiamo quindi bisogno di calcolare la derivata della funzione di costo rispetto ad ogni parametro.

Per lo strato finale ci sono alcune differenze, poiché contiene le $\hat{y}$ che vanno inserite direttamente nella funzione di costo e poiché assumiamo che la funzione di attivazione a sia un pochino differente da f . In generale, la derivata rispetto ad un qualunque parametro θ_r non è nient'altro che²

$$\begin{aligned} \frac{\partial \tilde{C}(\underline{t}, \underline{y})}{\partial \theta_r} &= \sum_{k=1}^q \frac{\partial \tilde{C}(\underline{t}, \underline{y})}{\partial z_k^{(L)}} \frac{\partial z_k^{(L)}}{\partial \theta_r} = \sum_k \delta_k^{(L)} \frac{\partial z_k^{(L)}}{\partial \theta_r} \\ \delta_k^{(L)} &\equiv \frac{\partial \tilde{C}}{\partial z_k^{(L)}} = \frac{\partial \tilde{C}}{\partial \hat{y}_k} \frac{da_k(z)}{dz} \Big|_{z=z_k}, \end{aligned}$$

infatti, visto che passiamo da uno strato al successivo passando attraverso gli input che vengono dati ad ogni funzione di attivazione (il link tra ogni strato non è altro che $f(z)$, oppure $a(z)$ nel caso dell'ultimo strato), diventa facile usare le z come punto di separazione tra uno strato e l'altro. Una volta che abbiamo le $z_k^{(L)}$, che sono i valori di input alla funzione di attivazione finale, non ci dobbiamo preoccupare dei valori che hanno assunto i nodi degli strati precedenti. La funzione di costo dipenderà unicamente da $z_k^{(L)}$; ogni stato dipende solo dagli z_k che ha in input.

Se θ_r è uno dei parametri dell'ultimo layer, quindi

$$\theta_r \in \{\mathbf{W}^{(L)}, \underline{b}^{(L)}\}$$

¹Questa assunzione permette di calcolare i gradienti dividendo in mini-batch; le funzioni di costo di ogni campionamento non dipendono dal set di training.

²Questo è possibile perché stiamo lavorando con una FFNN, che non permette dei feedback negli strati precedenti: possiamo usare la regola di derivazione di funzione di funzione.

allora:

$$\frac{\partial z_k^{(L)}}{\partial w_{ij}^{(L)}} = \delta_{kj} h_i^{(L-1)}$$

$$\frac{\partial z_k^{(L)}}{\partial b_j^{(L)}} = \delta_{kj}$$

ed è un risultato generale, si applica anche agli altri strati, ma per lo strato finale, possiamo scrivere

$$\frac{\partial \tilde{C}}{\partial w_{ij}^{(L)}} = \delta_j^{(L)} h_i^{(L-1)}$$

$$\frac{\partial \tilde{C}}{\partial b_j^{(L)}} = \delta_j^{(L)}$$

Immaginiamo di avere un parametro della rete che corrisponde ad una combinazione lineare in uno strato nascosto interno alla rete, avremo:

$$\begin{aligned} \frac{\partial \tilde{C}}{\partial \theta_r^{(\ell)}} &= \sum_k \frac{\partial \tilde{C}}{\partial z_k^{(L)}} \frac{\partial z_k^{(L)}}{\partial \theta_r^{(\ell)}} = \sum_k \frac{\partial \tilde{C}}{\partial z_k^{(L)}} \sum_i \frac{\partial z_k^{(L)}}{\partial z_i^{(L-1)}} \frac{\partial z_i^{(L-1)}}{\partial \theta_r^{(\ell)}} \\ &= \sum_{k,i,j,\dots,s} \delta_k^{(L)} \frac{\partial z_k^{(L)}}{\partial z_i^{(L-1)}} \frac{\partial z_i^{(L-1)}}{\partial z_j^{(L-2)}} \dots \frac{\partial z_s^{(\ell)}}{\partial \theta_r^{(\ell)}} \\ &\equiv \sum_{s=1}^{m(\ell)} \delta_s^{(\ell)} \frac{\partial z_s^{(\ell)}}{\partial \theta_r^{(\ell)}}. \end{aligned}$$

Il valore di $\delta_s^{(\ell)}$ si ottiene in maniera ricorsiva, infatti, dal momento che $f(z_i^{(\ell-1)}) = h_i^{(\ell-1)}$, abbiamo:

$$\begin{aligned} \frac{\partial z_k^{(\ell)}}{\partial z_j^{(\ell-1)}} &= \sum_i w_{ik}^{(\ell)} \frac{\partial h_i^{(\ell-1)}}{\partial z_j^{(\ell-1)}} = \sum_i w_{ik}^{(\ell)} \frac{df(z_i^{(\ell-1)})}{dz_j^{(\ell)}} = w_{jk}^{(\ell)} \frac{df}{dz} \Big|_{z_j^{(\ell-1)}} \\ \delta_s^{(\ell)} &= \sum_{i=1}^{m(\ell+1)} [f'(z_s^{(\ell)}) w_{si}^{(\ell+1)}] \delta_i^{(\ell+1)} \quad \forall \ell = 1, 2, \dots, L-1. \end{aligned}$$

Riassumendo, dobbiamo calcolare:

$$\delta_k^{(L)} = \frac{\partial \tilde{C}}{\partial \hat{y}_k} a'_k(z_k^{(L)}) \quad \forall k = 1, \dots, q$$

$$\delta_i^{(\ell)} = \sum_{j=1}^{m(\ell+1)} \delta_j^{(\ell+1)} w_{ij}^{(\ell+1)} f'(z_i^{(\ell)}) \quad \forall i = 1, \dots, m(\ell)$$

dove le derivate sono ottenute nel FFNN, quindi se si mantengono in memoria è facile fare il lavoro a ritroso.

Negli strati intermedi si ottiene:

$$\begin{aligned} {}^{n+1}w_{ij}^{(\ell)} &\leftarrow {}^n w_{ij}^{(\ell)} - \eta \delta_j^{(\ell)} h_i^{(\ell-1)} \\ {}^{n+1}b_j^{(\ell)} &\leftarrow {}^n b_j^{(\ell)} - \eta \delta_j^{(\ell)} \end{aligned}$$

Mostriamo adesso alcuni dei casi più utilizzati:

- nella funzione ReLU abbiamo

$$f(z) = \max\{0, z\} \quad \Rightarrow \quad f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0; \end{cases}$$

- il sigmoide è nel caso di classificazione binaria, ed ha:

$$f(z) = \sigma(z) = \frac{e^z}{1 + e^z} \quad \Rightarrow \quad \sigma'(z) = \sigma(z)[1 - \sigma(z)];$$

- il softmax è l'estensione del sigmoide per classificazioni a multiclassi:

$$a(z_k) = \frac{e^{z_k}}{\sum_i e^{z_i}} \quad \Rightarrow \quad \frac{da(z_k)}{dz_j} = a(z_j)[\delta_{kj} - a(z_k)]$$

e la derivata dipende sia dall'indice j che dall'indice k .

Le funzioni di costo più utilizzate sono:

- MSE:

$$\begin{aligned} \tilde{C}(\underline{t}, \underline{\hat{y}}) &= \frac{1}{2}(\underline{t} - \underline{\hat{y}})^T(\underline{t} - \underline{\hat{y}}) = \frac{1}{2} \sum_{k=1}^q (t_k - \hat{y}_k)^2 \\ \frac{\partial \tilde{C}}{\partial \hat{y}_k} &= \hat{y}_k - t_k = a(z_k^{(\ell)}) - t_k; \end{aligned}$$

- Cross-entropy, che deriva dal metodo MLE³:

$$\begin{aligned} \tilde{C}_{\text{CE}}(\underline{t}, \underline{\hat{y}}) &= -\log(\text{MLE}) = -\sum_{k=1}^q t_k \log \hat{y}_k + (1 - t_k) \log(1 - \hat{y}_k) \\ \frac{\partial \tilde{C}}{\partial \hat{y}_k} &= \frac{\hat{y}_k - t_k}{\hat{y}_k(1 - \hat{y}_k)}. \end{aligned}$$

3

$$\max \prod_{k=1}^q (p_k)^{t_k} (p_{-k})^{1-t_k} = \max \prod \mathcal{G}_k^{t_k} (1 - \hat{y}_k)^{t_k}$$

6.5 Convolutional Neural Network

Le reti neurali convoluzionali sono reti di tipo feed forward, quindi a strati che si susseguono e con l'informazione che si propaga da sinistra verso destra; tuttavia l'architettura complessiva è un pochino differente.

Le reti neurali di tipo convoluzionale sono basate sull'idea di diminuire il numero di parametri della rete; hanno struttura simile alla corteccia cerebrale degli animali dedicata alla visione e sono alla base di algoritmi per il riconoscimento delle immagini (computer vision, risonanze magnetiche, ecc...). Sono reti che fanno una convoluzione di punti locali nella rete (neuroni corrispondenti a dati che sono fisicamente correlati o vicini l'uno all'altro), riducendo il numero di neuroni trattati. Queste reti estraggono comportamenti e strutture microscopiche all'interno di un set di dati correlati nel sistema fisico.

Le Reti neurali convoluzionali (convolutional neural networks – CNN) sono state sviluppate principalmente durante gli anni '90, con l'obiettivo principale di assolvere compiti di riconoscimento dei caratteri (per esempio, la lettura di codici postali). Al giorno d'oggi CNN sono un elemento centrale nello spettacolare successo dei metodi di apprendimento automatico 'profondo'. La loro efficienza, per esempio per la classificazione di immagini, le ha rese uno dei metodi centrali in ML. Le CNN sono molto simili alle reti neurali ordinarie (FFNN) e sono costituite da neuroni (o perceptor) connessi attraverso pesi e bias che devono essere ottimizzati: ogni neurone riceve dei valori in ingresso dallo strato precedente, ne produce una combinazione lineare e opzionalmente applica una trasformazione non lineare. L'intera rete produce un singolo risultato, per esempio una score function: da una immagine formata di pixel in ingresso si arriva ad una serie di probabilità per classificazione dal lato opposto (di tipo softmax).

Le CNN hanno anch'esse una funzione di costo (per esempio, OLS o entropia incrociata) e uno strato finale completamente connesso (FFNN). Tutti gli strumenti già visti per le reti neurali standard (discesa stocastica del gradiente, propagazione a ritroso, ecc...) si applicano anche in questo caso.

Qual'è la differenza? Le architetture di tipo CNN partono dall'assunzione esplicita che i dati in ingresso siano immagini, il che permette di fare scelte mirate sulla struttura della rete. Questo rende l'implementazione delle connessioni fra strati successivi mirata e quindi più facile da implementare e riduce drasticamente il numero di parametri del modello (rispetto ad una rete FFNN corrispondente).

6.5.1 Reti Neurali Standard vs. Reti Neurali Convoluzionali

Le reti neurali comuni, FFNN, sono definite tramite trasformazioni affini, in cui un vettore di valori in ingresso viene moltiplicato per una matrice di parametri (tipicamente chiamati pesi) per produrre un risultato (a cui generalmente si aggiunge un vettore di bias prima di passare attraverso una funzione di attivazione non lineare). Questa procedura può essere applicata ad ogni tipo di valori, che sia un'immagine, un segnale sonoro qualunque altro insieme di caratteristiche non ordinate: qualunque sia la dimensionalità dei dati, questi possono sempre essere riorganizzati in un vettore lineare prima della trasformazione affine.

Tuttavia, quando si considerano immagini, registrazioni sonore e simili, i dati hanno una struttura intrinseca. Più precisamente, condividono alcune proprietà rilevanti:

- I dati sono organizzati in array multidimensionali (per esempio i pixel di una figura)
- Vi sono uno o più assi lungo i quali l'ordine dei dati è importante (lunghezza e altezza in una figura, tempo in una registrazione, ecc...).
- Può esservi un asse di 'canali' che permette di avere visioni complementari dei dati (esempio, RGB per le immagini od i segnali L ed R in tracciato stereo).

Questo tipo di proprietà non è sfruttato da trasformazioni affini; di fatto, queste ultime generano una trasformazione completamente connessa e omogenea di tutti i dati in input, senza tenere conto degli assi e quindi trascurando informazioni sulla topologia del sistema. Essere in grado di trarre vantaggio della struttura implicita dei dati può invece rivelarsi estremamente utile in per alcuni compiti, come visione computerizzata o riconoscimento di parole. Per far ciò tornano utili le trasformazioni di convoluzione (discretizzate).

Convoluzione discretizzata Una convoluzione discretizzata è una trasformazione lineare che conserva le proprietà di ordinamento dei dati. È sparsa (ad ogni dato di output contribuisce solo un piccolo sottoinsieme, locale, dei dati in ingresso) e riutilizza gli stessi parametri (gli stessi pesi sono applicati localmente in posizioni diverse del dataset).

Immaginiamo di avere un'immagine di 32 pixel, con $32 \times 32 \times 3 = 3072$ parametri p (il $\times 3$ è dato dalla presenza dei tre canali RGB), dobbiamo quindi avere 3072 pesi per ogni neurone del primo strato nascosto (assumendo di avere una rete neurale FF completamente connessa) $\mathbf{W} \in \mathbb{R}^{3072 \times m_1}$: in una situazione con anche solo $m_1 = 50$, il numero di pesi unicamente per il primo strato è 153600. Un'immagine un po' più realistica ha dimensione $200 \times 200 \times 3 = 120000$ parametri ed il numero di pesi sarà $120000 \times 50 = 6000000$: il numero di parametri è completamente proibitivo e non siamo interessati alla connessione di tutti i neuroni, ma vogliamo che tramite una convoluzione siano estratti caratteristiche nei dati in un formato locale, non è necessario che tutte le connessioni siano importanti.

Le reti neurali convoluzionali (CNN) partono dalla 'assunzione che gli dati in ingresso consistano di immagini e se ne avvantaggiano costruendo l'architettura in maniera più ragionevole. Differentemente dalle reti standard, gli strati delle CNN hanno i neuroni disposti lungo tre dimensioni: larghezza, altezza e profondità (notare che in questo caso il termine profondità non si riferisce alla terza direzione nello spazio, né alla 'profondità' della rete neurale che invece si riferisce al numero di strati utilizzato). Nell'esempio precedente di una immagine $32 \times 32 \times 3$, la rete è strutturata in 3D ed il numero di colori (3) rappresenta la profondità del volume di attivazione. I neuroni in uno strato saranno connessi solamente ad una piccola regione (in 3D) dello strato precedente, in contrasto ad una rete completamente connessa. Inoltre lo strato finale di una immagine simile avrà dimensioni $1 \times 1 \times P$ (tipicamente con profondità $P \leq 3$). Questo perché alla fine dell'architettura CNN l'immagine sarà ridotta ad un singolo vettore di probabilità (scores) arrangiate lungo la dimensione di profondità.

6.5.2 Convoluzione

Nelle reti neurali FFNN, si fa una trasformazione affine: dal vettore in ingresso $\underline{x}$ la cui combinazione lineare è un vettore $\underline{s} = \mathbf{W} \cdot \underline{x}$. Nelle reti neurali convoluzionali

(CNN), andiamo a fare una convoluzione del vettore in ingresso

$$s(t) = \int x(t-a)w(a)da$$

dove la funzione $w(a)$ serve ad estrarre un comportamento locale dei dati: se ad esempio avessimo dati della temperatura di una stanza ogni secondo, caratterizzati da un errore normale, la funzione $w(a)$ sarebbe quella che riesce a riconoscere l'errore e a rimuoverlo, ottenendo la temperatura "vera" $s(t)$.

Se consideriamo il caso di un'immagine in due dimensioni, il segnale sarà

$$\begin{aligned} s(i, j) &= (\mathbf{x} * w)(i, j) = \sum_{n, m} \mathbf{x}(i-m, j-n)w(m, n) \\ &= \sum_{n', m'} w(i-m', j-n')(m', n') = (w * \mathbf{x})(i, j) \end{aligned}$$

dove $i, j = 1, \dots, (\# \text{ di pixel})$. Noi vogliamo una funzione dei pesi che sia locale. Se limitiamo $m, n = -2, -1, 0, 1, 2$, ovvero correliamo 5 pixel di input in una direzione e nell'altra, ci ritroviamo con 25 pesi, che descrivono la struttura localizzata in un punto della nostra griglia.

Immaginiamo una matrice di input $x(i, j) 3 \times 4$ che vogliamo contrarre con dei pesi $\dim \mathbf{W} = 2 \times 2$. La convoluzione tra queste due matrici è una matrice $\mathbf{Z}$ di dimensione 2×3 , in cui ogni elemento è la somma pesata dei 4 pixel corrispondenti nella matrice di partenza.

α	δ	η	κ
β	ε	θ	λ
γ	ζ	ι	μ

a	c
b	d

A	Γ	E
B	Δ	Z

I primi termini di Z saranno quindi:

$$\begin{aligned} A &= \alpha a + \beta b + \delta c + \varepsilon d \\ B &= \beta a + \gamma b + \varepsilon c + \zeta d \\ \Gamma &= \delta a + \varepsilon b + \eta c + \theta d. \end{aligned}$$

In questo modo, partiamo dagli input, applichiamo la convoluzione ed otteniamo le componenti di Z (che sono minori in numero) ed applichiamo la funzione di attivazione non lineare $f(z)$. C'è un ulteriore step, il *pooling* (sorteggio), in cui i valori di attivazione h_i ottenuti vengono ridotti ad un numero ancora più ridotto.

Se abbiamo dei neuroni $f(z_i)$ con $i = 1, \dots, m$, possiamo prendere i primi k valori ed aggiungerli in un nuovo neurone $h_1 = \max(f(z_1), f(z_2), \dots, f(z_k))$ (questo è il max-pooling), e così per i successivi gruppi di k neuroni. Spesso si usa il max-pooling perché i valori di $h_1, h_2, \dots$ riflettono quanto la caratteristica spaziale che cerchiamo con la matrice W si adatti al gruppo di valori di input considerati e quindi saranno maggiori se i valori di input si riflettono bene in tale caratteristica. Raggruppando i valori h_i nello strato di pooling, scegliendone il massimo, si dà importanza ai valori di input che più si riflettevano nella caratteristica cercata. Queste strutture (a tre strati) di convoluzione, si possono ripetere fino ad arrivare ad uno strato finale che è *all-connected*: tutti i neuroni dello strato finale sono connessi con tutti i neuroni dell'ultimo strato di pooling in tutte le direzioni possibili.

Ogni volta che si fa una convoluzione si estraggono caratteristiche dei dati. Se non si hanno tante classi, ad ogni strato si fanno convoluzioni parallele ad ogni passo, fino, potenzialmente, ad avere tante convoluzioni fatte che hanno portato ad un unico neurone che vengono contratte nello strato finale. Non si fa un'unica convoluzione, ma tante in parallelo, in modo da estrarre caratteristiche diverse: si separano diversi comportamenti della rete. In figura 6.10 è mostrata la struttura di due CNN: nell'immagine

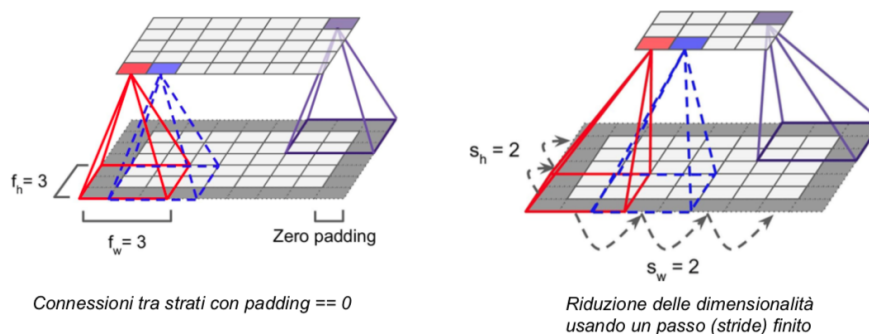


Figura 6.10

a sinistra abbiamo un'immagine filtrata, che però mantiene la stessa dimensionalità, ma siccome le caratteristiche che si ricercano solitamente sono a scala maggiore della dimensione dei pixel, spesso si utilizzano CNN che riducono la dimensionalità, come nell'immagine a destra.

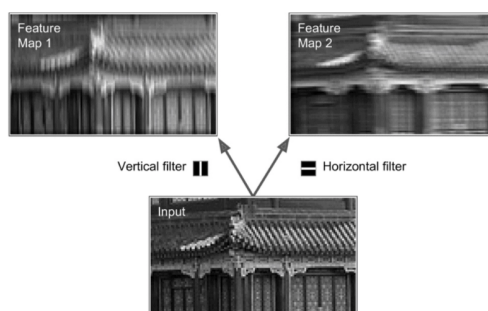


Figura 6.11

Nella figura 6.11 sono mostrate le applicazioni di due filtri differenti, uno verticale ed uno orizzontale, utili se si vogliono individuare strutture macroscopiche (come colonne, tetto, ecc...) e se non si è interessati a strutture di scala minore (come i ghirigori sulle finestre).

In figura 6.12 è mostrato il pooling di un'immagine, che porta l'immagine a colori in un'immagine più piccola ed in cui ogni 4 pixel viene riportato unicamente quello con colore più intenso.

In figura 6.13 è mostrata la struttura di una CNN con più layer convoluzionali: da un'immagine ad alta definizione, si sovrappongono tante immagini a definizione ridotta ma che estraggono determinate caratteristiche. Ogni gruppo dei dati iniziali viene filtrato in un pixel e mappato in una direzione; dopodiché viene ridotto con il pooling e viene sviluppato in un ulteriore layer convoluzionale. Un'esempio è in

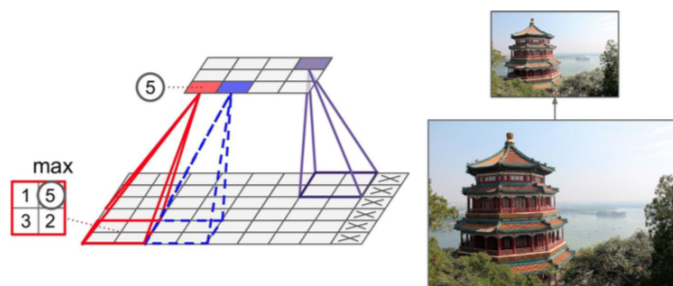


Figura 6.12

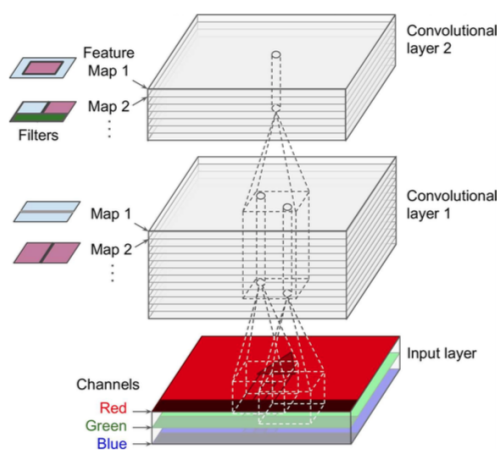


Figura 6.13

figura 6.14. Alla fine dell'ultimo pooling si crea una rete neurale di tipo FFNN di classificazione.

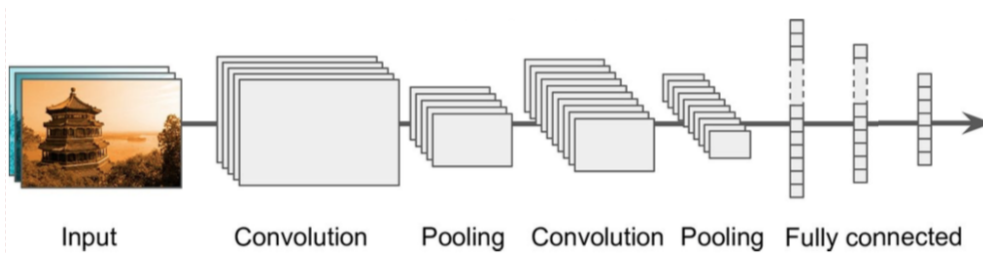


Figura 6.14

6.6 Recurrent Neural Networks

Le reti neurali ricorrenti si utilizzano per riconoscere sequenze temporali (come riconoscimenti vocali, Shazam, ecc...) e per trattarle; i dati che si hanno sono tutti correlati in una direzione temporale (o spaziale) e bisogna mantenere questa correlazione. Immaginiamo di avere una sequenza temporale $s(t)$ con dei parametri di controllo esterni $x(t)$ (forze esterne che influiscono sul moto). Una giustificazione un po' blanda è: se si ha una quantità dipendente dal tempo e se si vuole integrarla nel tempo, il metodo più semplice è calcolare con il metodo di Eulero:

$$\frac{ds(t)}{dt} = g(s, x, t) \quad \Rightarrow \quad s(t+w) = s(t) + wg(s(t), x(t), t)$$

si vede come siano necessari i valori del gradiente e come si trasforma in una regressione lineare. Si costruisce, quindi, la rete su strutture del tipo:

$$s(t) = f(s(t-1), \theta) = a(s(t-1) + \theta h(t-1)),$$

dove f è una funzione di attivazione non lineare.

Teoria dei Grafi

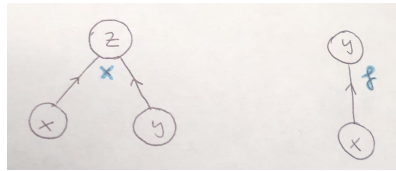


Figura 6.15: Rappresentazione su un grafo di prodotto algebrico e applicazione di una funzione

Per capire le RNN sono utili alcune informazioni che derivano dalla teoria dei grafi. Le variabili vengono disegnate in cerchietti che vengono collegati tra di loro. In figura 6.15 a sinistra è mostrata la rappresentazione di $z = x \times y$ (dove si intende il prodotto algebrico), mentre a destra si ha la rappresentazione di $y = f(x)$. Dipendenze più complesse sono date dalla combinazione di strutture più semplici. Immaginiamo di avere un numero finito di passi, per esempio $t = 1, \dots, \tau$ con $\tau = 3$: possiamo scrivere

$$\begin{aligned} s^{(2)} &= f(s^{(1)}; \theta) \\ s^{(3)} &= f(s^{(2)}; \theta) = f(f(s^{(1)}; \theta); \theta) \end{aligned}$$

che verrà rappresentato come in figura 6.16.



Figura 6.16: Rappresentazione di una struttura ricorsiva sotto forma di grafo

Immaginiamo il caso di un sistema dinamico forzato esternamente (un pendolo), per cui

$$m \frac{d^2 s}{dt^2} = -k \sin\left(\frac{s}{l}\right) + A \cos(\omega t).$$

Utilizzando il metodo di Eulero otteniamo:

$$\begin{cases} \frac{ds}{dt} = v(s, t) \\ \frac{dv}{dt} = -\frac{k}{m} \sin\left(\frac{s}{l}\right) + A \cos(\omega t) \end{cases} \quad \begin{cases} s(t+dt) = s(t) + dtv(t) \\ v(t+dt) = v(t) + dt[A \cos(\omega t) - \frac{k}{m} \sin\left(\frac{s}{l}\right)] \end{cases}$$

La sequenza delle velocità va creata in funzione di parametri esterni, e la sequenza dei neuroni (che danno le posizioni) dipende da quest'ultima sequenza. Bisogna creare un layer di valori di s ed una sequenza latente che raccolga le velocità. Abbiamo, se u rappresenta la forza esterna:

- segnale esterno:

$$s^{(t+1)} = f(s^{(t)}, x^{(t+1)}, \theta);$$

- strato latente:

$$h^{(t)} = \rho(x^{(t)}u + h^{(t-1)}w + b);$$

che graficamente è raffigurata dalla figura 6.17.

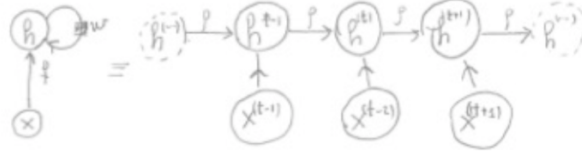


Figura 6.17: Esempio di utilizzo di un grafo per la descrizione di un sistema

Esempi di reti neurali ricorrenti

Vediamo ora due architetture semplici. Nel primo caso la rete RNN produce un segnale di uscita $y(t)$ per ogni passo temporale e le connessioni sono ricorsive tra i neuroni latenti $h(t_i) \rightarrow h(t_{i+1})$. Nel secondo caso la rete RNN produce un segnale di uscita $y(t)$ per ogni passo temporale e ha connessioni ricorsive tra i valori di uscita $y(t_i)$ ed i nodi latenti al passo successivo $h(t_{i+1})$.

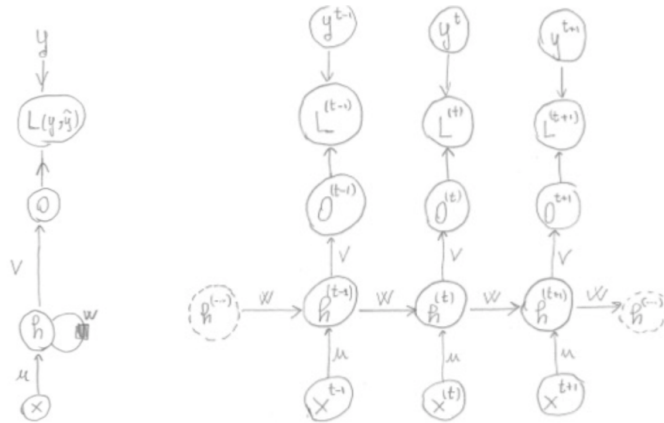


Figura 6.18: Rappresentazione fold e unfold del primo modello di RNN

Modello 1 La rappresentazione grafica del primo modello è in figura 6.18. Assumiamo che il dataset da cui vengono presi i dati di input sia

$$\mathcal{D} = \{(y(t_i), \underline{x}(t_i)) : i = 1, 2, 3 \dots\}.$$

Le operazioni che si svolgono sono:

1. per prima cosa l'input $\underline{x}$ pesato tramite $\underline{u}$ viene sommato all'informazione che arriva dal precedente hidden layer per ottenere l'argomento della funzione di attivazione:

$$\underline{a}^{(t)} = \underline{b} + \underline{w}^T \underline{h}^{(t-1)} + \underline{u}^T \underline{x}^{(t)};$$

2. viene valutato l'hidden layer applicando una funzione di attivazione ad $\underline{a}(t)$:

$$\underline{h}^{(t)} = \tanh(\underline{a}^{(t)}), \sigma(\underline{a}^{(t)}), \text{ReLU}(\underline{a}^{(t)}), \dots;$$

3. viene prodotto un output $\underline{o}(t)$

$$\underline{o}^{(t)} = \underline{c} + \underline{V}^T \underline{h}^{(t)};$$

4. viene calcolato l'output finale

$$\hat{\underline{y}}^{(t)} = \text{Softmax}(\underline{o}^{(t)}), \sigma(\underline{o}^{(t)}), \dots;$$

5. viene calcolata la funzione costo finale

$$L = \sum_i \tilde{C}(\underline{y}(t_i), \hat{\underline{y}}(t_i); \beta_{\text{reg}}).$$

È necessario che i dati non vengano mescolati, poiché la sequenza temporale è fondamentale, e $x(t)$ non dipende dai parametri della rete θ . L'addestramento è molto complesso, poiché avviene tramite la discesa del gradiente (stocastica o non), in cui si devono calcolare sette derivate diverse:

$$\nabla_{\underline{h}} L \quad \nabla_{\underline{o}} L \quad \nabla_{\underline{c}} L \quad \nabla_{\underline{b}} L \quad \nabla_{\underline{v}} L \quad \nabla_{\underline{w}} L \quad \nabla_{\underline{u}} L$$

sono molti dati e non banali da valutare, poiché i dati non si sommano in maniera arbitraria, ma si deve prendere tutto il risultato della rete e lo si fitta su tutta la sequenza temporale. A questo punto, si esegue un FF da sinistra a destra seguito dalla back-propagation attraverso il tempo. L'ottimizzazione è molto costosa a livello di calcolo ed è difficile la parallelizzazione.

Modello 2 In questo esempio, schematizzato in figura 6.19, il valore di uscita fa il fit sul neurone latente. Nel caso del pendolo sopracitato, la posizione dipende dalla velocità, che a sua volta dipende dalla posizione; dobbiamo trasporre questa caratteristica nella rete neurale. Assumiamo ancora una volta che il dataset di partenza sia

$$\mathcal{D} = \{(y(t_i), \underline{x}(t_i)) : i = 1, 2, 3 \dots\}.$$

In questo caso, nella definizione dell'architettura dobbiamo tenere presente che o alimenta h e h (che dipende da o e da x) alimenta o , quindi verranno calcolati come segue:

$$\underline{h}^{(t)} = f(\underline{b} + \underline{u}^T \underline{x}^{(t)} + \underline{w}^T \underline{o}^{(t-1)}); \quad \underline{O}^{(t)} = a(\underline{c} + \underline{v}^T \underline{h}^{(t)}) = \hat{\underline{y}}^{(t)}.$$

Oltre ai due modelli che abbiamo visto qui esistono altri modelli più complessi di RNNs, tra cui ricordiamo:



Figura 6.19: Rappresentazione unfold del secondo modello di RNN

- RNN bidirezionali;
- Deep RNN;
- Gated RNN (GRU);
- Long and Short-term RNN (LTN,STN).

Ricapitolando Nell'applicazione di una rete neurale ricorsiva bisogna:

- decidere la metrica: funzione di costo, accuratezza;
- decidere l'architettura: pensare ai colli di bottiglia, pensare alle caratteristiche che si vuole che la rete impari;
- andare per piccoli passi nel caso di dati che si aggiungono un po' per volta;
- capire quale sia la velocità di apprendimento ottimale (utile la grid search che viene utilizzata più spesso della random search);
- a maggior ragione non si possono mischiare i dati perchè si rischia di far apprendere alla rete false sequenze temporali.

6.7 Equazioni Differenziali Ordinarie

Le reti neurali si possono applicare alla risoluzione di ODE grazie ai teoremi di approssimazione universale, secondo cui con un solo strato latente è possibile approssimare qualsiasi funzione con un'accuratezza arbitraria. Questi teoremi garantiscono di poter rappresentare anche le soluzioni di equazioni differenziali ordinarie.

Tipicamente, ODE possono essere risolte numericamente attraverso l'uso di espansioni di Taylor e di differenze finite che permettono di ottenere la funzione incognita in modo accurato attraverso un'integrazione passo per passo.

Ci sono alcuni casi in cui anche con funzioni semplici l'utilizzo di reti neurali è più efficiente di metodi standard, ma normalmente, se il problema è poco complesso, non conviene usare le reti neurali; infatti bisogna ricordare che in questi casi le NN richiedono più lavoro (e cicli di CPU) che l'integrazione standard. Inoltre non tutte le ODE ammettono soluzioni analitiche (ovvero rappresentabili con una NN).

Nei grandi sistemi, con tanti input e potenzialmente anche con tanti output, diviene molto più conveniente utilizzare le reti neurali. Più la rete neurale diventa larga e profonda, più l'accuratezza migliora, ma ci sono anche funzioni non analitiche, che si possono approssimare fino ad un certo punto. Come sempre, per far convergere alla soluzione corretta è cruciale trovare l'architettura adeguata, scegliendo opportunamente la funzione di costo, i pesi (W), i bias (b), ecc..

6.7.1 Note sull'approccio

Di seguito, prima di andare più nel dettaglio viene spiegata per sommi capi l'idea che sta alla base dell'utilizzo di reti neurali per la soluzione di equazioni differenziali.

1. **Rappresentazione della Soluzione:** La rete neurale viene addestrata per approssimare la soluzione di un'ODE. I parametri della rete neurale sono regolati in modo che la soluzione approssimata soddisfi le condizioni al contorno fornite.
2. **Funzione di Costo:** La funzione di costo è definita in modo da incorporare l'ODE originaria. Spesso, si utilizza una forma di norma quadratica che penalizza la differenza tra il valore della derivata predetto dalla rete neurale e il valore effettivo della derivata data dall'ODE.
3. **Metodi di Ottimizzazione:** L'obiettivo dell'addestramento è minimizzare la funzione di costo. Questo viene tipicamente fatto utilizzando algoritmi di ottimizzazione come la discesa del gradiente stocastico.
4. **Campionamento dei Punti:** La funzione costo può coinvolgere il campionamento di punti nell'intervallo di interesse dell'ODE. In altre parole, la rete neurale viene allenata a minimizzare la discrepanza tra i valori predetti e i valori effettivi dell'ODE in diversi punti all'interno dell'intervallo.
5. **Risultati:** Dopo l'allenamento, la rete neurale dovrebbe essere in grado di fornire una buona approssimazione della soluzione dell'ODE in punti non visti durante l'allenamento.

6.7.2 Costruzione di una rete per la soluzione di ODE

In generale si ha un'equazione differenziale della forma

$$f(x, g(x), g'(x), g''(x), \dots, g^{(n)}(x)) = 0$$

dove ricerchiamo la funzione $g(x)$ che la soddisfi, con le condizioni al contorno

$$g(x_0) = g_0 \quad g'(x_1) = g_1 \quad g''(x_2) = g_2 \quad \dots$$

Usando una rete neurale, cerchiamo una soluzione del tipo

$$g_t(x) = h_1(x) + h_2(x, N(x, \underline{\theta}))$$

dove h_1 e h_2 sono scelte in modo che a priori siano imposte le condizioni al contorno, così che la rete neurale abbia meno lavoro da fare; infatti se scegliessimo la soluzione

$$g_t(x) = N(x, \underline{\theta})$$

dovremmo implementare la rete, cercare la funzione di costo e poi imporre le condizioni al contorno, che sarebbe un'enorme spreco di tempo computazionale. Quello che si fa è campionare la funzione

$$\tilde{C}(x, \{\theta\}) = [f(x, g(x), g'(x), \dots, g^{(n)}(x))]^2$$

nell'intervallo di interesse con

$$\vec{x} = \{x_i \in [a, b]; i = 1, \dots, N\} \in \mathbb{R}^N.$$

Quindi la funzione di costo totale sarà:

$$C(\vec{x}, \{\theta\}) = \frac{1}{N} \sum_{i=1}^N \tilde{C}(x_i, \{\theta\}) = \frac{1}{N} \sum_{i=1}^N [f(x_i, g_t(x_i), g'_t(x_i), \dots, g_t^{(n)}(x_i))]^2.$$

Per la minimizzazione si avranno diversi metodi

- discesa del gradiente (GD, SGD, Adam,...);
- sono necessari i gradienti di $g'_t(x, \theta)$ rispetto ai parametri: $\nabla_{\underline{\theta}}$;
- le derivazioni e le backward propagation possono essere molto complesse, per cui usiamo la derivazione automatica (Autograd).

Esempio 1 (Decadimento Esponenziale)

Il problema di Cauchy è:

$$\begin{cases} g'(x) = -\gamma g(x) \\ g(0) = g_0 \end{cases}$$

la cui soluzione sappiamo essere $g(x) = g_0 e^{-\gamma x}$.

Per le funzioni di prova possiamo scrivere

$$g_t(x, \{\theta\}) = h_1(x) + h_2(x, N(x, \{\theta\})) = g_0 + x N^{\text{FF}}(x, \{\theta\})$$

in modo che per $x = 0$ valga $g_t(0) = g_0$ e che la rete neurale si approssimi pian piano a $g_T(x)/x$. Possiamo vedere un esempio in cui si ha un layer nascosto, che avrà la struttura in figura 6.20. Il numero di parametri se $p = q = 1$ sarà $3N^h + 1$ dove N^h è il numero di strati nascosti e possiamo scegliere come funzione di attivazione il sigmoide

$$z(a) = \frac{1}{[1 + \exp(-a)]};$$

si minimizza

$$\hat{\underline{\theta}} = \min_{\underline{\theta}} \left\{ \frac{1}{N} \sum_i^N [g'_t(x_i, \underline{\theta}) + \gamma g(x_i, \underline{\theta})]^2 \right\}$$

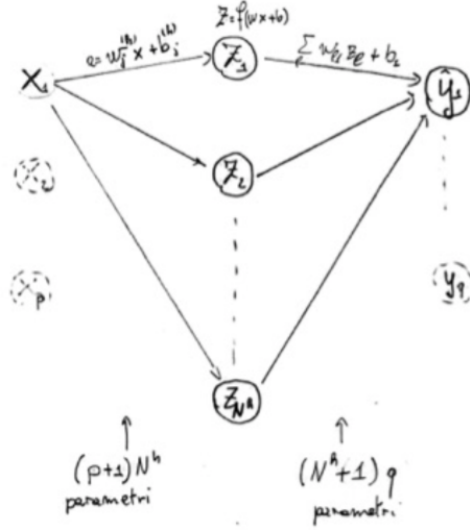


Figura 6.20: Esempio di funzionamento di una rete neurale per la soluzione di una ODE

6.7.3 Equazioni alle Derivate Parziali

Nelle equazioni alle derivate parziali dobbiamo risolvere l'equazione

$$f\left(x_1, \dots, x_p, \frac{\partial g(x_1, \dots, x_p)}{\partial x_1}, \dots, \frac{\partial g(x_1, \dots, x_p)}{\partial x_p}, \frac{\partial^2 g(x_1, \dots, x_p)}{\partial x_1 \partial x_2}, \dots, \frac{\partial^{(n)} g(x_1, \dots, x_p)}{\partial x_p^n}\right) = 0;$$

la funzione di prova sarà della forma

$$g_t(x_1, \dots, x_p) = h_1(x_1, \dots, x_p) + h_2(x_1, \dots, x_p, N(x_1, \dots, x_p, P)).$$

La funzione di costo parziale sarà

$$\tilde{C}(\underline{x}, P) = f\left(\left(\underline{x}, \frac{\partial g(\underline{x})}{\partial x_1}, \dots, \frac{\partial^2 g(\underline{x})}{\partial x_1 \partial x_2}, \dots, \frac{\partial^{(n)} g(\underline{x})}{\partial x_p^n}\right)\right)^2$$

e la funzione di costo totale sarà:

$$C(\underline{X}, P) = \sum_{i=1}^M f\left(\left(\underline{x}_i, \frac{\partial g(\underline{x}_i)}{\partial x_1}, \dots, \frac{\partial^2 g(\underline{x}_i)}{\partial x_1 \partial x_2}, \dots, \frac{\partial^{(n)} g(\underline{x}_i)}{\partial x_p^n}\right)\right)^2$$

Capitolo 7

Riduzione di Dimensionalità e Analisi della Componenti Principali (PCA)

Ci sono delle tecniche per ridurre lo spazio di parametri, queste tecniche vengono spesso utilizzate nel caso di quantità di dati molto grandi. Ci sono diverse ragioni che possono portare alla scelta di ridurre il numero originale di features $\dim(\mathbf{X}) = N \times p$.

1. Alcune volte possono essere troppe, negli studi biologici ogni campionamento può consistere in milioni di geni differenti;
2. alcuni input non sono importanti, come i bordi di un'immagine, i pixel vicini di colore simile, geni non necessari, in generale può essere che tra le features ci siano informazioni ridondanti (correlazioni tra gli input) o non rilevanti (bordi di una immagine)
3. se si vuole visualizzare i dati in 2D o 3D per una maggiore leggibilità dei dati;
4. la *Curse of Dimensionality*: le probabilità di trattare dati ed ottenere informazioni sufficientemente accurate con un numero finito di dati diventa sempre più bassa al crescere delle dimensioni, bisogna togliere dal dataset tutto ciò che non è strettamente necessario.

Si noti però che a priori non è sempre efficiente ridurre le dimensioni da studiare, ci sono casi in cui è meglio evitare.

7.1 Curse of Dimensionality

La maledizione del numero di dimensioni è dovuta al fatto che al crescere del numero di dati, la probabilità di essere vicino alla superficie cresce con il numero di dimensioni, infatti la differenza tra l'area totale e l'area lontana dal bordo cresce con la dimensionalità: per un quadrato di $L = 1$ in 2D la probabilità di essere a meno di 0.01 dal bordo è $\approx 4\%$, mentre per un ipercubo con $D = 10^4$ è $\approx 999.99999\%$, la superficie ingloba quasi tutto il volume e rimuove informazioni importanti e la densità dei

punti decresce drammaticamente. Anche con metodi MC il numero di campionamenti necessari per un buon fit aumenta esponenzialmente.

Due degli approcci possibili per risolvere il problema della dimensionalità sono:

- **Proiezione:** l'idea di questo approccio è di identificare una direzione perpendicolare ad un piano lungo cui i punti non si muovono troppo (vedi in figura 7.1 il grafico in alto a destra) e di farne una proiezione, che perde alcune informazioni, ma riduce la dimensionalità e quindi ne permette un fit migliore;
- **Ricostruzione della Varietà:** Questo approccio vuole cercare di ricostruire la superficie (generalmente iper-superficie), se esiste, su cui i dati sono veramente distribuiti (vedi in figura 7.1).

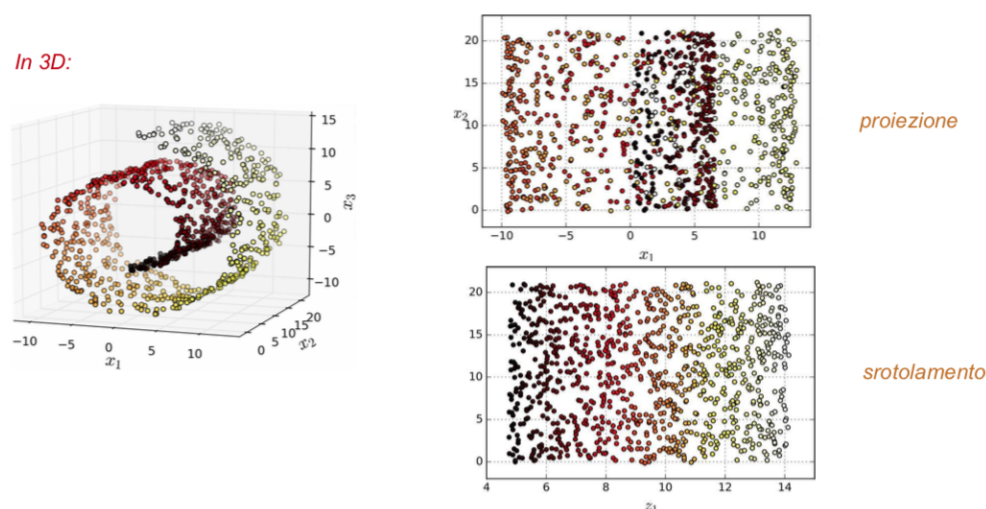


Figura 7.1: In figura è rappresentato il dataset originale 3D - costituito da una sorta di rotolo di punti. Sulla destra invece sono rappresentati due esempi di riduzione della dimensionalità il primo realizzato effettuando una semplice proiezione dei punti sul piano x_1, x_2 e il secondo invece li "srotola" su un nuovo piano dopo aver sostanzialmente ricostruito la varietà su cui erano disposti i punti originariamente

Non è sempre bene ridurre le superfici; in figura 7.2 sono mostrati due casi opposti: nel primo la riduzione di dimensionalità semplifica enormemente il problema, riducendo il tutto ad una separazione dei punti tramite una retta; nel secondo invece, il problema iniziale permetteva una separazione ovvia tra i punti (destra o sinistra del piano mostrato), mentre il problema di dimensionalità ridotta ha bisogno di fit più complessi di una regressione lineare.

7.2 Principal Components Analysis

La domanda fondamentale che sorge quindi si ha a che fare con le tecniche di riduzione della dimensionalità è come si può scegliere il piano di proiezione corretto. La tecnica si utilizza generalmente, quella della PCA, è molto legata al metodo Ridge e LASSO, poiché si cerca di identificare le direzioni corrispondenti alla maggior varianza dei dati,

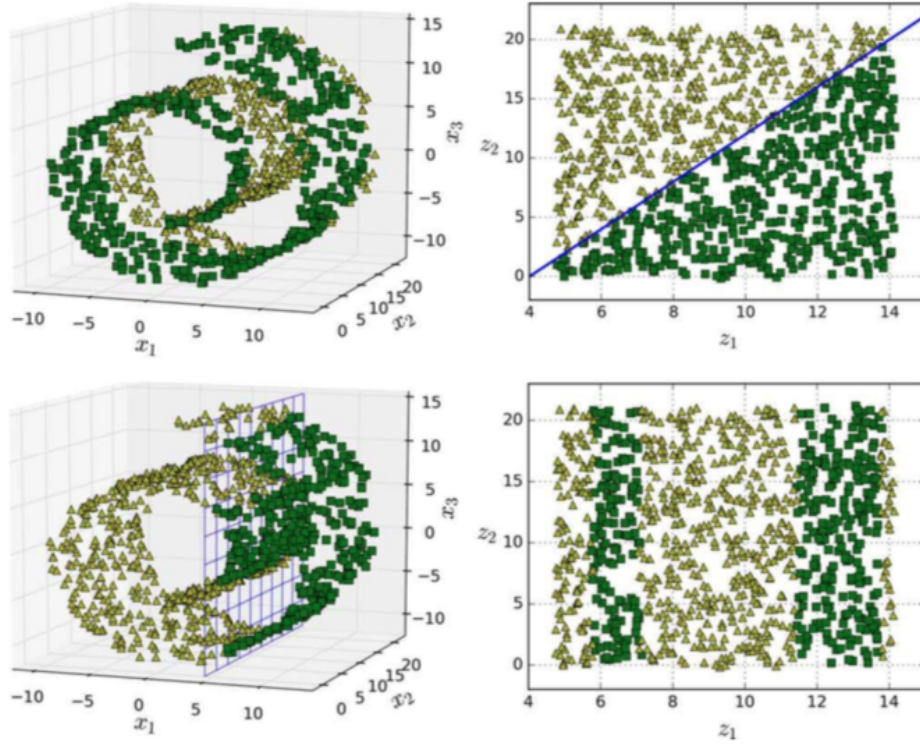


Figura 7.2: La figura rappresenta due casi in cui la riduzione di dimensionalità ha portato a due risultati opposti: in un caso rende immediata la classificazione dei dati. Nel secondo caso (sotto) invece, una classificazione che nello spazio di partenza era immediata diventa molto più complicata nello spazio ridotto

quindi dove la distribuzione è più significativa, per poi proiettare su queste direzioni (si rimuovono le direzioni poco influenti, vedi figura 7.3).

Questo approccio è analogo alla ricerca degli assi di inerzia in meccanica analitica, con $I = \sum_i m_i (x_i - R_{CM})^2$, dove però $R_{CM} = \langle x \rangle$.

Consideriamo una serie di campionamenti $\underline{x}_i$ con $i = 1, \dots, N$ ognuno con P features; la matrice di design sarà:

$$\mathbf{X} = \begin{pmatrix} \underline{x}_1^T \\ \underline{x}_2^T \\ \vdots \\ \underline{x}_N^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1P} \\ x_{21} & x_{22} & \cdots & x_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{NP} \end{pmatrix} \in \mathbb{R}^{N \times P}$$

L'algoritmo che utilizziamo per fare la PCA è di unsupervised learning, perché estraiamo dai dati le informazioni che ci interessano, non estraiamo una funzione che poi dobbiamo controllare. Se consideriamo due delle P variabili (features), ad esempio

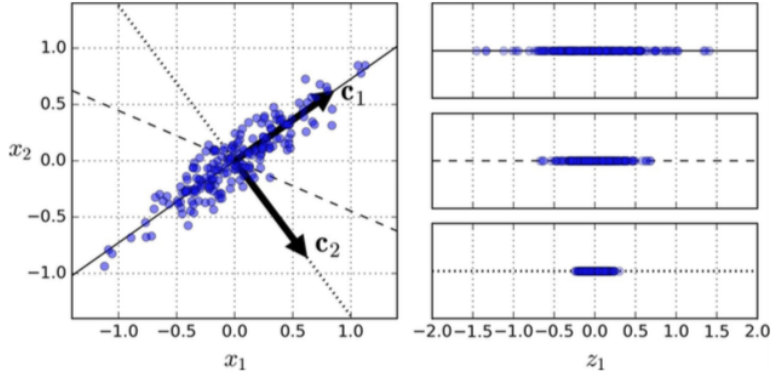


Figura 7.3: Esempio di PCA

$x = x_1$ e $y = x_j$, la serie di N campionamenti (data-points) genera i vettori:

$$\vec{x} = \begin{pmatrix} x_{11} \\ x_{21} \\ \vdots \\ x_{N1} \end{pmatrix}, \vec{y} = \begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{Nj} \end{pmatrix} \in \mathbb{R}^N$$

che rappresentano N campionamenti delle due caratteristiche. Avremo pertanto

$$\begin{aligned} \langle x \rangle &= \frac{1}{N} \sum_{i=1}^N x_i & \langle y \rangle &= \frac{1}{N} \sum_{i=1}^N y_i \\ \text{var}[x] &= \frac{1}{N-1} \sum_{i=1}^N (x_i - \langle x \rangle)^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i)^2 - \frac{N}{N-1} \langle x_i \rangle^2 \\ \text{cov}[x, y] &= \frac{1}{N-1} \sum_{i=1}^N (x_i - \langle x \rangle)(y_i - \langle y \rangle). \end{aligned}$$

A noi interessa il comportamento rispetto al baricentro dei punti, quindi $\mathbf{X} \rightarrow \mathbf{X} - \langle \mathbf{X} \rangle$, con la nuova matrice delle caratteristiche

$$\langle \mathbf{X} \rangle = \begin{pmatrix} \langle x_1 \rangle & \langle x_2 \rangle & \cdots & \langle x_P \rangle \\ \langle x_1 \rangle & \langle x_2 \rangle & \cdots & \langle x_P \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle x_1 \rangle & \langle x_2 \rangle & \cdots & \langle x_P \rangle \end{pmatrix} \in \mathbb{R}^{N \times P}$$

Da qui in avanti, poniamo per comodità di calcolo $\langle \mathbf{X} \rangle = 0$.

La covarianza ci dà informazioni sulle correlazioni tra i dati, ma la correlazione

sarà:

$$\begin{aligned} C[\mathbf{X}] &= \frac{1}{N-1} \sum_{i=1}^N (\underline{x}_i - \langle \underline{x} \rangle)(\underline{x}_i - \langle \underline{x} \rangle)^T = \mathbb{E}[(\underline{x}_i - \langle \underline{x} \rangle)(\underline{x}_i - \langle \underline{x} \rangle)^T] \\ &= \frac{1}{N-1} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{P \times P} \\ &= \begin{pmatrix} \text{Var}(x_1) & \text{cov}(x_1 x_2) & \cdots \\ \text{cov}(x_2 x_1) & \text{Var}(x_2) & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} \end{aligned}$$

se le variabili sono totalmente non-correlate allora la matrice è diagonale e ci interessiamo unicamente ai parametri con varianza maggiore. Se la matrice non è diagonale, dobbiamo diagonalizzarla e siccome sappiamo che è simmetrica ed è semidefinita a valori positivi (quindi ha tutte entrate non negative. Di conseguenza può essere diagonalizzata ed avrà autovettori con autovalori non negativi; si può quindi applicare la Singular Value Decomposition (SVD), quindi:

$$C[x] = V \Sigma^2 V^T$$

con

$$\begin{aligned} V &= (\vec{v}_1 \quad \vec{v}_2 \quad \cdots \quad \vec{v}_P) & V^T V &= I \\ \Sigma^2 &= \text{diag}(\lambda_i) & \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_P \geq 0 \end{aligned}$$

Ovvero, la matrice Σ è una matrice diagonale le cui entrate non nulle sono gli autovalori della matrice C e questi autovalori sono ordinati in ordine decrescente. Quindi, data la matrice di correlazione ed il problema agli autovalori:

$$\begin{aligned} \vec{v}_i \lambda_i &= C[x] \vec{v}_i \\ \Sigma^2 V &= C[x] V \end{aligned}$$

dove $\vec{v}_i$ sono le componenti principali della distribuzione $P(x)$, ovvero dei dati campionati $\mathbf{X}$, e λ_i sono le varianze della distribuzione lungo ogni componente $\vec{v}_i$, i vettori $\{\vec{v}_i\}$ formano una base ortogonale in $\mathbb{R}^P$.

7.2.1 Teorema PCA

Teorema Se $\underline{x}$ è una distribuzione multivariata di P variabili a media nulla ($\underline{x} \in \mathbb{R}^P$, $\langle \underline{x} \rangle = 0$) e $p < P$, allora il sottospazio $S \in \mathbb{R}^p$ di dimensione p che massimizza la varianza della distribuzione è dato dai primi p autovettori di $C[x]$ corrispondenti ai p autovalori più grandi.

Metodo Normalizziamo la matrice di design ($\tilde{\mathbf{X}} \rightarrow \mathbf{X} - \langle \mathbf{X} \rangle$), diagonalizziamo la matrice di covarianza, proiettiamo sui primi p autovettori e costruiamo la base $\tilde{V} = \vec{v}_1, \vec{v}_2, \dots, \vec{v}_p) \in \mathbb{R}^{P \times p}$ ed otteniamo

$$\tilde{\mathbf{X}} \tilde{V} \in \mathbb{R}^{N \times p}$$

Dimostrazione Data

$$\mathbf{X} = \begin{pmatrix} \underline{x}_1^T \\ \underline{x}_2^T \\ \vdots \\ \underline{x}_N^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1P} \\ x_{21} & x_{22} & \cdots & x_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{NP} \end{pmatrix} \in \mathbb{R}^{N \times P}$$

vogliamo approssimare ogni campionamento $\underline{x}_i \in \mathbb{R}^P$ con un nuovo vettore $\hat{x}_i \in \mathbb{R}^P$ che proietti $\underline{x}_i$ in uno spazio a p dimensioni. Ovvero, ogni $\hat{x}_i$ deve essere esprimibile come al più p vettori $\underline{u}_l$ ortotnormali:

$$\hat{x}_i = \sum_{l=1}^p z_{il} \underline{u}_l, \quad \underline{u}_l \underline{u}_{l'}^T = \delta_{ll'}. \quad (7.1)$$

Per ottenere i vettori $\underline{u}_l \in \mathbb{R}^P (l = 1, \dots, p)$ ed i coefficienti z_{il} con $i = 1, \dots, N$ minimizziamo una funzione di costo J , sotto la condizione che $\{\underline{u}_l\}$ sia un set di vettori ortonormali:

$$J(\{z_{il}\}, \{\underline{u}_l\}) = \frac{1}{N} \sum_{i=1}^N \|\underline{x}_i - \hat{x}_i\|^2 - \sum_{l,j=1}^p \lambda_{l,j} (\underline{u}_l^T \underline{u}_j - \delta_{l,j}) \quad (7.2)$$

con $\lambda_{l,j}$ opportuni moltiplicatori di Lagrange. A questo punto cerco i valori dei parametri che minimizzano la funzione di costo, quindi impongo

$$\frac{\partial J}{\partial \lambda_{l,j}} = 0 \Rightarrow \underline{u}_l^T \underline{u}_j = \delta_{l,j} \forall l, j = 1, \dots, p \quad (7.3)$$

Ovvero, da questa prima condizione si ottiene che i vettori $\underline{u}_l^T$ e $\underline{u}_j$ sono ortonormali. Impongo poi

$$\frac{\partial J}{\partial z_{i,l}} = \quad (7.4)$$

$$\frac{\partial}{\partial z_{i,l}} \left\{ \frac{1}{N} \sum_i \left(\underline{x}_i - \sum_l z_{i,l} \underline{u}_l \right)^2 - \sum_{l',j'} \lambda_{l',j'} (\underline{u}_{l'}^T \underline{u}_{j'} - \delta_{l',j'}) \right\} = \quad (7.5)$$

$$\frac{2}{N} \sum_{i'} -\underline{u}_l^T \delta_{i,i'} \left(\underline{x}_{i'} - \sum_{l'} z_{i',l'} \underline{u}_{l'} \right) = 0 \quad (7.6)$$

$$(7.7)$$

Da cui segue

$$\frac{-2}{N} \underline{u}_l^T (\underline{x}_i - z_{il} \underline{u}_l) = 0 \quad (7.8)$$

$$\Rightarrow z_{il} = \underline{u}_l^T \underline{x}_i \quad (7.9)$$

ovvero, la componente di $\hat{x}_i$ lungo $\underline{u}_l$ non è nient'altro che la proiezione di $\underline{x}_i$ su $\underline{u}_l$.

Inserendo poi $z_{i,l}$ all'interno della funzione di costo si ottiene

$$\frac{1}{N} \sum_i \left[\underline{x}_i - \sum_l \underline{u}_l (\underline{u}_l^T \underline{x}_i) \right]^2 = \quad (7.10)$$

$$\frac{1}{N} \sum_i \left[\underline{x}_i^T \underline{x}_i - \sum_l (z_{i,l})^2 \right] = \quad (7.11)$$

$$\frac{1}{N} \sum_i \left[\underline{x}_i^T \underline{x}_i - \sum_l \underline{u}_l^T \underline{x}_i \underline{x}_i^T \underline{u}_l \right] \quad (7.12)$$

$$\frac{1}{N} \sum_i \|\underline{x}_i\|^2 - \sum_l \underline{u}_l^T \left(\frac{1}{N} \sum_i \underline{x}_i \underline{x}_i^T \right) \underline{u}_l \quad (7.13)$$

L'ultimo termine contiene la matrice di covarianza:

$$\frac{1}{N} \sum_i (\underline{x}_i \underline{x}_i^T) = \mathbb{E}[\underline{x} \underline{x}^T] = \frac{1}{N} X^T X = C[X] \quad (7.14)$$

Quindi la funzione di costo diventa:

$$J(\{\underline{u}_l\}, \{\lambda_{lj}\}) = \mathbb{E}[\|\underline{x}\|^2] - \sum_l \underline{u}_l^T C[X] \underline{u}_l - \sum_{l,j} \lambda_{l,j} (\underline{u}_l^T \underline{u}_j \delta_{lj}) \quad (7.15)$$

Le direzioni delle componenti $\underline{u}_l$ ed i moltiplicatori si ottengono minimizzando rispetto a $\underline{u}_l$

$$\frac{\partial J}{\partial \underline{u}_l} = 0 + (C[x] + C[X]^T) \underline{u}_l - \sum_j (\lambda_{lj} + \lambda_{jl}) \underline{u}_j = 0 \quad (7.16)$$

$$\Rightarrow C[x] \underline{u}_l = \sum_{j=1}^p \lambda_{lj} \underline{u}_j \Rightarrow \underline{u}_l^T C[x] \underline{u}_l = \sum_j \lambda_{lj} \delta_{lj} = \lambda_{ll} \quad (7.17)$$

$$(7.18)$$

Quindi utilizzando $\underline{u}_l^T \underline{u}_j = \delta_{lj}$, si ottiene

$$J(\underline{u}_l, \lambda_{lj}) = \mathbb{E}[\|\underline{x}\|^2] - \sum_{l=1}^p \lambda_{ll} \quad (7.19)$$

Quindi, per minimizzare J:

$$J(\{\underline{u}_l\}, \{\lambda_{lj}\}) = \mathbb{E}[\|\underline{x}\|^2] - \sum_{l=1}^p \lambda_{ll}; \quad C[x] \underline{u}_l = \sum_{j=1}^p \lambda_{lj} \underline{u}_j \quad (7.20)$$

A questo punto

1. se $p = 1$: $C \underline{u}_1 = \sigma_1^2 \underline{u}_1$ quindi si minimizza J scegliendo l'autovalore λ_1 più grande ($\lambda_{11} = \sigma_1^2$)
2. se $p = 2$: $\underline{u}_1 C \underline{u}_2 = \underline{u}_1^T (\sum_{j=1}^2 \lambda_{2j} \underline{u}_j) \longleftrightarrow \lambda_{11} \underline{u}_1^T \underline{u}_2 = 0 = \lambda_{21} \underline{u}_1^T \underline{u}_1 + \lambda_{22} \underline{u}_1^T \underline{u}_2 \Rightarrow \lambda_{21} = \lambda_{12} = 0$ allora $C \underline{u}_2 = \lambda_{22} \underline{u}_2$ allora minimizzo J scegliendo il secondo autovalore più grande.
3. se $p = 3$: $\lambda_{13} = \lambda_{23} = 0$ e $\lambda_{33} = (\sigma_3)^2$; $C \underline{u}_3 = \sigma_3^2 \underline{u}_3$

e così via.

PCA e SVD C'è una relazione con la decomposizione ai valori singoli, perché la matrice di design può essere scritta come (*Singular Value Decomposition*)

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

dove $\mathbf{X} \in \mathbb{R}^{N \times P}$, $\mathbf{U} \in \mathbb{R}^{N \times P}$, $\mathbf{V} \in \mathbb{R}^{P \times P}$ e $\mathbf{\Sigma} \in \mathbb{R}^{P \times P}$ diagonale, ma sappiamo che

$$C[\mathbf{X}] = \mathbf{X}^T \mathbf{X} = \mathbf{V} \mathbf{\Sigma} \mathbf{U}^T \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T = \mathbf{V} \mathbf{\Sigma}^2 \mathbf{V}^T$$

Le componenti principali sono le prime p colonne di $\mathbf{V}$ e la nuova matrice di design è:

$$\mathbf{Z}^{\text{PCA}} = \mathbf{X} \tilde{\mathbf{V}}$$

con $\langle \mathbf{X} \rangle = 0$; per ricostruire la matrice $N \times P$ dopo la proiezione scriviamo:

$$\hat{\mathbf{X}} = \mathbf{Z}^{\text{PCA}} \tilde{\mathbf{V}}^T + \langle \mathbf{X} \rangle.$$

Esempio dataset MNIST

Il dataset originale ha migliaia di immagini, con 784 pixel (28^2), ma se prendiamo anche solo i primi $p = 150$ valori, manteniamo il 95% della varianza dei dati (vedi figure)

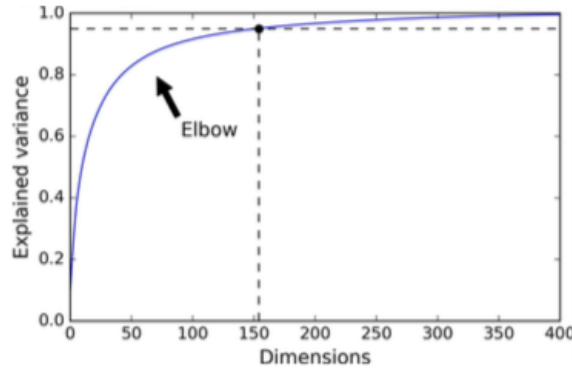


Figura 7.4: Rappresentazione della varianza in funzione del numero di pixels

Derivazione del Teorema PCA

Nel presente paragrafo si presenta nuovamente la dimostrazione del teorema di PCA, in questo caso però utilizzando una notazione leggermente diversa. Al lettore la scelta di quale delle due notazioni utilizzare.

Immaginiamo di avere N dati con P componenti

$$\mathbf{X} = \begin{pmatrix} \underline{x}_1^T \\ \underline{x}_2^T \\ \vdots \\ \underline{x}_N^T \end{pmatrix}$$

vogliamo trovare una nuova serie di vettori $\hat{\underline{x}}_i \in \mathbb{R}^P$ che proiettino in uno spazio a p dimensioni i campionamenti $\underline{x}_i \in \mathbb{R}^P$. Per ogni campionamento, scriviamo il nuovo

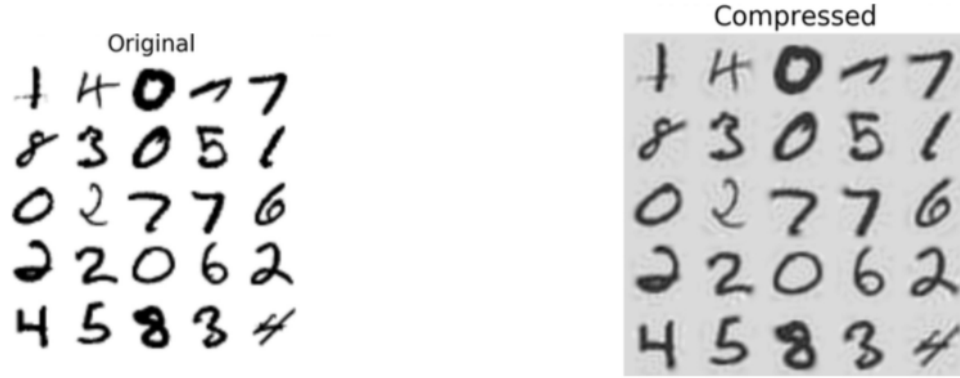


Figura 7.5: Rappresentazione di alcuni campioni del dataset MNIST prima e a seguito della compressione, quindi dell'eliminazione di alcuni pixel che non contribuivano alla varianza in maniera significativa

vettore dei campionamenti (sempre di P dimensioni), ma che si esprime con al più p vettori ortonormali $\underline{u}_\ell$:

$$\hat{\underline{x}}_i = \sum_{\ell=1}^P z_{i\ell} \underline{u}_\ell, \quad \forall i$$

con $\underline{u}_\ell^T \underline{u}_{\ell'} = \delta_{\ell\ell'}$. Per ottenere i vettori $\underline{u}_\ell$ con $\ell = 1, \dots, p$ ed i coefficienti $z_{i\ell}$ minimizziamo una funzione di costo con la condizione che i vettori $\{\underline{u}_\ell\}$ siano ortonormali:

$$J(\{z_{i\ell}\}, \{\underline{u}_\ell\}) = \frac{1}{N} \sum_{i=1}^N \|\underline{x}_i - \hat{\underline{x}}_i\|^2 - \sum_{\ell,j=1}^P \lambda_{\ell,j} (\underline{u}_\ell^T \underline{u}_j - \delta_{\ell,j})$$

con $\lambda_{\ell,j}$ opportuni moltiplicatori di Lagrange. Se si minimizza questa funzione di costo, si massimizza la varianza dei dati e la soluzione corrispondente è data dai p autovettori corrispondenti agli autovalori maggiori. Risolviamo imponendo le derivate nulle:

$$\begin{aligned} \frac{\partial J}{\partial \lambda_{\ell,j}} = 0 & \Rightarrow \underline{u}_\ell^T \underline{u}_j = \delta_{\ell,j} \text{ (ortonormalità) } \forall \ell, j = 1, 2, \dots, P \\ \frac{\partial J}{\partial z_{i\ell}} = \frac{\partial}{\partial} \left\{ \frac{1}{N} \sum_{i'} (\underline{x}_{i'} - \sum_{\ell'} z_{i'\ell'} \underline{u}_{\ell'})^2 - \sum_{i', \ell'} \lambda_{\ell', j'} \dots \right\} \\ &= \frac{2}{N} \sum_{i'} -\underline{u}_\ell^T \delta_{ii'} (\underline{x}_{i'} - \sum_{\ell'} z_{i'\ell'} \underline{u}_{\ell'}) = 0 \\ &- \frac{2}{N} \underline{u}_\ell^T (\underline{x}_i - z_{i\ell} \underline{u}_\ell) = 0 \Rightarrow z_{i\ell} = \underline{u}_\ell^T \underline{x}_i \end{aligned}$$

e quindi la componente di $\hat{\underline{x}}_i$ lungo $\underline{u}_\ell$ non è nient'altro che la proiezione di $\underline{x}_i$ su $\underline{u}_\ell$.

Inserendo il valore di $z_{i\ell}$ nella funzione di costo (assumendo di ricercare solo p

autovettori) otteniamo:

$$\begin{aligned}
\frac{1}{N} \sum_i [\underline{x}_i - \sum_{\ell}^p \underline{u}_{\ell} (\underline{u}_{\ell}^T \underline{x}_i)]^2 &= \frac{1}{N} \sum_i [\underline{x}_i^T \underline{x}_i + \sum_{\ell} (z_{i\ell})^2 \underline{u}_{\ell}^T \underline{u}_{\ell} - 2 \sum_{\ell} \underline{x}_i^T \underline{u}_{\ell} (z_{i\ell})] \\
&= \frac{1}{N} \sum_i [\underline{x}_i^T \underline{x}_i - \sum_{\ell} (z_{i\ell})^2] = \frac{1}{N} \sum_i [\underline{x}_i^T \underline{x}_i - \sum_{\ell} \underline{u}_{\ell}^T \underline{x}_i \underline{x}_i^T \underline{u}_{\ell}] \\
&= \frac{1}{N} \sum_i \|\underline{x}_i\|^2 - \sum_{\ell} \underline{u}_{\ell}^T \left(\frac{1}{N} \sum_i \underline{x}_i \underline{x}_i^T \right) \underline{u}_{\ell};
\end{aligned}$$

l'ultimo termine si può riscrivere tramite la matrice di covarianza:

$$\frac{1}{N} \sum_i (\underline{x}_i, \underline{x}_i^T) = \mathbb{E}[\underline{x} \underline{x}^T] = \frac{1}{N} \mathbf{X}^T \mathbf{X} = C[\mathbf{X}].$$

La funzione di costo diventa¹, quindi:

$$J(\{\underline{u}_{\ell}\}, \{\lambda_{\ell,j}\}) = \mathbb{E}[\|\underline{x}\|^2] - \sum_{\ell'=1}^p \underline{u}_{\ell'}^T C[\mathbf{X}] \underline{u}_{\ell'} - \sum_{\ell',j} \lambda_{\ell',j} (\underline{u}_{\ell'}^T \underline{u}_{\ell} - \delta_{\ell',j}) \quad (7.21)$$

dove il primo termine è una costante e dipende solo dai dati, e la minimizzazione degli ultimi termini è:

$$\frac{\partial J}{\partial \underline{u}_{\ell}} = 0 + (C[\mathbf{X}] + C[\mathbf{X}]^T) \underline{u}_{\ell} - \sum_j (\lambda_{\ell,j} + \lambda_{j,\ell}) \underline{u}_j = 0$$

che comporta effettivamente degli autovalori $\lambda_{\ell\ell}$:

$$C[\mathbf{X}] \underline{u}_{\ell} = \sum_{j=1}^p \lambda_{\ell,j} \underline{u}_j \quad \Rightarrow \quad \underline{u}_{\ell}^T C[\mathbf{X}] \underline{u}_{\ell} = \sum_j \lambda_{\ell,j} \delta_{\ell,j} = \lambda_{\ell\ell}$$

e vediamo quindi che le $\lambda_{\ell\ell}$ non sono nient'altro che la varianza della distribuzione dei punti rispetto alla proiezione dei valori di $\underline{x}$ su $\underline{u}_{\ell}$.

Sfruttando il fatto che $\underline{u}_{\ell}^T \underline{u}_j = \delta_{\ell,j}$, la funzione di costo diventa:

$$J(\{\underline{u}_{\ell}\}, \{\lambda_{\ell,j}\}) = \mathbb{E}[\|\underline{x}\|^2] - \sum_{\ell=1}^p \lambda_{\ell\ell} \quad (7.22)$$

abbiamo tante direzioni possibili, ma desideriamo prendere le p direzioni che massimizzano la varianza.

Ricapitolando: abbiamo una serie di vettori $\underline{x}_i \in \mathbb{R}^P$ che vogliamo riportare in un'altra serie di vettori $\hat{\underline{x}}_i = \sum_{\ell=1}^p z_{i\ell} \underline{u}_{\ell} \in \mathbb{R}^P$, che sono ancora definiti in uno spazio P dimensionale, ma tramite unicamente p vettori $\underline{u}_{\ell}$ con elementi

$$z_i = \begin{pmatrix} z_{i1} \\ z_{i2} \\ \vdots \\ z_{ip} \end{pmatrix} \in \mathbb{R}^p$$

¹Ricordiamo che la sommatoria su ℓ', j è data da $p(p-1)/2$ elementi, perché è una matrice simmetrica.

che costruisce la nuova matrice di design da fittare:

$$\mathbf{Z} = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_p \end{pmatrix} \in \mathbb{R}^{N \times p};$$

il processo porta quindi ad un nuovo set di dati:

$$(\vec{y}, \mathbf{X}) \rightarrow (\vec{y}, \mathbf{Z}).$$

Per costruzione abbiamo (ricordando che la media degli $\underline{x}_i = 0$):

$$\begin{aligned} \langle Z \rangle &= \langle u^T \underline{x}_i \rangle = 0 \\ C[Z]_{\ell\ell'} &= \sum_i (z_{i\ell} - \langle z_\ell \rangle)(z_{i\ell'} - \langle z_{\ell'} \rangle) = \frac{1}{N} \sum_i z_{i\ell} z_{i\ell'} \\ u_\ell^T C[X] u_{\ell'} &= C[Z]_{\ell\ell'} \end{aligned}$$

Per minimizzare J , dividiamo nei due casi: $p = 1$ e $p \neq 1$. Nel primo caso $p = 1$, otteniamo:

$$C \underline{u}_1 = \sigma_1^2 \underline{u}_1$$

e per minimizzare la funzione di costo, dobbiamo prendere l'autovettore con autovalore maggiore λ_1 ($\lambda_{\ell\ell|_{\ell=1}} = \sigma_1^2$).

Se invece vogliamo aggiungere un nuovo vettore ($p = 2$), otteniamo:

$$\begin{aligned} \underline{u}_1^T C \underline{u}_2 &= \underline{u}_1^T \left(\sum_{j=1}^2 \lambda_{2j} \underline{u}_j \right) \\ \Rightarrow \lambda_1 \underline{u}_1^T \underline{u}_2 &= 0 = \lambda_{21} \underline{u}_1^T \underline{u}_1 + \lambda_{22} \underline{u}_1^T \underline{u}_2 \\ \Rightarrow \lambda_{21} &= \lambda_{12} = 0 \\ \Rightarrow C \underline{u}_2 &= \lambda_{22} \underline{u}_2 \end{aligned}$$

che è un'altra equazione agli autovalori, per cui per minimizzare J usiamo il secondo autovalore più grande (perché il primo è già stato preso). Per p successivi il processo è sempre lo stesso: la funzione è minimizzata per gli autovalori maggiori.

Se vogliamo essere pignoli, possiamo chiederci, se $p > 1$, come mai la soluzione non possa essere una combinazione di autovettori. Possiamo risolvere il problema in un modo alternativo:

$$J = \mathbb{E}[||\underline{x}||^2] - \sum_{\ell} \underline{u}_{\ell}^T C[x] \underline{u}_{\ell}.$$

Immaginiamo di diagonalizzare C , e di ottenere:

$$\begin{aligned} C[x] \vec{v}_j &= \sigma_j^2 \vec{v}_j \quad \forall j \\ \sigma_1^2 &\geq \sigma_2^2 \geq \dots \geq \sigma_P^2 \geq 0 \end{aligned}$$

sapendo di avere una base completa ortonormale di dimensioni P ($\vec{v}_j$), vogliamo cercare una base ortonormale di dimensione $p < P$ ($\underline{u}_{\ell}$). Di sicuro possiamo espandere:

$$\underline{u}_{\ell} = \sum_j s_{\ell j} \vec{v}_j \quad \sum_{j=1}^P |s_{\ell j}|^2 = 1 \quad \forall \ell \quad a_j \equiv \sum_{\ell=1}^p |s_{\ell j}|^2 \leq 1$$

ed avremo che $\sum_j a_j = p$. Sostituendo otteniamo:

$$\begin{aligned}\sum_{\ell} \underline{u}_{\ell}^T C \underline{u}_{\ell} &= \sum_{\ell} \sum_{jj'} \vec{v}_j s_{\ell j} C s_{\ell j'} \vec{v}_{j'} \\ &= \sum_{j\ell} (s_{\ell j})^2 \sigma_j^2 \\ &= \sum_j a_j \sigma_j^2\end{aligned}$$

che è una media pesata.

L'unico modo per massimizzare questo termine, dobbiamo scegliere (sapendo che la loro somma è pari a p e che $\sigma_j^2 \geq \sigma_{j+1}^2$):

$$a_j = \begin{cases} 1 & j = 1, \dots, p \\ 0 & j = p+1, \dots, P \end{cases}$$

Quindi abbiamo che lo spazio coperto dagli $\underline{u}_{\ell}$ è equivalente a quello coperto da $\{\vec{v}_j; j = 1, \dots, p\}$.

Capitolo 8

Alberi Decisionali e Random Forest

In primo luogo, discutiamo l'idea fondamentale di questi metodi. Immaginiamo di avere una serie di variabili x_{ij} con $j = 1, \dots, p$ classificati in diversi modi, come in figura ????. Nella figura riportata, per classificare i diversi colori, possiamo chiederci se $x_2 > s_1$ e se $x_1 > s_2$, che mi identificherà la classificazione "verde". Si sviluppa quindi una struttura ad albero, dove ogni domanda è un *nodo* e ogni classificazione è una *foglia*. La predizione sarà:

$$\hat{y} = f(\underline{x}_i) = \sum_m c_m I\{\underline{x}_i \in B_m\}$$

dove $\underline{x}_i \in \mathbb{R}^P = \bigcup_{m=1}^M B_m$.

Quando si ha una serie di misure e si vuole classificare il parametro di uscita, può essere utile utilizzare un albero decisionale, come nel caso delle mail SPAM, di cui è riportato l'albero decisionale in figura 8.1.

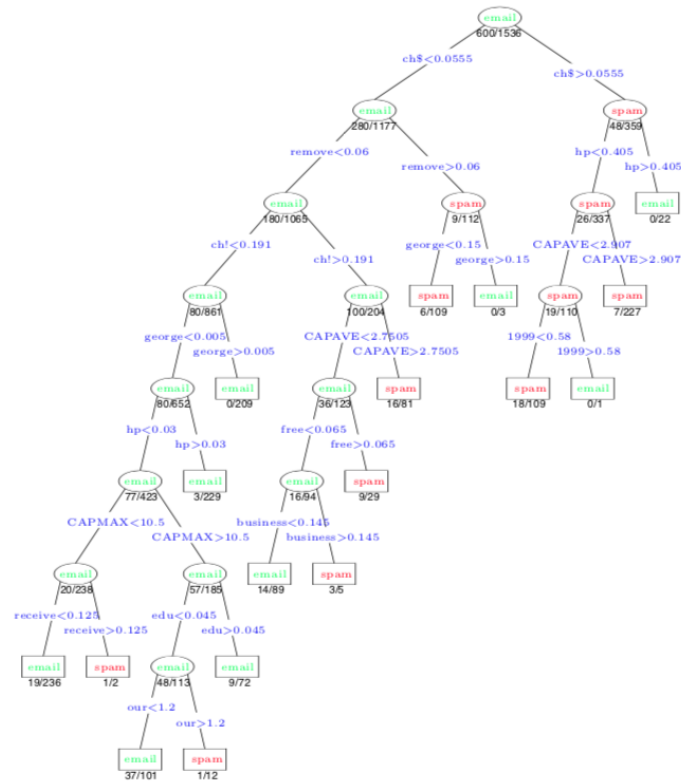
Quando si arriva alla fine dell'albero, avremo una serie di foglie che contiene un'area dello spazio delle caratteristiche. Può anche essere possibile che ci siano dei casi di *mis-classification*, per cui ci possono essere errori. Quando si ha una separazione di questo tipo si separa lo spazio degli input e lo si divide in sottoinsiemi a cui associamo un determinato output $\hat{y}$. Se si infittisce troppo l'albero si rischia un overfitting, che riduce al minimo l'errore di training, ma che provoca un aumento dell'errore di test.

8.1 Algoritmo CART

Uno degli algoritmi più utilizzati nella costruzione di decision-trees è l'algoritmo CART (*Classification And Regression Trees*) che solitamente viene utilizzato per ottimizzare gli alberi decisionali. Come tutti gli algoritmi basati su alberi decisionali funziona iterativamente per partizioni successive dello spazio delle caratteristiche. Assumiamo di avere un dataset

$$\mathcal{D} = \{(y_i, \underline{x}_i) : i = 1, \dots, N\}$$

e di suddividere lo spazio delle caratteristiche in M ($\in \mathcal{N}$) regioni che costituiscono una partizione dello spazio di partenza. Allora, utilizzando l'algoritmo CART, la



The pruned tree for the **spam** example. The split variables are shown in blue on the branches, and the classification is shown in every node. The numbers under the terminal nodes indicate misclassification rates on the test data.

Figura 8.1

predizione per un certo punto $\underline{x}$ dipende unicamente dall'area della regione in cui si trova:

$$\hat{y} := f(\underline{x}) = \sum_{m=1}^M c_m I(\underline{x} \in R_m), \quad I = \begin{cases} 1 & x \in R_m \\ 0 & \text{altrove} \end{cases}$$

dove c_m è una costante che dipende alla regione in cui si trova, R_m indica l' m -esima regione e $m = 1, \dots, M$.

Nel caso di una regressione ad esempio, si utilizza il metodo dei minimi quadrati, quindi si minimizza la funzione di costo:

$$C = \sum_i [y_i - f(\underline{x}_i)]^2$$

Dove, ancora una volta è stato fissato il numero di regioni finali M (anche chiamato numero di foglie), R_m è la regione di appartenenza del data point e c_m è il valore assegnato alla regione m -esima. Una volta determinate le regioni R_m , l'ascelta del parametro c_m è molto semplice, poiché è dato dalla media dei campionamenti, infatti:

$$\frac{\partial C}{\partial c_m} = \frac{\partial}{\partial c_m} \sum_i (y_i - \sum_{m'} c_{m'} I(\underline{x}_i, m'))^2 \quad (8.1)$$

$$= -2 \sum_i [y_i - f(\underline{x}_i)] I(\underline{x}_i, R_m) = 0 \quad (8.2)$$

$$\Rightarrow \sum_{\underline{x}_i \in R_m} |y_i - \hat{c}_m| = 0 \quad (8.3)$$

$$\Rightarrow \hat{c}_m = \frac{\sum_{i \in k_m} y_i}{\sum_{i \in k_m} 1} = \langle y_i \rangle_{\underline{x}_i \in R_m}. \quad (8.4)$$

È complesso invece scegliere i valori di R , ovvero determinare la suddivisione in regioni. La strategia migliore è quella scegliere una variabile alla volta X_j con $j = 1, \dots, p$ e trovare il valore di spartizione s (o di soglia) tale che se $X_j \leq s$ allora questo sta in R_1 , altrimenti in R_2 (assumendo in questo caso di suddividere lo spazio di partenza in due sotto-spazi $R_1 = \{X|X_j \leq s\}$ e $R_2 = \{X|X_j > s\}$).

Se si sta affrontando un problema di regressione, allora la minimizzazione (assumendo come funzione costo MSE) sarà:

$$J(j, s) = \sum_{\underline{x}_i \in R_1(j, s)} (y_i - \hat{c}_1)^2 + \sum_{\underline{x}_i \in R_2(j, s)} (y_i - \hat{c}_2)^2 \quad (8.5)$$

con soluzione:

$$\begin{aligned} \hat{c}_1 &= \langle y_i \rangle_{R_1} \\ \hat{c}_2 &= \langle y_i \rangle_{R_2} \end{aligned}$$

andando poi a cambiare i valori di j ed s , proviamo vari punti di separazione ed i rispettivi valori della funzione di costo; sceglieremo poi la coppia (j, s) che minimizza J .

Nel caso invece di problemi di classificazione, la funzione di costo ottimale è:

$$J(j, s) = \frac{N_1^{(n)}}{N^{(n)}} Q_1 + \frac{N_2^{(n)}}{N^{(n)}} Q_2 \quad (8.6)$$

dove

$$N_\ell^{(n)} = \sum_{\underline{x}_i \in R_\ell} 1 \quad N^{(n)} = N_1^{(n)} + N_2^{(n)} \quad (8.7)$$

sono le densità di punti in ogni regione e dove Q_i è un errore di classificazione della regione i , detto anche *impurità*.

8.1.1 Problemi di regressione

In realtà anche per problemi di regressione si può ottenere per la funzione costo una forma simile a quella della funzione di costo dei problemi di classificazione. Infatti quello che si fa nel caso generale è minimizzare la funzione costo 8.6 con come impurità

$$Q_\ell^{(n)} = \frac{1}{N_\ell^{(n)}} \sum_{\underline{x}_i \in R_\ell^{(n)}} (y_i - c_\ell^{(n)})^2. \quad (8.8)$$

Nel limite in cui il numero di regioni M è pari al numero N di punti nel dataset ($M \rightarrow N^{\text{total}}$) l'albero decisionale si riporta sostanzialmente ad un k-nearest neighbour (kNN), con alcune differenze (nel kNN i confini delle superfici sono definite calcolando le distanze tra i punti e dimezzandole, mentre qui i bordi delle regioni sono creati tramite iper-piani paralleli allo spazio delle variabili di ingresso): il risultato è comunque un overfitting. Per evitare overfitting, si possono imporre limiti al numero di punti per nodo, oppure si può utilizzare la tecnica del *pruning*, una sorta di potatura dell'albero che si è costruito.

Come mostrato finora l'implementazione dell'algoritmo CART è estremamente semplice, tuttavia ha, come ogni algoritmo che utilizzi alberi decisionali, un costo computazionale che cresce esponenzialmente con il numero di nodi: $O(e^M)$ dove M è il numero di nodi. Tuttavia il costo di implementazione è estremamente basso:

$$\underline{x} \rightarrow f(\underline{x}) \simeq O(\log_2(M)) \quad (8.9)$$

Dove si intende dire che, essendo i decision tree generalmente sufficientemente bilanciati, per ottenere una predizione per una data-point bisogna attraversare un numero $O(\log_2(M))$ di foglie.

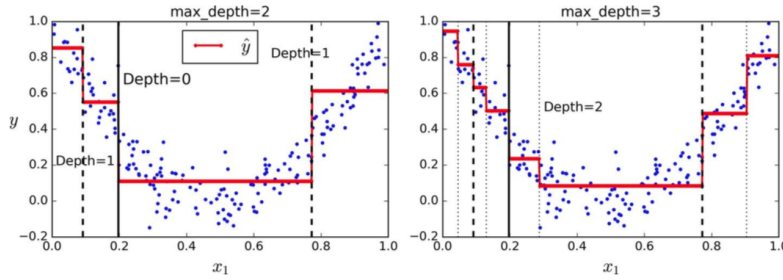


Figura 8.2: Esempio di risultato ottenuto con l'algoritmo CART utilizzando diverse profondità per l'albero decisionale.

In figura 8.2 è riportato l'esempio di un albero decisionale nel caso di un problema di regressione, dove il livello zero è $x < 0.2$, il primo è $x < 0.1$ e $x < 0.75$, mentre il secondo livello di profondità è $y < 0.83$, $y < 0.57$, $y < 0.1$, $y < 0.6$. Aggiungendo più livelli di profondità, le separazioni aumentano. Se non ci sono restrizioni, l'algoritmo va avanti fino a quando non fitta ogni punto (overfitting), come in figura 8.3 a destra. Per limitare questo tipo di errori e non sapendo a priori la profondità ottimale, lasciamo che la rete scelga la miglior profondità, ma indicando un numero massimo di foglie (in figura 8.3 a sinistra il numero di foglie è limitato dalla condizione sul numero minimo di data-points per foglia). Le linee dell'algoritmo sono scelte in base al valore medio in ogni regione.

8.1.2 Classificazione

Per i problemi di classificazione, la struttura dell'algoritmo è la medesima, ma l'errore di costo dipende dalle funzioni di impurità, che vengono scelte sulla base del problema che si vuole risolvere (di conseguenza cambiano leggermente i criteri sulla base dei quali si fa lo splitting dei nodi e il pruning. Al posto di prendere il valore medio di ogni regione, nella classificazione si possono calcolare le percentuali delle varie classi in ogni

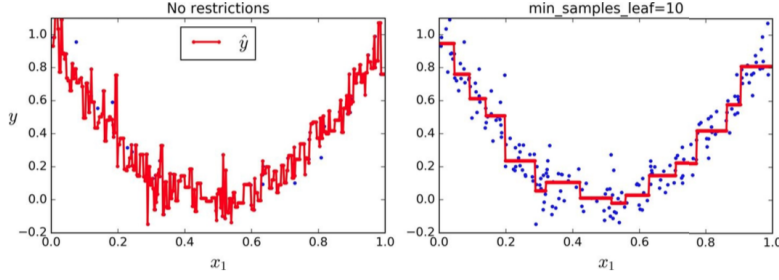


Figura 8.3: Esempio di implementazione dell'algoritmo CART senza restrizioni sul numero di nodi (sinistra), in questo caso l'algoritmo va in overfitting. Esempio di implementazione dell'algoritmo CART imponendo un numero minimo (10) di elementi per ciascuna foglia.

regione, ed associare la classe con probabilità maggiore, che è quella che minimizza l'errore.

Assumiamo di voler arrivare ad una classificazione in K classi, quindi $y = 1, 2, \dots, k$. In un nodo m con regione R_m , avremo:

$$N_m = \sum_{x_i \in R_m} 1$$

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k)$$

dove N_m è il numero di osservazioni in R_m e $\hat{p}_{mk}$ è la proporzione di data-points appartenenti alla classe k in m . In sostanza la probabilità che un punto nella regione m appartenga alla classe k non è altro che il numero di punti nella regione diviso il numero di punti totali.

Quindi classifichiamo un data-point che si trova in m all'interno della regione $k(m) = \arg \max_k \hat{p}_{mk}$ ovvero la classe con il rapporto maggiore nel nodo m . In questo contesto si possono utilizzare diverse formule per il calcolo dell'impurità:

- errore di errata classificazione, per cui si sommano i punti che non appartengono alla classe scelta:

$$Q_m = \frac{1}{N_m} \sum_{i \in R_m} I(y_i \neq k(m)) = 1 - p_{mk(m)}; \quad (8.10)$$

- l'indice di Gini (nullo per classificazioni pure, mentre è massimo quando le classi sono equiprobabili), che misura l'impurità delle varie classi nella stessa regione m calcolando la somma su k e k' dei prodotti delle probabilità:

$$G_m = \sum_{k \neq k'} \hat{p}_{mk} \hat{p}_{mk'} = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^K (\hat{p}_{mk})^2; \quad (8.11)$$

- l'entropia o devianza che somma le entropie di ogni classe:

$$Q_m = - \sum_{k=1}^K \hat{p}_{mk} \log_2 \hat{p}_{mk}. \quad (8.12)$$

I metodi utilizzati più frequentemente sono l'indice di Gini (scelta di default) e la devianza, ma solitamente l'entropia è la scelta migliore. Nel caso però in cui i risultati siano simili, conviene utilizzare l'indice di Gini, poiché molto più veloce da calcolare.

8.1.3 Pruning

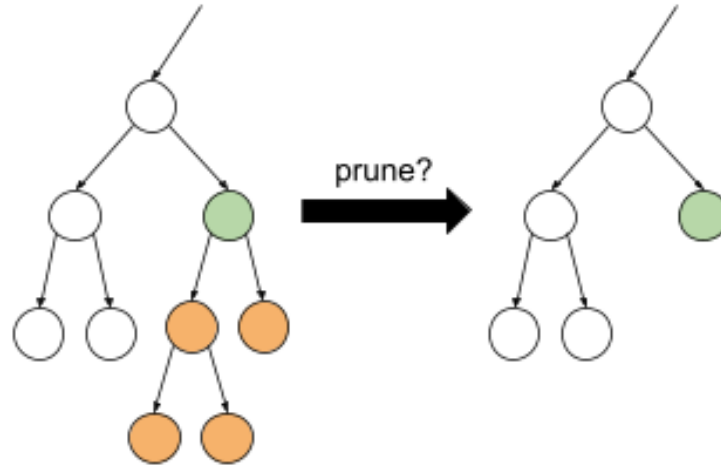


Figura 8.4: In figura è rappresentato un esempio di possibile potatura su un albero decisionale.

Per ottimizzare gli alberi decisionali, è utile utilizzare il metodo di pruning, o di *potatura*, in cui si rimuovono le parti più ridondanti dell'albero. La prima cosa da fare per evitare overfitting è fissare un minimo di data-point che possono trovarsi all'interno di un nodo, in modo da estrarre delle medie ancora accurate, e in questo modo costruiamo un albero T_0 . Il metodo di potatura consiste nel rimuovere dei rami che non contribuiscono molto alla minimizzazione della funzione di costo, come in figura 8.4. In questo modo si ottiene un nuovo albero $T \subset T_0$ "collassando" un numero di nodi interni (o non interni), per la cui funzione costo vale la seguente relazione:

$$C_\alpha(T) = J(j, s) + \alpha|T| = \sum_{m=1}^{|T|} N_m Q_m(T) + \alpha|T|$$

dove $|T|$ è il numero di foglie in T (l'albero ridotto), N_m è il numero di elementi all'interno della regione R_m , $Q_m = \frac{1}{N_m} \sum_{\underline{x}_i \in R_m} (y_i - \hat{c}_m)^2$ è l'impurità associata alla regione R_m e $\hat{c}_m = \frac{1}{N_m} \sum_{\underline{x}_i \in R_m} y_i$. Il problema, anche in questo caso è che bisogna bilanciare bias e varianza, bisogna trovare un compromesso tra il numero di foglie ed il risultato finale. Per $\alpha \rightarrow 0$, abbiamo che $T_\alpha \rightarrow T_0$ (quindi sono più vicina alla condizione iniziale in cui ho troppi nodi), mentre per $\alpha \rightarrow \infty$ porta ad avere un unico nodo. Per calcolare il bilanciamento, dobbiamo calcolare la funzione di costo *estesa* (con l'aggiunta di $\alpha|T|$, che mi regola l'aumento del numero di foglie, che lo limita).

Il processo parte da $T = T_0$ e continua diminuendo T e valutando la funzione di costo (J aumenterà mano a mano, ma il termine $\alpha|T|$ contiene il numero di foglie) fino

a raggiungere $\alpha \rightarrow \infty$. A questo punto creeremo una serie di alberi, tra cui è presente anche quello ottimale, con funzione di costo minima.

8.1.4 Instabilità

Non sempre l'albero decisionale è la scelta migliore. In figura 8.5 è mostrato un caso in cui la regressione lineare può funzionare tranquillamente, mentre l'albero decisionale, essendo sensibile alla rotazione del dataset, peggiora drasticamente nella figura di sinistra.

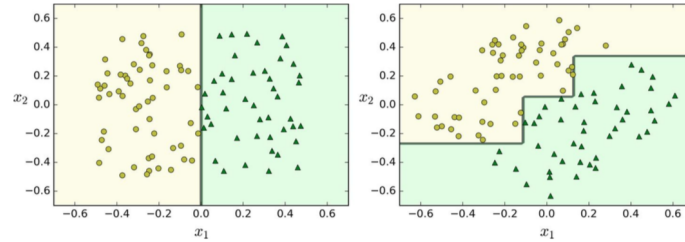


Figura 8.5: In questo caso il dataset potrebbe essere separato tranquillamente da una semplice regressione lineare, un decision tree rischia di funzionare peggio. Nell'immagine di sinistra infatti si osserva come basti ruotare il dataset per rendere molto più dispendioso l'utilizzo di un decision tree

8.2 Random Forests

L'idea che sta alla base dell'approccio *random forest* è che se utilizzo un solo albero decisionale la predizione avrà una certa probabilità di essere errata, se invece utilizzo più alberi decisionali e considero come miglior risultato la media delle predizioni dei singoli risultati migliorerò la mia predizione. Il motivo fondamentale di questo fatto risiede nel teorema del limite centrale (CLT).

Ad esempio, è possibile addestrare un gruppo di classificatori basati su alberi decisionali, ognuno su un diverso sottoinsieme casuale dell'insieme di addestramento. Per effettuare predizioni, è sufficiente ottenere le predizioni di tutti gli alberi individuali, per poi predire la classe che ottiene il maggior numero di voti. Un tale insieme di alberi decisionali è chiamato una Foresta Casuale, e nonostante la sua semplicità, è uno degli algoritmi di Apprendimento Automatico più potenti disponibili oggi.

8.2.1 Voting Classifiers

Nel caso in cui avessimo tanti approcci per fare una regressione (regressione logistica, SVM, ecc...) in cui però nessuno spicca particolarmente, se facciamo una media delle diverse predizioni, in genere il risultato della maggioranza è il migliore, grazie alla legge dei grandi numeri: se abbiamo tante misure di una certa quantità con media μ distribuite secondo una grande varianza, se calcoliamo la media campionata $\hat{\mu}$, la varianza di questo dato varia con il numero di campionamenti come $1/\sqrt{N}$.

In figura Fig. 8.6 è rappresentato un esempio di classificatore costruito utilizzando diversi tipi di predittori e la cui decisione finale sull'output è determinata sulla base del voto dei singoli predittori. Possiamo distinguere almeno due diversi tipi di *voting*

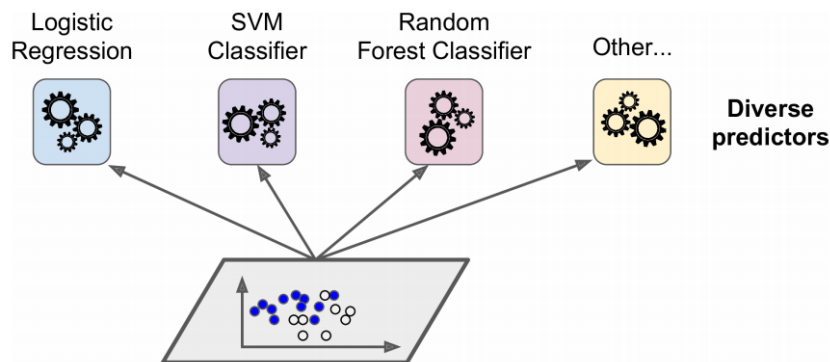


Figura 8.6: Esempio di voting classifiers

classifiers sulla base di come viene presa la decisione finale. Si parla di **hard voting** nel caso in cui la scelta finale coincida con la classe che ha ricevuto il maggior numero di voti per un dato input (vedi ad esempio Fig. 8.7) e di **soft voting** nel caso in cui la predizione finale è calcolata tramite una media di tutte le probabilità e viene dopo di che scelta la classe con probabilità maggiore. In sostanza, nel caso di soft voting viene assegnato più peso al singolo predittore che ha una *confidence* maggiore.

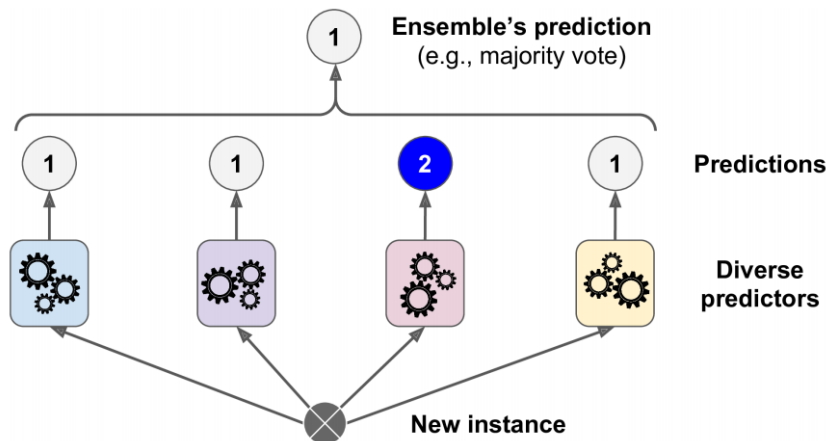


Figura 8.7: Esempio di random forest con protocollo Hard Voting

8.2.2 Bagging and Pasting

Nella sottosezione precedente abbiamo visto come costruire classificatori utilizzando e mediando i risultati di diversi tipi di predittori. Un altro possibile approccio prevede l'utilizzo dello **stesso algoritmo di training** per ogni diptero, ciascuno dei quali viene però allenato su un diverso sottoinsieme casuale del training set originale. Anche in questo caso, sulla base di come vengono scelti i dati di training per i singoli predittori si possono distinguere due tecniche:

- **bagging**: il termine bagging è la contrazione di *bootstrap aggregating*, infatti come per l'algoritmo di bootstrap il campionamento viene effettuato con

sostituzione, quindi i sottoinsiemi di training sono campionati con possibili ripetizioni

- **pasting**: il campionamento degli input per il singolo predittore avviene senza ripetizioni quindi su sottoinsiemi disgiunti del set di training

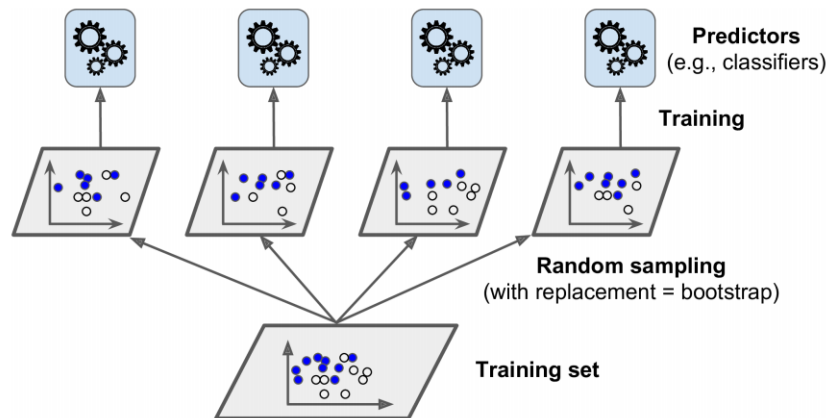


Figura 8.8: Esempio di applicazione della routine di bagging and pasting

Nota: Importanza delle variabili di input La profondità in cui compare una caratteristica in un albero dipende dalla sua importanza (più è profonda, più è importante!). Si può semplicemente calcolare la profondità media per ciascuna caratteristica attraverso tutti i nodi (e alberi) per ottenere una misura appropriata dell'importanza.

Capitolo 9

Support Vector Machines

Una Support Vector Machine (SVM) è un modello di Apprendimento Automatico molto potente e versatile, in grado di effettuare classificazioni lineari o non lineari, regressione e persino rilevamento degli outlier. È uno dei modelli più popolari nell'ambito dell'Apprendimento Automatico. Le Support Vector Machine (SVM) sono particolarmente adatte per la classificazione di dataset complessi ma di dimensioni ridotte o medie.

9.0.1 Condizioni Karush-Kuhn-Tucker

Prima di trattare l'argomento delle SVMs facciamo una breve digressione sulle condizioni di Karush-Kuhn-Tucker (KKT) che utilizzeremo nello studio delle SVMs.

Dato un problema di minimizzazione in cui è richiesto di trovare $\min_{\underline{x} \in \mathbb{R}^n} \{f(\underline{x})\}$ soggetto ai vincoli

$$h_i(\underline{x}) = 0 \quad i = 1, \dots, m \quad \forall \underline{x} \quad (9.1)$$

$$l_j(\underline{x}) \leq 0 \quad j = 1, \dots, r \quad \forall \underline{x} \quad (9.2)$$

Allora si definisce la Lagrangiana associata come segue:

$$L(\underline{x}, \underline{u}, \underline{v}) = f(\underline{x}) + \sum_{i=1}^m u_i h_i(\underline{x}) + \sum_{j=1}^r v_j l_j(\underline{x}) \quad (9.3)$$

e la funzione duale di Lagrange

$$g(\underline{u}, \underline{v}) = \min_{\underline{x} \in \mathbb{R}^n} L(\underline{x}, \underline{u}, \underline{v}). \quad (9.4)$$

Il problema duale che si ottiene è quindi

$$\max_{\underline{u} \in \mathbb{R}^m, \underline{v} \in \mathbb{R}^r} \{g(\underline{u}, \underline{v})\} \quad \text{soggetto al vincolo } v \geq 0 \quad (9.5)$$

Si ha allora che il problema è sempre convesso, ovvero $g(\cdot)$ è sempre concava, anche se $f(\cdot)$ non lo è. Di conseguenza si ha

$$f^* \geq g^* \quad (9.6)$$

dove f^* e g^* sono i due valori ottimali (rispettivamente minimo e massimo). **Nota:** $f(\underline{x}) - f^* \leq f(\underline{x}) - g(\underline{u}, \underline{v})$, quindi se il gap tra le due funzioni tende a 0 si ha che $f(\underline{x}) \rightarrow f^*$.

In sostanza, il teorema delle condizioni necessarie e sufficienti di KKT afferma che:

Teorema KKT

1. Condizione sufficiente: Se esiste una soluzione x^* al problema primario, una soluzione (μ^*, λ^*) al problema duale, tale che insieme soddisfano le condizioni KKT, allora la coppia di problemi ha una dualità forte, e $x^*, (\mu^*, \lambda^*)$ è una soluzione per il problema primario e per il problema duale.
2. Condizione necessaria: Se la coppia di problemi ha una dualità forte, allora per ogni soluzione x^* per il problema primario, e ogni soluzione (μ^*, λ^*) al problema duale, la coppia $x^*, (\mu^*, \lambda^*)$ deve soddisfare le condizioni KKT.

Le KKT generalizzano il metodo dei moltiplicatori di Lagrange a ottimizzazioni vincolate in cui il vincolo è definito con una disuguaglianza. Le condizioni KKT sono le seguenti:

1. **stazionarietà:**

$$\partial f(\underline{x}) + \sum_{i=1}^m u_i \partial h_i(\underline{x}) + \sum_{j=1}^r v_j \partial l_j(\underline{x}) \Big|_{\underline{x}=\underline{x}^*} = 0 \quad (9.7)$$

dove ∂ indica la frontiera

2. **fattibilità del problema primario:**

$$h_i(\underline{x}^*) = 0 \quad i = 1, \dots, m \quad (9.8)$$

$$l_j(\underline{x}^*) \leq 0 \quad j = 1, \dots, r \quad (9.9)$$

3. **fattibilità del problema duale:** $v_i \geq 0 \quad i = 1, \dots, r$

4. **complementarietà:** $\sum_{j=1}^r v_j l_j(\underline{x}^*) = 0$

Nota: La richiesta delle condizioni di fattibilità primaria e fattibilità duale su $l_j(\underline{x})$ e v_i in sostanza richiedono che $v_j l_j(\underline{x}^*) = 0 \quad \forall j = 1, \dots, r$.

9.1 Support Vector Machines

Si consideri un vettore $\underline{\beta}$ con vettore unitario $\hat{\underline{\beta}} \equiv \frac{\underline{\beta}}{\|\underline{\beta}\|}$, con $\underline{\beta}, \hat{\underline{\beta}} \in \mathbb{R}^p$. L'iperpiano ortonormale a $\underline{\beta}$ e passante per un punto $\underline{x}_0$ è dato da

$$S = \{\underline{x} \in \mathbb{R}^p : \underline{x}^T \hat{\underline{\beta}} + \beta_0 = 0\} \quad (9.10)$$

con $\beta_0 \equiv -\underline{x}_0^T \hat{\underline{\beta}}$. Dato che $\|\hat{\underline{\beta}}\| = 1$, la distanza *con segno* di qualsiasi punto $\underline{x}$ da tale iperpiano è

$$f(\underline{x}) = \underline{x}^T \hat{\underline{\beta}} + \beta_0 \quad (9.11)$$

e $f(\underline{x})$ è maggiore o minore di 0 a seconda che $\underline{x}$ si trovi da un lato o dall'altro dell'iperpiano.

Si consideri allora un problema di classificazione binaria con un dataset

$$\mathcal{D} = \{(y_i, \underline{x}_i) : \underline{x}_i \in \chi \subset \mathbb{R}^p, y_i \in \{-1, +1\}, i = 1, \dots, N\}$$

, se esiste un iperpiano che separa perfettamente le due classi, allora può essere scelto in maniera tale che

$$y_i f(\underline{x}_i) > 0 \quad \forall i = 1, \dots, N \quad (9.12)$$

Il problema della SVM consiste allora nel trovare tale piano che abbia il più grande margine M possibile, ovvero:

$$\hat{\underline{\beta}}, \beta_0 = \arg \max_{\underline{\beta}, \beta_0} \{M\} \quad (9.13)$$

soggetto al vincolo

$$y_i(\underline{x}_i^T \hat{\underline{\beta}} + \beta_0) \geq M \quad (9.14)$$

9.1.1 Caviats

Per prima cosa osserviamo sempre è possibile realizzare una separazione esatta, per questo motivo consentiamo una rottura delle frontiere in alcuni punti ξ_i con $i = 1, \dots, N$:

$$y_i(\underline{x}_i^T \hat{\underline{\beta}} + \beta_0) \geq M - \xi_i \quad \forall i = 1, \dots, N \quad (9.15)$$

oppure

$$y_i(\underline{x}_i^T \hat{\underline{\beta}} + \beta_0) \geq M(1 - \xi_i) \quad \forall i = 1, \dots, N \quad (9.16)$$

Queste due scelte implementano il medesimo concetto ma *non sono equivalenti*, hanno soluzioni differenti. Il secondo caso è quello che funziona: ha un problema duale convesso quindi è più semplice da risolvere. Bisogna però limitare "l'invasione" complessiva, quindi imponiamo che $\sum_{i=1}^n \xi_i$ sia inferiore o uguale a una qualche costante che deve essere fissata.

In secondo luogo, osserviamo che invece di richiedere $\|\hat{\underline{\beta}}\| = 1$ si può rilassare la richiesta di norma unitaria ed utilizzare $\|\underline{\beta}\|$ per definire M , semplicemente scegliendo $M = \frac{1}{\|\underline{\beta}\|}$. Infatti

$$\max_{\underline{\beta}, \beta_0, \|\underline{\beta}\|=1} M \quad (9.17)$$

soggetto al vincolo $y_i(\underline{x}_i^T \underline{\beta} + \beta_0) \geq M$ è equivalente a

$$\min_{\underline{\beta}, \beta_0, \|\underline{\beta}\|} \quad (9.18)$$

soggetto al vincolo $y_i(\underline{x}_i^T \underline{\beta} + \beta_0) \geq 1$

Di conseguenza, il problema della SVM è la ricerca di $\min_{\underline{\beta}, \beta_0} \|\underline{\beta}\|$ soggetto ai seguenti vincoli:

$$y_i(\underline{x}_i^T \underline{\beta} + \beta_0) \geq 1 - \xi_i \quad \forall i = 1, \dots, N \quad \xi_i \geq 0 \quad \forall i = 1, \dots, N \quad \sum_{i=1}^n \xi_i \leq \text{const} \quad (9.19)$$

O equivalentemente alla minimizzazione di

$$\underline{\beta}, \beta_0 = \arg \min_{\underline{\beta}, \beta_0} \left\{ \frac{1}{2} \|\underline{\beta}\|^2 + C \sum_{i=1}^N \xi_i \right\} \quad (9.20)$$

sotto le condizioni

$$y_i(\underline{x}_i^T \underline{\beta} + \beta_0) \geq (1 - \xi_i) \quad \xi_i \geq 0 \quad (9.21)$$

Applicazione del teorema KKT Questo problema si risolve utilizzando le KKT. La lagrangiana è

$$\mathcal{L}(\underline{\beta}, \beta_0, \underline{\alpha}, \underline{\mu}) = \frac{1}{2} \|\underline{\beta}\|^2 + C \sum_{i=1}^N \xi_i - \sum_i \alpha_i \left[y_i(\underline{x}_i^T \underline{\beta} + \beta_0) - (1 - \xi_i) \right] - \sum_i \mu_i \xi_i \quad (9.22)$$

Le KKT sono:

1. stazionarietà:

$$\frac{\partial \mathcal{L}}{\partial \underline{\beta}} = \underline{\beta} - \sum_i \alpha_i y_i \underline{x}_i = 0 \quad (9.23)$$

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = - \sum_i \alpha_i y_i = 0 \quad (9.24)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \mu_i - \alpha_i = 0 \quad (9.25)$$

2. primary feasibility:

$$y_i(\underline{x}_i^T \underline{\beta} + \beta_0) \geq (1 - \xi_i) \quad (9.26)$$

$$\xi_i \geq 0 \quad (9.27)$$

3. dual feasibility:

$$\mu_i \geq 0 \quad (9.28)$$

$$\alpha_i \geq 0 \quad (9.29)$$

4. Complementary slackness:

$$[y_i(\underline{x}_i^T \underline{\beta} + \beta_0) - 1 + \xi_i] \alpha_i = 0 \quad (9.30)$$

$$\text{con } \xi_i \mu_i = 0 \quad \forall i = 1, \dots, N$$

Punti che non violano il margine

Per i punti che non violano il margine e sono fuori dall'iperpiano che costituisce il margine si ha

$$y_i(\underline{x}_i^T \underline{\beta} + \beta_0) > 1 \Rightarrow [y_i(\underline{x}_i^T \underline{\beta} + \beta_0) - 1 + \xi_i] > 0 \quad \forall \xi_i \geq 0, \quad (9.31)$$

di conseguenza dalla (9.30) si ottiene $\alpha_i = 0$, dalla (9.23) $\mu_i = C$ e quindi, sempre dalla (9.30) si ottiene $\xi_i = 0$

Punti che violano il margine

Per tutti i punti che violano il margine si ha che

$$\exists \xi_i > 0 : y_i(\underline{x}_i^T \underline{\beta} + \beta_0) = 1 - \xi_i, \quad (9.32)$$

di conseguenza, dalla (9.30) si ottiene $\mu_i = 0$ e dalla (9.23) si ottiene $\alpha_i = C$

Punti esattamente sul margine

Per i punti che si trovano esattamente sul margine si ha $y_i(\underline{x}_i^T \underline{\beta} + \beta_0) = 1$ da cui segue

$$\xi_i = 0 \quad (9.33)$$

$$0 \leq \alpha_i \leq C \quad \mu_i = C - \alpha_i \geq 0 \quad (9.34)$$

La condizione di stazionarietà quindi implica

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad (9.35)$$

$$\alpha_i = C - \mu_i \quad (9.36)$$

$$\underline{\beta}^* = \sum_{i=1}^N \alpha_i y_i \underline{x}_i \quad (9.37)$$

Si può quindi calcolare la lagrangiana duale sostituendo questi risultati:

$$\begin{aligned} \mathcal{L}_D(\alpha_i, \mu_i = C - \alpha_i) &= \mathcal{L}_D(\{\alpha_i\}) = \\ \mathcal{L}_D(\underline{\beta} = \underline{\beta}^*, \beta_0 = \beta_0^*, \xi_i, \alpha_i, \mu_i) &= \\ \sum_i \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \underline{x}_i^T \underline{x}_j + \sum_{i=1}^N (C - \mu_i - \alpha_i) \xi_i + \sum_i \alpha_i + \\ &\quad - \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \underline{x}_i^T \underline{x}_j = \\ \sum_i \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \underline{x}_i^T \underline{x}_j \end{aligned}$$

Conclusioni

Si nota che

1. $\alpha_i = C$ per i vettori di supporto dei punti che si trovano all'interno del margine ($\xi_i = 0$)
2. $0 \leq \alpha_i \leq C$ per i vettori di supporto dei punti che si trovano esattamente sul margine ($\xi_i = 0$)
3. $\beta_0^* = 1 - \underline{x}_i^T \underline{\beta}^* = 1 - \sum_{j=1}^N \alpha_j y_j \underline{x}_i^T \underline{x}_j \quad \forall i$, con $\xi_i = 0, \alpha_i = 0$. Quindi β_0^* può essere stimato da uno qualunque dei vettori di supporto che si trovano sul margine.
4. $x_i = 1 - \underline{x}_j^T \underline{\beta}^* - \beta_0^* = 1 - \beta_0^* - \sum_{l=1}^N \alpha_l y_l \underline{x}_j^T \underline{x}_l$ per i vettori di supporto che violano il margine

Quindi gli α_i si trovano massimizzando

$$\mathcal{L}_D(\{\alpha_i\}) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \underline{x}_i^T \underline{x}_j \quad (9.38)$$

rispetto a tutti gli $\{\alpha_i\}$

9.2 Kernels di Mercer e SVMs

Le SVMs sono allenate massimizzando la

$$\mathcal{L}_D(\{\alpha_i\}) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \underline{x}_i^T \underline{x}_j \quad (9.39)$$

rispetto a $\{\alpha_i\}$. Le predizioni sono fatte da

$$\hat{y}(\underline{x}) = \text{sign}(f(\underline{x})) = \text{sign}(\underline{x}^T \underline{\beta} + \beta_0), \quad (9.40)$$

la 9.40 implementa una separazione dei dati tramite un iperpiano lineare.

Per affrontare, come può capitare, problemi non lineari con questa tecnica abbiamo bisogno di *features generalizzate* $\vec{h}(\underline{x})$ con

$$\vec{h} : \chi \longrightarrow \mathbb{R}^P \quad \text{dove } \underline{x} \in \chi \subset \mathbb{R}^P$$

e generalmente $p \leq P$; quindi $\vec{h}(\underline{x})$ è il vettore P -dimensionale delle features generalizzate.

A questo punto il training della SVM diventa:

$$\mathcal{L}_D = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \vec{h}(\underline{x}_i)^T \vec{h}(\underline{x}_j). \quad (9.41)$$

Dai parametri $\vec{\beta} \in \mathbb{R}^P$ e β_0 si ottengono

$$\vec{\beta}^* = \sum_{i=1}^N \alpha_i y_i \vec{h}(\underline{x}_i) \quad (9.42)$$

$$\beta_0^* = 1 - \sum_{j=1}^N \alpha_j y_j \vec{h}(\underline{x}_i)^T \vec{h}(\underline{x}_j), \quad (9.43)$$

dove gli indici i corrispondono a ciascuno degli SV, che si trovano sul bordo (quindi con $\alpha_i > 0$, $\xi_i = 0$). Le predizioni con le features generalizzate sono date da

$$\hat{y}(\underline{x}) = \text{sign}(f(\underline{x})) \quad \text{dove} \quad (9.44)$$

$$f(\underline{x}) = \vec{h}(\underline{x}) \vec{\beta}^* + \beta_0^* = \sum_{j=1}^N \alpha_j y_j \vec{h}(\underline{x})^T \vec{h}(\underline{x}_j) + \beta_0^*. \quad (9.45)$$

Dato che $\mathcal{L}_D(\{\alpha_i\})$, β_0^* e $f(\underline{x})$ sono scritte solamente in termini di prodotti scalari di vettori di caratteristiche $(\underline{x}_i^T \underline{x}_j \vec{h}(\underline{x}_i)^T \vec{h}(\underline{x}_j))$, si possono sfruttare i kernel e sostituire

$$\vec{h}(\underline{x}_i)^T \vec{h}(\underline{x}_j) \longrightarrow K(\underline{x}_i, \underline{x}_j). \quad (9.46)$$

Di conseguenza

Dove i Kernel più comunemente utilizzati sono

$$K(\underline{x}, \underline{x}') = (1 + \underline{x}^T \underline{x}')^d \quad \text{Polinomials up to order 'd'} \quad (9.47)$$

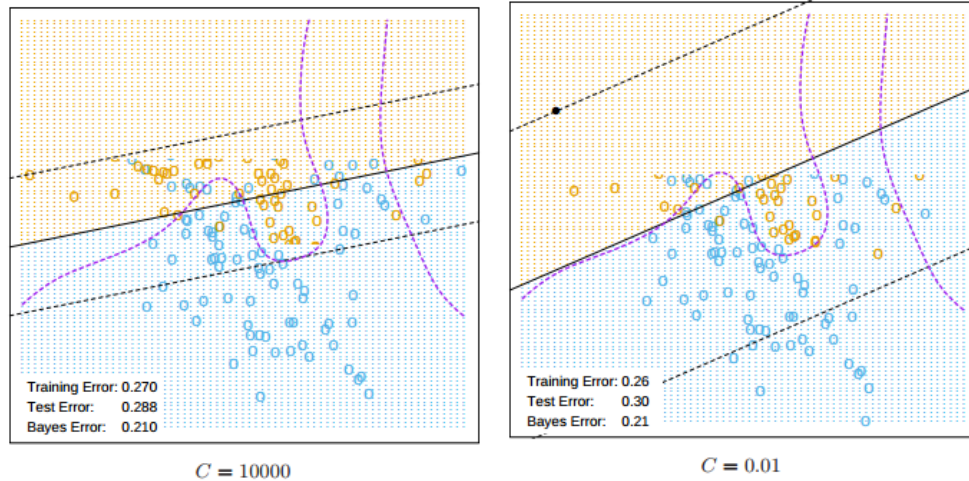
$$K(\underline{x}, \underline{x}') = \exp(-\gamma \|\underline{x} - \underline{x}'\|) \quad \text{Radial Gaussian Kernel} \quad (9.48)$$

$$K(\underline{x}, \underline{x}') = \tanh k_1 \underline{x}^T \underline{x}' + k_2 \quad \text{Hyperbolic Tangent Kernel} \quad (9.49)$$

dove si noti che l'ultimo kernel in realtà non è un kernel di Mercer.

9.3 Esempio di utilizzo di SVMs

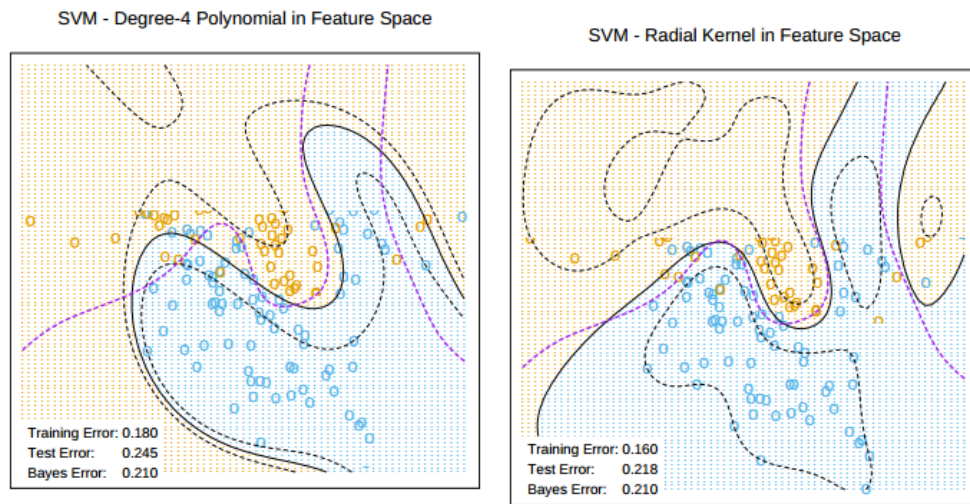
In questa sezione sono presentati alcuni esempio di utilizzo delle SVMs e confrontate con l'utilizzo di altre tecniche.



(a) Esempio di SVM con parametro di costo $C = 10000$ (b) Esempio di SVM con parametro di costo $C = 0,01$

Figura 9.1: La frontiera lineare del vettore di supporto per l'esempio di dati misti con due classi sovrapposte, per due diversi valori di C . Le linee tratteggiate indicano i margini, dove $(f(x) = \pm 1)$. I punti di supporto (ovvero quelli con $\alpha_i > 0$) sono tutti i punti sul lato sbagliato del loro margine. I punti neri sono quelli che cadono esattamente sul margine ($\xi_i = 0$), ($\alpha_i > 0$). Nell'immagine 9.1a, il 62% dei data points risultano punti di supporto, mentre nell'immagine 9.1b costituiscono l'85%. La curva viola tratteggiata sullo sfondo è la frontiera decisionale di Bayes, ovvero del predittore ottimale.

La Figura 9.1 mostra la linea di separazione del vettore di supporto per un esempio di miscela di dati con due classi sovrapposte, per due diversi valori del parametro di costo C . I due classificatori sono piuttosto simili nelle loro prestazioni. I punti sul lato sbagliato del confine sono vettori di supporto. Inoltre, anche i punti sul lato corretto della linea di separazione ma vicini ad essa (ovvero all'interno del margine) sono anch'essi vettori di supporto. Il margine è più ampio per $C = 0,01$ (vedi Fig. 9.1a) rispetto a $C = 10000$ (vedi Fig 9.1b). Pertanto, valori più elevati di C concentrano l'attenzione sui punti (correttamente classificati) vicino al confine decisionale, mentre valori più piccoli coinvolgono dati più lontani. In entrambi i casi, i punti classificati male vengono ponderati, indipendentemente dalla distanza. In questo esempio, la procedura di classificazione non è molto sensibile alle scelte di C , a causa della rigidità di un confine lineare. Il valore ottimale per C può essere stimato mediante la cross-validazione.



(a) Classificazione con SVM con Kernel polinomiale di grado 4

(b) Classificazione con SVM con GR Kernel

Figura 9.2: Due SVM non lineari per dati che risultano mischiati nello spazio delle caratteristiche. Il grafico in figura 9.2a utilizza un kernel polinomiale di quarto grado, mentre quello in figura 9.2b utilizza un kernel a base radiale (GRK) con $\gamma = 1$. In entrambi i casi, C è stato regolato per ottenere approssimativamente la migliore prestazione in termini di errore di test, e $C = 1$ ha funzionato bene in entrambi i casi. Il kernel a base radiale ottiene la migliore prestazione (vicino all'ottimo di Bayes), come ci si potrebbe aspettare dato che i dati derivano da miscele di gaussiane. La curva viola tratteggiata sullo sfondo è la frontiera decisionale di Bayes.

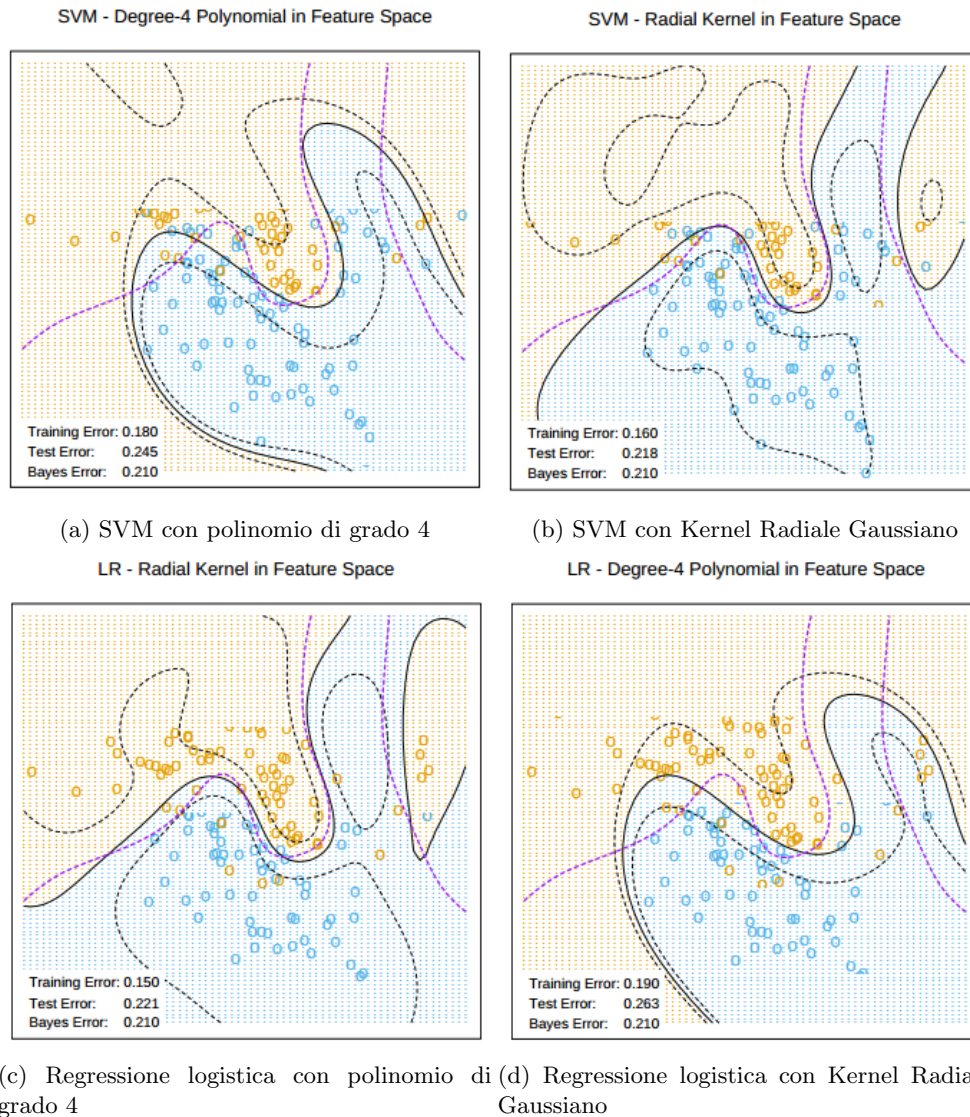


Figura 9.3: Nelle immagini 9.3c e 9.3d sono presentate due versioni di regressione logistica, utilizzando gli stessi Kernel e di conseguenza le stesse penalità che sono state utilizzate nella SVM, ma con come loss-function la log-likelihood invece della funzione di perdita definita dalla SVM. Le due curve tratteggiate corrispondono a probabilità posteriori del 0.75 e 0.25 per la classe +1 (o viceversa). La curva viola tratteggiata sullo sfondo invece è la frontiera decisionale di Bayes.